Faculty of Computers and Artificial Intelligence,
Beni-Suef National University, Egypt
Artificial Intelligence Department

AD GENERATOR AI

Submitted By

Youssef Abdelnasser Ahmed

Mennatullah Ahmed Amer Ali

Manal Ramadan Ibrahim

Farah Mohsen Mohamed

Rawda Mohamed Ahmed

Mennatullah Ibrahim Abdelhamed

Supervised By

Dr.Amal Fouad

A.T:Ehab Ibrahim

Submitted for the partial fulfillment of Bachelor of Computer Science degree.

JULY 2026

Faculty of Computers and Artificial Intelligence,
Beni-Suef National University, Egypt
Artificial Intelligence Department

Ad Generaotor AI

Submitted By

Youssef Abdelnasser Ahmed

Mennatullah Ahmed Amer Ali

Manal Ramadan Ibrahim

Farah Mohsen Mohamed

Rawda Mohamed Ahmed

Mennatullah Ibrahim Abdelhamed

Supervised By

Dr.Amal Fouad

A.T:Ehab Ibrahim

Submitted for the partial fulfillment of Bachelor of Computer Science degree.

JULY 2026

ABSTRACT

With the rapid growth of Artificial Intelligence and the increasing demand for digital marketing solutions, businesses and content creators face challenges in creating effective marketing campaigns that attract audiences and increase engagement across multiple platforms. This graduation project presents **AD GENERATOR AI**, an intelligent mobile-based marketing system designed to generate AI-powered advertising campaigns for different digital platforms, including Facebook, Instagram, TikTok, and Websites/Ads.

The system allows users to create personalized marketing campaigns by entering campaign-related information such as campaign goals, target platforms, product details, target audience preferences, creative style, emotions, occasions, and call-to-action preferences. In addition, the application enables users to create a single marketing campaign for more than one product within the same campaign, allowing businesses to promote multiple products together in an organized and efficient way. This feature helps users manage product collections, seasonal offers, bundles, and multi-product promotions without creating separate campaigns for each product.

Based on these inputs, the system automatically generates complete advertising campaigns containing three different post variations: **Product-Focused Posts, Campaign-Focused Posts, and Lifestyle-Focused Posts.** Each generated post includes AI-generated captions, hashtags tailored to the selected platform, and promotional images to enhance marketing effectiveness.

To improve campaign performance and user experience, the application provides smart hashtag generation according to platform requirements, where Instagram campaigns generate up to 20 hashtags, Facebook generates 6 hashtags, while TikTok and Websites/Ads generate 5 hashtags. Additionally, the system recommends optimal posting schedules for each platform to maximize audience reach and engagement.

The project also integrates an AI Marketing Chatbot Assistant that enables users to ask marketing-related questions, receive business and sales advice, generate content ideas, and improve advertising strategies using conversational AI. Furthermore, the application includes campaign management and

library features that allow users to save, manage, and review previously generated campaigns, including campaigns that contain single or multiple products.

The mobile application is designed with a modern and user-friendly interface to ensure simplicity, accessibility, and efficient navigation. The project aims to provide a smart, automated, and time-saving marketing solution that helps businesses and individuals create professional digital campaigns using Artificial Intelligence technologies.

Acknowledgement

First and foremost, we would like to express our deepest gratitude to **Allah Almighty**, whose blessings, guidance, and mercy gave us the strength, patience, and determination to complete this graduation project successfully. Without His grace, this achievement would not have been possible.

We would also like to extend our sincere appreciation to the **Faculty of Computers and Artificial Intelligence** for providing us with an outstanding academic environment, valuable learning opportunities, and the resources that helped us build the knowledge and skills required to accomplish this project. The education and support we received throughout our academic journey laid the foundation for the successful completion of this work.

Our heartfelt thanks and deepest appreciation go to **Dr. Amal Fouad**, our project supervisor, for her exceptional guidance, continuous support, and invaluable academic advice throughout the development of this project. Her insightful feedback, constructive suggestions, and constant encouragement inspired us to think creatively, overcome challenges, and maintain high standards of quality during every stage of the project.

We would also like to express our sincere gratitude to **Eng. Ehab Ebrahim**, our teaching assistant, for his remarkable technical support, patience, continuous follow-up, and valuable recommendations during the implementation phase. His expertise and willingness to help greatly contributed to improving the quality of our work and overcoming many technical difficulties.

Our graduation project, **AD GENERATOR AI**, aims to develop an intelligent AI-powered platform capable of generating professional marketing campaigns for various social media platforms such as Instagram, Facebook, and TikTok. The system generates AI-powered posts, captions, hashtags, images, and publishing schedules while also providing users with an AI chatbot assistant for marketing guidance and support. Additionally, the platform enables users to create a single marketing campaign for multiple products, allowing businesses to promote product collections, bundles, seasonal offers, or related products within one organized campaign instead of creating separate campaigns for each product. Achieving this vision would not have been possible without the continuous guidance and encouragement of our supervisors.

Finally, we would like to express our deepest gratitude to our families for their unconditional love, endless encouragement, patience, and unwavering support throughout our academic journey and during the completion of this project. Their belief in our abilities motivated us to overcome every

challenge and achieve this milestone. We are also grateful to our friends and colleagues whose encouragement and support made this journey more meaningful.

To everyone who contributed, directly or indirectly, to the success of this project, we extend our sincere appreciation. Your support and kindness will always be remembered.

TABLE OF CONTENTS

Chapter **Page**

CHAPTER 1.....14

1.0 Introduction.....16

1.1 Problem domain.....16

1.2 Problem statement.....17

1.3 Proposed System.....19

1.4 Project Methodology.....23

1.5 Resource Requirements.....25

1.6 Report Layout.....28

CHAPTER 2.....30

2.0 Introduction.....32

2.1 Overview of existing projects.....32

2.2 Related Research Papers.....33

2.3 Limitations of Existing Projects.....36

2.4 Innovations of Our Project.....38

2.5 Comparative Analysis.....40

2.6 Methodology Selection.....42

2.7 Justification.....42

CHAPTER 3.....43

3.0 Introduction.....45

3.1 Intended Audience and Reading Suggestions.....46

3.2 Product Scope.....50

3.3 Overall Description 51
3.4 Product Functions 53
3.5 User Classes and Characteristics 55
3.6 Operating Environment 57
3.7 Design and Implementation Constraints 59
3.8 User Documentation 61
3.9 External Interface Requirements 62
3.10 System Features 63
3.11 Analysis Phase 72
CHAPTER 4 134
4.0 Application Logo 136
4.1 Onboarding Screen – Welcome Screen 137
4.2 Onboarding Screen – Dashboard Overview 138
4.3 Onboarding Screen – Manage & Customize 139
4.4 Sign Up Interface 140
4.5 Sing in Interface 141
4.6 Forgot Password Interface 142
4.7 Reset Password Interface 143
4.8 Dashboard Overview 144
4.9 Campaign Library 145
4.10Create Campaign – Basic Information 146
4.11 Create Campaign – Audience Selection 147
4.12 Create Campaign – Product Information 148
4.13 Create Campaign – Creative Requirements 149
4.14 Create Campaign – Brand Assets Upload 150

4.15 AI Generated Campaign Details 152
4.16 Campaign Variants and Generated Content 153
4.17 AI Marketing Assistant 155
4.18 Profile Settings 156
4.19 User Interface Prototype 157
4.20 Project Banner 158

CHAPTER 5 159
5.1 Introduction 160
5.2 Frontend Implementation 161
5.3 Database Implementation 165
5.4 Backend Implementation 176
5.5 APIs Implementation 188
5.6 AI Implementation— Text and Hashtag Generation 192
5.7 AI Implementation — AI Advertising Image Generation Module 211
5.8 AI Implementation— AI Marketing Chatbot Module 218
CHAPTER 6 224
6.1 Introduction 225
6.2 Functional Testing 225
6.3 API Testing 227
6.4 AI Output Quality Evaluation 228
6.5 Security Testing 230
6.6 Performance Testing 231
6.7 Usability Testing 231

6.8 Integration Testing 231

6.9 Database Integrity Testing 232

CHAPTER 7 234

7.1Conclusion 235

7.2 Limitations 236

7.3Future Work 237

CHAPTER 8 240

A.1 T5 Model Training Setup and Execution 241

A.2 Dataset Structure 243

A.3 Generated Output Variants and Evaluation 244

A.4 API Structured Output and Platform Configuration 245

A.5 Mobile Application Implementation (Flutter) 246

A.6 Database Schema (SQL Server) 248

A.7 Retrieval-Augmented Generation (RAG) Pipeline 252

A.8 LoRA Image Generation Training Script 253

References 252

LIST OF FIGURES

Figure 1 UseCase Diagram 116
Figure 2 Class Diagram 120
Figure 3 Sequence Diagram 124
Figure 4 Activity Diagram 130
Figure 5 ERD 133
Figure 6 Application Logo 136
Figure 7 Onboarding Screen – Welcome Screen 137
Figure 8 Onboarding Screen – Dashboard Overview 138
Figure 9 Onboarding Screen – Manage & Customize 139
Figure 10 Sign Up Interface 140
Figure 11 Sign In Interface 141
Figure 12 Forgot Password Interface 142
Figure 13 Reset Password Interface 143
Figure 14 Dashboard Overview 144
Figure 15 Campaign Library 145
Figure 16 Campaign Creation – Basic Information 146
Figure 17 Campaign Creation – Audience Selection 147
Figure 18 Campaign Creation – Product Information 148
Figure 19 Campaign Creation – Creative Requirements 149
Figure 20 Campaign Creation – Brand Assets Upload 150
Figure 21 Campaign Summary Screen 151
Figure 22 AI Generated Campaign Details 152
Figure 23 AI Marketing Assistant 155
Figure 24 Profile Settings 156
Figure 25 User Interface Prototype 157
Figure 26 Project Banner 158
Figure 27 Database Schema Mapping 174
Figure 28 Inference 197

Figure 29 Overfitting Analysis 207
Figure 30 Human Evaluation Framework 208
Figure 31 Dataset — AdImageNet 211
Figure 32 Results 214
Figure 33 T5 Model Training Setup and Execution 241
Figure 34 Dataset Structure 243
Figure 33 Generated Output Variants and Evaluation 244
Figure 34 API Structured Output and Platform Configuration 245
Figure 35 Mobile Application Implementation (Flutter) 246
Figure 36 Database Schema (SQL Server) 248
Figure 37 Retrieval-Augmented Generation (RAG) Pipeline 252
Figure 38 LoRA Image Generation Training Script 253

CHAPTER 1

INTRODUCTION

CHAPTER 1

INTRODUCTION

In this chapter an introduction about the project should be provided. Problem statement, Objectives, details of resources required for the completion, development methodology and organization of the project report.

1.0 Introduction

1.1 Problem domain

1.2 Problem statement

1.3 Proposed system

1.4 Project Methodology

1.5 Resource Requirement

1.6 Report Layout

1.0 Introduction

The rapid growth of Artificial Intelligence and digital marketing technologies has significantly changed the way businesses promote products and communicate with customers. Social media platforms and online advertising have become essential tools for business growth, especially for small business owners who depend on digital platforms to increase brand awareness, attract customers, and improve sales performance.

Despite the availability of many marketing platforms, creating professional marketing campaigns still requires creativity, marketing knowledge, content writing skills, and continuous analysis of audience behavior and platform trends. Many small business owners face difficulties in creating effective advertisements, writing attractive captions, selecting suitable hashtags, and choosing the right marketing style for different platforms such as Facebook, Instagram, TikTok, and Websites/Ads.

In addition, managing marketing campaigns manually can consume a significant amount of time and effort, especially for users who do not have sufficient experience in digital marketing. Many existing tools focus only on limited content generation features and do not provide integrated AI-powered assistance that helps users improve their marketing strategies and business performance.

To solve these challenges, **AD GENERATOR AI** was developed as an intelligent mobile-based application that combines AI-powered campaign generation with an AI Marketing Chatbot Assistant. The system allows users to create personalized advertising campaigns by entering campaign details such as product information, campaign goals, platform selection, occasion, style, emotion, and call-to-action preferences.

The system enables users to create personalized marketing campaigns by providing campaign details such as product information, campaign goals, target platforms, language, style, and audience preferences. Based on these inputs, AI generates complete marketing content including promotional posts, captions, hashtags, images, and posting schedule recommendations. The platform also integrates an AI marketing chatbot to provide instant marketing guidance and supports both single-product and multi-product campaigns. Overall, the project aims to simplify digital marketing by providing an intelligent, automated, and user-friendly solution that saves time, effort, and marketing costs.

1.1 Problem domain

The rapid expansion of digital marketing and social media platforms has created significant opportunities for businesses to promote their products and services online. Small business owners and content creators increasingly rely on platforms such as Facebook, Instagram, TikTok, and Websites/Ads

to reach customers, improve engagement, and increase sales. However, creating professional and effective marketing campaigns remains a major challenge for many users.

Many small businesses struggle with producing attractive marketing content due to limited marketing experience, lack of creative resources, and the high cost of professional advertising services. Creating marketing campaigns requires writing engaging captions, selecting suitable hashtags, generating promotional images, and adapting content to different platforms and target audiences. This process can be difficult, repetitive, and time-consuming.

Moreover, many small business owners need to promote more than one product within the same marketing campaign, such as product collections, bundles, seasonal offers, or related items. Creating separate campaigns for each product can increase the time, effort, and complexity of campaign management. Therefore, users need a solution that allows them to organize and generate marketing content for multiple products within a single campaign in a simple and efficient way.

In addition, each platform requires different marketing styles, content formats, and hashtag strategies. Users often face difficulties choosing the appropriate campaign tone, emotional style, occasion, and call-to-action that best fit their products and audience preferences. Existing tools usually focus on limited features and do not provide an integrated AI-powered solution that combines campaign generation with intelligent marketing assistance.

Another important challenge is the lack of accessible marketing guidance for small business owners who need support in improving their advertising strategies, increasing customer engagement, and understanding digital marketing concepts. Many users need a simple and intelligent assistant that can answer marketing-related questions and provide instant recommendations.

The problem domain addressed in this project focuses on simplifying the digital marketing process for small business owners and content creators through an AI-powered platform that automatically generates professional marketing campaigns and provides AI-based marketing assistance. The proposed system aims to reduce effort, save time, improve marketing quality, and make digital marketing tools more accessible and effective for all users.

1.2 Problem statement

With the rapid growth of digital marketing and online business platforms, small business owners and content creators increasingly depend on social media and online advertising to promote their products and services. Despite the availability of many digital marketing tools, creating professional and effective

marketing campaigns remains a difficult and time-consuming task for many users, especially those with limited marketing knowledge and creative experience.

One of the major challenges in digital marketing is generating attractive and platform-specific content that matches audience interests and increases engagement. Each platform such as Facebook, Instagram, TikTok, and Websites/Ads requires different content styles, hashtag strategies, promotional approaches, and posting schedules. Many users struggle to create suitable captions, select effective hashtags, and design engaging promotional content manually.

Moreover, many users need to promote multiple products within the same marketing campaign, such as product collections, bundles, seasonal offers, or related products. Creating separate campaigns for each product increases the time, effort, and complexity of campaign management, especially for small business owners with limited marketing resources. Therefore, there is a need for a system that can support multi-product campaign creation in a single organized campaign.

Another significant issue is the lack of intelligent marketing guidance for small business owners who need support in improving their advertising strategies, increasing sales, and understanding marketing concepts. Existing tools often focus only on limited content generation features and do not provide integrated AI assistance that combines campaign generation with marketing consultation in a single platform.

In addition, managing campaigns manually requires considerable time and effort, which can reduce productivity and marketing consistency. Users may also face difficulties choosing the appropriate campaign tone, emotion, occasion, style, and call-to-action that best fit their products and target audiences.

To overcome these challenges, there is a growing need for an intelligent AI-powered marketing platform that simplifies campaign creation, automates content generation, and provides users with smart marketing assistance. Such a system should help users generate professional campaigns efficiently, save time and effort, and improve overall marketing performance through AI-driven solutions.

1.2.1 Motivation

The motivation behind developing **AD GENERATOR AI** stems from the growing demand for intelligent and accessible digital marketing solutions that simplify campaign creation for small business owners and content creators. While numerous marketing tools are available, most focus on isolated tasks such as content generation or image creation and require users to switch between multiple platforms. In addition, many

existing solutions lack support for multi-product campaign creation, bilingual content generation, and integrated AI-based marketing guidance.

Small businesses often face limited financial resources, insufficient marketing expertise, and time constraints, making it difficult to create professional and effective advertising campaigns. These challenges highlight the need for a unified AI-powered platform that automates campaign generation, provides marketing recommendations through an intelligent chatbot, and enables users to manage multiple products within a single campaign. By addressing these real-world gaps, **AD GENERATOR AI** aims to make digital marketing more efficient, affordable, and accessible for businesses of all sizes.

1.3 Proposed system

AD GENERATOR AI is an intelligent AI-powered marketing platform designed to simplify the process of creating digital marketing campaigns for small business owners, entrepreneurs, and content creators. The proposed system aims to help users generate professional and engaging marketing campaigns automatically without requiring advanced marketing knowledge or design experience.

The system allows users to create personalized campaigns by entering campaign-related information such as product details, campaign goals, target platforms, output language, occasion, style, emotion, and call-to-action preferences. Based on these inputs, the system uses Artificial Intelligence techniques to generate complete marketing campaigns tailored to different digital platforms including Facebook, Instagram, TikTok, and Websites/Ads.

In addition, the system supports multi-product campaign creation, allowing users to include more than one product within the same marketing campaign. This feature enables users to promote product collections, bundles, seasonal offers, or related products in one organized campaign instead of creating separate campaigns for each product. As a result, the system provides greater flexibility, reduces repetitive work, and helps users manage multi-product promotions more efficiently.

The system supports bilingual campaign generation, allowing users to generate captions, hashtags, and promotional content in either English or Arabic.

To improve marketing effectiveness and audience engagement, the system generates three different post variations for each campaign:

- Product-Focused Post
- Campaign-Focused Post

- Lifestyle-Focused Post

Each generated post includes AI-generated captions, promotional images, and platform-specific hashtags according to the selected platform requirements. The system also recommends suitable posting schedules to help users publish content at optimal times for better reach and engagement.

In addition to campaign generation, the proposed system integrates an AI Marketing Chatbot Assistant that enables users to ask questions related to marketing strategies, sales improvement, customer engagement, branding, and content creation. The chatbot provides intelligent recommendations and marketing guidance to support users in improving their business performance.

The proposed system also includes campaign management features such as saving generated campaigns, reviewing previous campaigns through a campaign library, and managing user profiles within the application. The platform is designed with a modern and user-friendly interface to ensure accessibility, usability, and efficient navigation for all users.

By combining AI-powered campaign generation, intelligent marketing assistance, and campaign management features in one integrated application, **AD GENERATOR AI** aims to provide a smart, efficient, and time-saving solution for modern digital marketing challenges.

1.3.1 Aims and Objectives

The primary aim of **AD GENERATOR AI** is to simplify digital marketing campaign creation for small business owners and content creators using Artificial Intelligence technologies.

The objectives of the proposed system include:

1. Generating professional AI-powered marketing campaigns automatically.
2. Supporting multiple platforms including Facebook, Instagram, TikTok, and Websites/Ads.
3. Creating different campaign styles such as Product-Focused, Campaign-Focused, and Lifestyle-Focused post.
4. Supporting bilingual campaign generation in English and Arabic.
5. Supporting multi-product campaign creation within a single campaign.
6. Generating AI-based captions, promotional images, and platform-specific hashtags.
7. Providing intelligent posting schedule recommendations.
8. Assisting users through an AI Marketing Chatbot Assistant.

9. Helping users improve marketing strategies and customer engagement.

10. Saving and managing campaigns through a campaign library.

11. Providing a modern, user-friendly, and accessible mobile application interface.

1.3.2 Proposed system features

1. Campaign Creation

- Create customized marketing campaigns.
- Enter campaign details and objectives.
- Select target platforms for content generation and publishing.
- Add a single product or multiple products within the same campaign.
- Manage multi-product campaign details in one organized campaign.

2. Bilingual Campaign Generation

- Support campaign generation in both English and Arabic.
- Allow users to select the campaign output language.
- Generate captions, hashtags, and promotional content according to the selected language.

3. AI Content Generation

- Generate AI-powered captions
- Generate promotional marketing content.
- Generate AI-powered promotional images.
- Produce content tailored to different social media and advertising platforms.

4. Post Variations Generation

- Generate three different post variations for each campaign.
- Provide multiple marketing approaches.
- Improve audience engagement and content diversity.

5. Hashtag Generation

- Generate platform-specific hashtags.

- Instagram: Up to 20 hashtags.
- Facebook: Up to 6 hashtags
- TikTok: Up to 5 hashtags.
- Websites/Ads: Up to 5 hashtags.

6. AI Marketing Chatbot

- Provide marketing-related assistance.
- Answer questions about branding and digital marketing.
- Offer customer engagement recommendations.
- Suggest strategies to improve sales performance.

7. Campaign Scheduling Recommendations

- Recommend suitable posting schedules.
- Suggest optimal publishing times.
- Support better campaign performance and audience reach.

8. Campaign Management

- Save generated campaigns.
- Access and review previous campaigns.
- Manage campaign information efficiently.

9. Campaign Library

- Store all generated campaigns in one place.
- Organize campaign records.
- Enable easy retrieval and reuse of campaign content.

10. User Authentication

- Sign up and sign in using email and password.
- Sign in using a Google account (Login with Google).
- Reset a forgotten password through a secure Forgot Password / Reset Password process.

11. User Profile Management

- Manage user account information.
- Update profile details.
- Maintain personalized user settings

12. User Interface and User Experience

- Provide a clean and modern interface.
- Ensure simple and intuitive navigation.
- Improve usability and user interaction.
- Deliver an efficient and user-friendly experience.

1.4 Project Methodology

The development of **AD GENERATOR AI** follows the Agile Scrum methodology to ensure flexibility, continuous improvement, effective collaboration, and efficient project management throughout the development lifecycle. This methodology was selected because it supports incremental development, regular feedback, and the ability to adapt to changing requirements during the project implementation.

1. Agile Scrum Approach

The **AD GENERATOR AI** system is developed using the Agile Scrum methodology. Agile Scrum provides a flexible and iterative development process that allows the project team to divide the work into smaller manageable tasks. It also supports continuous improvement, effective teamwork, and efficient handling of changing requirements throughout the development lifecycle.

2. System Components

The proposed system consists of the following main components:

- Mobile Application: Handles user interaction, campaign creation, multi-product campaign input, chatbot communication, campaign management, and user profile management.
- .NET Backend System: Manages user authentication, data processing, database operations, API requests, and communication between the mobile application and AI services
- AI Services: Generate AI-powered captions, hashtags, campaign content, promotional

images, posting recommendations, and chatbot responses.

3. Sprint-Based Development

The development process is divided into multiple sprints. Each sprint focuses on implementing and improving specific system features. These features include user authentication, campaign creation, multi-product campaign support, AI content generation, chatbot integration, campaign scheduling recommendations, campaign library management, and user profile management.

4. Planning and Design

At the beginning of each sprint, the project requirements and priorities are reviewed and organized according to the project objectives. UML diagrams, database designs, API structures, and UI/UX prototypes are prepared and updated when needed. This stage helps ensure that the system architecture and design remain aligned with the functional requirements of the project.

5. Implementation and Integration

The system features are implemented incrementally during the development sprints. The mobile application is developed to provide a smooth and user-friendly experience, while the backend services are implemented to manage data, authentication, and communication with AI services. AI modules are integrated with the system to support campaign content generation, promotional image generation, hashtag creation, posting schedule recommendations, and chatbot functionality.

6. Testing and Quality Assurance

Continuous testing is performed throughout the development process to ensure that the system works correctly and meets user requirements. Functional testing, usability testing, security testing, and performance evaluation are carried out during each sprint. Any issues or errors are identified, reviewed, and resolved to improve the quality and reliability of the system.

7. Team Collaboration and Review

Regular sprint meetings are conducted to review progress, discuss completed tasks, identify challenges, and plan upcoming work. Team members collaborate continuously to ensure that the development process remains organized and aligned with the project goals. Feedback from supervisors and team discussions is used to improve the system and enhance the overall project quality.

8. Project Outcomes

By following the Agile Scrum methodology, the project achieves better flexibility, scalability, and maintainability. The methodology helps the team adapt to requirement changes, improve development efficiency, and ensure effective integration between the mobile application, backend system, and AI-powered services. As a result, **AD GENERATOR AI** is developed as an intelligent, user-friendly, and efficient marketing campaign generation platform.

1.5 Resource Requirements (AD GENERATOR AI System)

This section presents the software, tools, technologies, and resources required for the development, implementation, and deployment of the **AD GENERATOR AI** system.

1.5.1 Software and Tools

Programming Languages:

- Dart is used to develop the mobile application using Flutter, targeting the Android platform.
- C# is used for backend development using ASP.NET Core Web API, handling authentication, APIs, database communication, campaign management, and AI integration.
- Python is used for AI-related functionalities including caption generation, hashtag generation, chatbot responses, and content generation.

Development Frameworks:

- Flutter framework is used for building the mobile application UI and logic.
- ASP.NET Core Web API is used to build RESTful APIs and manage communication between frontend, backend, and AI services.

API Testing Tools

- Swagger is used for testing, documenting, and validating backend API endpoints.

AI Support Tools

ChatGPT, Claude AI, and Manus AI are used during development as supportive tools for debugging, fixing errors, improving implementation decisions, and assisting in development-related tasks.

AI and Machine Learning Tools

- Google Colab is used for experimenting and testing AI-related Python code.
- AI models are used for generating marketing content including captions, hashtags, posts, and chatbot responses.
- English and Arabic datasets are used to train and support content generation for different marketing styles.

Integrated Development Environment (IDE)

- Visual Studio is used for backend development using ASP.NET Core.
- Visual Studio Code is used for Flutter development and supporting AI-related scripts.

Database Management System

- Microsoft SQL Server is used to store user accounts, campaigns, product details, generated content, chatbot interactions, campaign library data, and system information.
- The database also supports storing single-product and multi-product campaign data, including the relationship between campaigns and their related products.

ORM

- Entity Framework Core is used for database operations and managing data access in the backend system.
- It helps manage relationships between users, campaigns, products, generated posts, hashtags, images, and chatbot records.

Cloud Hosting Platform

- MonsterASP.NET is used to deploy and host the backend services and APIs.

Cloud Storage Service

- Cloudinary is used for storing and managing images, logos, product assets, campaign media files, and promotional images generated or uploaded within the application.

UI/UX Design Tool

- Figma is used to design wireframes, prototypes, user flows, and the application interface.

1.5.2 Data Required

User Data

- User profile information including username, email, authentication credentials, selected platforms, user preferences, and campaign history.

Campaign Data

- Campaign name, objectives, target audience, selected platforms, output language, occasion, tone, emotion, call-to-action, and marketing style.

Product Data

- Product name, product description, product category, product features, product images, and other product-related details required for campaign generation.

Multi-Product Campaign Data

- Data required to support single-product and multi-product campaigns, including the relationship

between each campaign and its related products.

AI Training Data

- English marketing datasets used for generating English content.
- Arabic marketing datasets used for generating Arabic content.
- Datasets supporting different marketing types such as product-focused, campaign-focused, and lifestyle content.

Chatbot (RAG) Data

- Marketing knowledge documents used in retrieval-augmented generation.
- Data sources used to generate accurate responses for marketing, branding, sales, and customer engagement queries.

Generated Campaign Data

- AI-generated captions, hashtags, posts, promotional images, chatbot responses, posting schedule recommendations, and platform-specific content variations.

Media and Asset Data

- Uploaded images, logos, product assets, campaign images, and promotional media files stored via Cloudinary.

1.5.3 Hardware Requirements

- User devices such as smartphones or tablets with internet access to use the mobile application.
- Development devices such as laptops or desktop computers for Flutter development, backend development, database management, and AI-related testing.
- Cloud or server infrastructure to host backend services, APIs, database communication, and AI-related processing.
- Reliable internet connection to support communication between the mobile application, backend system, cloud storage, and AI services.

1.5.4 Human Resources

Development Team:

- Flutter Developer responsible for mobile application development and user interface implementation.
- Backend Developer (.NET) responsible for API development, authentication, database communication, and system logic.
- AI Developer responsible for AI integration, content generation, chatbot functionality, and AI-related testing.
- Database Administrator responsible for managing Microsoft SQL Server, data storage, and database relationships.

- UI/UX Designer responsible for designing wireframes, prototypes, user flows, and the application interface.

Support / Advisory

- Academic supervisor responsible for providing project guidance, feedback, and academic supervision.
- Marketing consultants or reviewers responsible for validating the generated campaign content and ensuring that the marketing output is suitable and effective.

1.5.5 Other Resources

- Cloud computing services for deployment, hosting and storage.
- Internet connectivity for AI services, API communication, and system integration.
- External APIs and AI services for supporting captions, hashtags, promotional content, image generation, and chatbot responses.
- Cloud storage services for storing uploaded images, product assets, logos, campaign media, and generated promotional images.
- Development and testing accounts for services such as hosting platforms, Cloudinary, and AI service providers.

1.6 Report Layout

1. **Abstract**: Provides an overview of the project idea, objectives, main features, and expected outcomes.
2. **Introduction**: Presents the project background, problem domain, problem statement, proposed system, aims and objectives, system features, project methodology, and resource requirements
3. **Literature Review:** Reviews related work, existing digital marketing tools, AI-based content generation systems, chatbot technologies, and the technologies used in the project.
4. **System Analysis and Design:** Describes the system architecture, UML diagrams, use case diagrams, database design, API structure, user interface wireframes, and system workflow.
5. **Implementation**: Explains the technologies, tools, frameworks, and development steps used to build **AD GENERATOR AI**, including Flutter for the mobile application, ASP.NET Core Web API for the backend, Microsoft SQL Server for the database, Cloudinary for media storage, and AI services for content generation.
6. **Testing and Evaluation:** Presents the testing methods used to evaluate the system functionality, usability, performance, security, API integration, AI-generated outputs, chatbot responses, and overall user experience.

7. **Conclusion and Future Work:** Summarizes the project achievements, limitations, and possible future improvements such as advanced analytics, more platform integrations, improved AI personalization, and enhanced campaign performance tracking.

8. **References:** Lists all academic sources, research papers, documentation, tools, and online resources used during the development of the project.

CHAPTER 2

BACKGROUND/EXISTING WORK

(Literature Review, Existing Work)

BACKGROUND/EXISTING WORK

(Literature Review, Existing Work)

This chapter presents the background and existing work related to AI-powered marketing campaign generation. It discusses therole of Artificial Intelligence in digital marketing, reviews existing platforms and tools, identifies their limitations, and explains the innovations introduced by the proposed **AD GENERATOR AI** system.

The chapter is organized as follows:

2.0 Introduction

2.1 Overview of Existing Projects

2.2 Related Research Papers

2.3 Limitations of Existing Projects

2.4 Innovations of Our Project

2.5 Comparative Analysis

2.6 Methodology Selection

2.7 Justification

2.0 Introduction

Digital marketing has become one of the most important tools for businesses and content creators to promote products and services across online platforms. With the rapid growth of Artificial Intelligence technologies, many businesses are seeking smarter and faster ways to create advertising campaigns, generate marketing content, and improve customer engagement.

Small business owners and startups increasingly rely on digital platforms such as Instagram, Facebook, TikTok, and Websites/Ads to promote their products and reach larger audiences. However, creating effective marketing campaigns still requires significant time, marketing knowledge, creative writing skills, and content planning.

Many small business owners face challenges in creating professional campaigns due to limited marketing experience, lack of creative resources, and the time required to prepare captions, hashtags, promotional images, and publishing plans for different platforms. In addition, users who promote more than one product often need to create separate campaigns manually, which increases effort and makes campaign management more complex.

Many existing systems provide limited customization and do not fully support multilingual campaign generation, platform-specific content generation, multi-product campaign creation, or intelligent marketing assistance. These limitations make it difficult for beginners and small business owners to create professional campaigns efficiently.

The proposed **AD GENERATOR AI** system aims to simplify the campaign creation process by providing AI-powered marketing assistance, bilingual campaign generation, intelligent hashtag creation, promotional image generation, campaign scheduling recommendations, and AI chatbot support. The system allows users to generate marketing campaigns for platforms such as Instagram, Facebook, TikTok, and Websites/Ads using customizable campaign inputs and Artificial Intelligence technologies.

This chapter reviews existing AI-powered marketing systems and related platforms, identifies their limitations, and discusses the innovative features introduced by the proposed system.

2.1 Overview of existing projects

The **AD GENERATOR AI** system combines campaign generation, AI-powered content creation, multilingual support, multi-product campaign support, and intelligent chatbot assistance into one integrated mobile application. Although several existing platforms provide some marketing functionalities, most systems focus on limited features and do not offer a complete AI-driven

campaign generation experience.

Existing systems can be categorized as follows:

1. AI Content Generation Platforms

Platforms such as Canva AI, Jasper AI, and Copy.ai provide AI-generated marketing captions and promotional text. These systems help users create content quickly; however, they often lack platform-specific optimization, personalized campaign customization, and integrated campaign scheduling features.

2.Social Media Marketing Tools

Platforms such as Hootsuite and Buffer assist users in managing social media accounts and scheduling posts across multiple platforms. While these systems are useful for content publishing and analytics, they do not provide complete AI-generated campaigns with customized captions, hashtags, promotional images, bilingual support, and intelligent campaign style generation.

3. AI Chatbot and Marketing Assistance Systems

Some AI chatbot systems provide users with marketing tips and customer support assistance. However, many existing chatbot solutions are general-purpose systems and are not specifically designed to support campaign generation, hashtag recommendations, product promotion planning, or personalized marketing guidance for small business owners.

4. Graphic and Advertisement Design Platforms

Applications such as Canva and Adobe Express provide templates and design tools for creating advertisements and promotional images. Although these platforms support visual content creation, they still require manual effort from users and do not fully automate campaign generation using AI.

In summary, existing platforms provide valuable marketing and content creation services, but most of them focus on separate functionalities. The proposed **AD GENERATOR AI** system stands out by integrating AI-powered campaign generation, multilingual content creation, multi-product campaign support, hashtag generation, AI chatbot assistance, promotional image generation, and posting schedule recommendations into a single intelligent platform.

2.2 Related Research Papers

This section reviews the existing research and academic literature most relevant to the core components of **AD GENERATOR AI**. The related work is organized into four areas: AI-powered digital marketing, text generation using transformer models, AI image generation for advertising, and Retrieval-Augmented Generation (RAG) for chatbot systems.

2.2.1 AI-Powered Digital Marketing

[1] From Ideation to Execution: Unleashing the Power of Generative AI in Modern Digital Marketing

Al-Emadi et al. — ScienceDirect, 2025

Relevance: A systematic literature review confirming that Generative AI can automate ad copy creation, platform-specific content, and campaign personalization — directly aligning with the core objectives of **AD GENERATOR AI**.

Available at: https://www.sciencedirect.com/science/article/pii/S2590005625002577

[2] AI Adoption in Marketing Strategies: Navigating the Present and Shaping the Future Business Landscape

Kumar et al. — ScienceDirect, 2025

Relevance: Demonstrates that AI enhances customer engagement, decision-making, and content targeting for small businesses, validating the target audience and use case of this project.

Available at: https://www.sciencedirect.com/science/article/pii/S2590291125007776

[3] LLM-Generated Ads: From Personalization Parity to Persuasion Superiority

Meguellati et al. — arXiv, 2025

Relevance: Proves that LLM-generated advertisements achieved a 59.1% audience preference rate versus 40.9% for human-written ads, strongly supporting the quality and viability of AI-generated campaign content.

Available at: https://arxiv.org/abs/2512.03373

[4] Leveraging Artificial Intelligence as a Strategic Growth Catalyst for Small and Medium-sized Enterprises

Al-Khalifa et al. — arXiv, 2025

Relevance: Confirms that Generative AI content creation tools — including text, images, and social media posts — are a key competitive advantage for small businesses, directly matching the target user segment of **AD GENERATOR AI.**

Available at: https://arxiv.org/abs/2509.14532

2.2.2 Text Generation Using Transformer Models

[5] Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5)

Raffel et al. — Journal of Machine Learning Research, 2020

Relevance: The foundational paper introducing the T5 architecture used in the English text generation pipeline of **AD GENERATOR AI**. Establishes the text-to-text paradigm and prefix-driven task formatting that the fine-tuned model builds upon.

Available at: https://arxiv.org/abs/1910.10683

[6] SLM4Offer: Personalized Marketing Offer Generation Using Contrastive Learning Based Fine-Tuning of T5

Naous et al. — arXiv, 2025

Relevance: Uses T5-Small (60M parameters) — the same model variant as **AD GENERATOR AI** — to generate personalized marketing offers from structured customer profiles, demonstrating the model's suitability for domain-specific marketing text generation.

Available at: https://arxiv.org/abs/2508.15471

[7] Enhancing Automatic Keyphrase Labelling with Text-to-Text Transfer Transformer (T5)

Gabín et al. — arXiv, 2024

Relevance: Shows that T5 with prefix-driven formatting is highly effective for structured short-form output generation from document content — directly analogous to the hashtag generation task in AD GENERATOR AI.

Available at: https://arxiv.org/abs/2409.16760

[8] Machine Generated Product Advertisements: Benchmarking LLMs Against Human Performance

Zhang et al. — arXiv, 2024

Relevance: Benchmarks GPT-2, LLaMA 3.1-8B, Gemma-2B, and GPT-4 for product ad generation and finds that smaller fine-tuned models are competitive for domain-specific tasks, supporting the choice of T5-Small with full fine-tuning over larger general-purpose models.

Available at: https://arxiv.org/abs/2412.19610

2.2.3 AI Image Generation for Advertising

[9] High-Resolution Image Synthesis with Latent Diffusion Models

Rombach et al. — CVPR, 2022

Relevance: The foundational paper introducing Stable Diffusion, which is the base model used in Stages 1 and 2 of the AD GENERATOR AI image generation pipeline. Establishes the latent diffusion approach for high-quality promotional image synthesis from text prompts.

Available at: https://arxiv.org/abs/2112.10752

[10] LoRA: Low-Rank Adaptation of Large Language Models

Hu et al. — arXiv, 2021

Relevance: Introduces the LoRA fine-tuning technique used in Stages 1 and 2 of the image pipeline. LoRA reduces trainable parameters to only 0.09% of the full UNet, enabling efficient domain-specific adaptation on free-tier GPU hardware within the project constraints.

Available at: https://arxiv.org/abs/2106.09685

[11] LoRA-Based Image Generation for Reflecting Fashion Brand Characteristics Using Stable Diffusion

Kim et al. — Springer, 2024

Relevance: Applies LoRA combined with Stable Diffusion in the fashion product domain — the same technical approach and product domain as the **AD GENERATOR AI** image pipeline — demonstrating the effectiveness of this combination for fashion marketing visuals.

Available at: https://link.springer.com/chapter/10.1007/978-981-95-0887-7_15

[12] T-Stars-Poster: A Framework for Product-Centric Advertising Image Design

Chen et al. — arXiv, 2025

Relevance: Proposes a diffusion-based framework specifically for generating promotional product images aligned with campaign context and style — directly comparable to the Stage 3 image generation module of **AD GENERATOR AI.**

Available at: https://arxiv.org/abs/2501.14316

2.2.4 Retrieval-Augmented Generation for AI Chatbot

[13] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Lewis et al. — NeurIPS, 2020

Relevance: The foundational RAG paper defining the retrieval-then-generate architecture used in the **AD GENERATOR AI** marketing chatbot. The system retrieves relevant marketing knowledge from a curated knowledge base before generating responses, following this exact paradigm.

Available at: https://arxiv.org/abs/2005.11401

[14] FACTS About Building Retrieval Augmented Generation-based Chatbots

Akkiraju et al. — arXiv, 2024

Relevance: Provides practical engineering guidelines for building production RAG chatbots, including retrieval quality thresholds, confidence scoring, and fallback strategies — all directly implemented in the AD GENERATOR AI chatbot through the judge node and Tavily web search fallback.

Available at: https://arxiv.org/abs/2407.07858

[15] Harnessing the Potential of Large Language Models in Modern Marketing Management

Mogaji et al. — arXiv, 2025

Relevance: Shows that raw LLM output lacks the domain-specific creativity required for effective marketing without adaptation, directly motivating the RAG-based approach for the chatbot and the fine-tuning approach for the T5 text generation pipeline.

Available at: https://arxiv.org/abs/2501.10685

2.3 Limitations of existing projects

Despite the availability of many digital marketing and AI-powered content generation platforms, several limitations reduce their effectiveness in providing a complete and intelligent campaign generation experience for users and small business owners.

1. Limited Personalization

Many existing marketing platforms generate generic promotional content without deeply customizing campaigns based on product type, target audience, campaign emotion, occasion, marketing style, or call-to-action preferences. Users often receive repetitive content that does not fully match their business needs.

2. Weak Multilingual Support

Several systems mainly focus on English content generation and provide limited support for Arabic marketing campaigns. In many cases, Arabic-generated captions and hashtags may lack accuracy, creativity, or proper marketing tone, reducing campaign quality for Arabic-speaking users.

3. Lack of Platform-Specific Optimization

Different social media platforms require different content styles and hashtag strategies. However,

many existing systems generate the same type of content for all platforms without considering platform-specific requirements such as hashtag count, caption style, audience behavior, or post formatting for Instagram, Facebook, TikTok, and Websites/Ads.

4.Limited Multi-Product Campaign Support

Many existing tools focus on creating content for a single product at a time. Small business owners who need to promote product collections, bundles, seasonal offers, or related products often have to create separate campaigns manually. This increases time, effort, and complexity, especially when managing multiple products within one marketing campaign.

5. Limited AI Assistance

Most existing tools focus only on content generation and do not provide intelligent marketing guidance or interactive chatbot support. Users cannot ask marketing-related questions or receive AI-based recommendations to improve campaign performance, branding, customer engagement, or sales strategies.

6. Lack of Complete Campaign Generation

Many applications provide only one part of the campaign creation process, such as caption generation or image design. Users still need additional tools to create hashtags, posting schedules, platform-specific content, promotional images, and marketing guidance, leading to a fragmented user experience.

7. Limited Automation

Some marketing systems require significant manual editing and content adjustments before publishing. This increases the time and effort needed to prepare professional campaigns, especially for small business owners with limited marketing experience.

8. Poor Integration Between AI and Marketing Features

Existing platforms often separate AI content generation from campaign management features. As a result, users must switch between multiple applications for content writing, image creation, hashtag generation, scheduling, product organization, and marketing consultation.

9. Lack of Intelligent Scheduling Recommendations

Many systems do not provide recommended posting times based on the selected platform. Users are responsible for determining suitable publishing schedules manually, which may reduce campaign reach and audience engagement.

10. Limited User Guidance

Current systems may be difficult for beginners who have limited marketing knowledge. Most platforms do not provide simple AI-powered assistance to help users understand marketing concepts, campaign strategies, audience targeting techniques, or content improvement methods.

In summary, existing systems provide useful marketing functionalities but still suffer from limitations related to personalization, multilingual support, multi-product campaign support, intelligent assistance, automation, and platform-specific optimization. These limitations highlight the need for an integrated AI-powered solution such as the proposed **AD GENERATOR AI** system.

2.4 Innovations of our project

The proposed **AD GENERATOR AI** system has been developed to overcome the limitations of existing marketing and content generation platforms by introducing several innovative AI-powered features that improve campaign creation, marketing assistance, multilingual content generation, and campaign management.

1. AI-Powered Campaign Generation

The system automatically generates complete marketing campaigns using artificial intelligence. Users can enter product information, campaign goals, target platforms, style preferences, emotions, occasions, and call-to-action details, and the system generates professional promotional content accordingly.

2. Bilingual Campaign Generation

One of the important innovations of the proposed system is supporting multi-product campaign creation. Users can add a single product or multiple products within the same marketing campaign. This feature allows small business owners to promote product collections, bundles, seasonal offers, or related products in one organized campaign instead of creating separate campaigns for each product.

3. Bilingual Campaign Generation

The system supports both English and Arabic campaign generation. Users can select their preferred output language, and the system generates captions, hashtags, and marketing content based on the selected language using suitable language resources and marketing datasets.

4. Platform-Specific Content Generation

The system generates customized content for different platforms such as Instagram, Facebook,

TikTok, and Websites/Ads. Each platform receives optimized captions and hashtags according to its marketing style, audience behavior, and platform requirements.

5. Intelligent Hashtag Generation

The application generates hashtags automatically based on the campaign content and selected platform. Different platforms receive different numbers of hashtags to improve audience reach and engagement.

6. Multiple Campaign Styles

The system generates three different campaign variations for each marketing campaign:

- Product-Focused Campaign
- Campaign-Focused Campaign
- Lifestyle-Focused Campaign

This provides users with multiple marketing approaches for promoting their products and helps increase content diversity.

7. AI Chatbot Assistance Using RAG System

The project integrates an intelligent chatbot based on a Retrieval-Augmented Generation (RAG) system. The chatbot allows users to ask marketing-related questions and receive AI-powered guidance about branding, sales improvement, audience targeting, customer engagement, and advertising strategies.

8. AI-Generated Marketing Images

The system supports generating promotional campaign images that match the campaign style and marketing content, helping users create visually attractive advertisements automatically.

9. Intelligent Posting Schedule Recommendations

The application recommends suitable posting times for each platform to improve campaign visibility, audience interaction, and marketing performance.

10. User-Friendly Mobile Application

The system is designed as an easy-to-use mobile application that simplifies the campaign creation process for small business owners, content creators, and beginners with limited marketing experience.

11. Integrated Marketing Solution

Unlike many existing systems that focus on only one functionality, the proposed project combines

campaign generation, multi-product campaign support, hashtag creation, AI chatbot assistance, image generation, bilingual support, and posting schedule recommendations in one integrated platform.

By introducing these innovations, the **AD GENERATOR AI** system provides a smart, efficient, and user-friendly marketing experience that helps users create professional AI-powered campaigns quickly and effectively.

2.5 Comparative Analysis

This section compares the proposed **AD GENERATOR AI** system with existing AI-powered marketing platforms, social media management tools, and content generation applications. The comparison focuses on key aspects such as multilingual support, AI-generated captions, hashtag generation, chatbot assistance, platform-specific optimization, multi-product campaign support, and intelligent campaign automation.

This analysis highlights the limitations of current systems and demonstrates how the proposed project provides a more integrated, intelligent, and user-friendly marketing experience for small business owners and content creators.

Feature	AD GENERATOR AI	AI Content Generation Platforms	Social Media Management Tools	Design & Advertisement Platforms
AI-Generated Campaign Captions	✓	✓	✗	✗
Arabic & English Support	✓	Limited	✗	Limited
Platform-Specific Hashtag Generation	✓	✗	✗	✗
AI Chatbot Marketing Assistance	✓	Limited	✗	✗
Campaign Style Customization	✓	Limited	✗	✗
Product-Focused Campaign Generation	✓	✓	✗	✗
Lifestyle-Focused	✓	✗	✗	✗

Campaign Generation				
Campaign-Focused Marketing Content	✓	Limited	✗	Limited
AI-Generated Promotional Images	✓	Limited	✗	✗
Intelligent Posting Schedule	✓	✗	✓	✗
Multi-Platform Campaign Support	✓	Limited	✓	✗
Mobile Application Support	✓	✓	✓	✓
Integrated Marketing Solution	✓	✗	✗	✗
RAG-Based AI Chatbot	✓	✗	✗	✗
Beginner-Friendly User Experience	✓	Limited	Moderate	Moderate
Multi-Product Campaign Support	✓	Limited	✗	✗

Figure 1 Literature review

The comparison shows that most existing systems focus on only one aspect of digital marketing, such as content writing, social media scheduling, or graphic design. In contrast, the proposed **AD GENERATOR AI** system combines AI-powered campaign generation, multilingual support, multi-product campaign support, hashtag generation, AI chatbot assistance, promotional image generation, and intelligent scheduling into one integrated platform.

By addressing the limitations of current solutions, the proposed system provides a smarter and more efficient marketing experience that helps users create professional campaigns quickly and effectively.

2.6 Methodology Selection

The development of **AD GENERATOR AI** adopts the Agile Scrum methodology.This methodology was selected because it supports iterative development, continuous testing, flexibility, and effective collaboration among team members. It also enables the project team to deliver system features incrementally while adapting to changing requirements throughout the development lifecycle.

2.7 Justification

The proposed **AD GENERATOR AI** system was developed using a combination of modern technologies selected based on their performance, scalability, ease of integration, and suitability for AI-powered mobile applications.

- **Agile Scrum** was selected as the development methodology because it supports iterative development, continuous testing, effective teamwork, and the ability to adapt to changing project requirements.
- **Flutter** was chosen for mobile application development because it enables the creation of a modern, responsive, and high-performance cross-platform user interface using a single codebase.
- **ASP.NET Core Web API** was selected for backend development due to its high performance, security, scalability, and seamless integration with databases and AI services through RESTful APIs.
- **Microsoft SQL Server** was used as the database management system because it provides reliable data storage, efficient relationship management, strong security features, and excellent support for large-scale applications.
- **Entity Framework Core** was adopted to simplify database operations through Object-Relational Mapping (ORM), reducing development time and improving code maintainability.
- **Cloudinary** was selected as the cloud storage service because it provides secure storage, fast image delivery, and efficient management of uploaded and AI-generated media files.
- **Retrieval-Augmented Generation (RAG)** was integrated into the AI chatbot to improve the accuracy and relevance of responses by combining retrieved marketing knowledge with large language model capabilities.

The combination of these technologies provides an integrated architecture that supports intelligent campaign generation, efficient data management, AI-powered assistance, and a scalable mobile application capable of meeting the requirements of modern digital marketing.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

SOFTWARE REQUIREMENTS SPECIFICATION

This chapter presents the Software Requirements Specification (SRS) for **AD GENERATOR AI**. It defines the functional and non-functional requirements of the system, describes the intended audience, product scope, system overview, product functions, user classes, operating environment, constraints, user documentation, external interface requirements, system features, and analysis phase diagrams.

The chapter is organized as follows:

3.0 Introduction

3.1 Intended Audience and Reading Suggestions

3.2 Product Scope

3.3 Overall Description

3.4 Product Functions

3.5 User Classes and Characteristics

3.6 Operating Environment

3.7 Design and Implementation Constraints

3.8 User Documentation

3.9 External Interface Requirements

3.10 System Features

3.11 Analysis Phase

3.0 Introduction

Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the **AD GENERATOR AI** system. **AD GENERATOR AI** is an AI-powered mobile marketing platform designed to help users create professional digital marketing campaigns for platforms such as Instagram, Facebook, TikTok, and websites. The system automatically generates marketing content, including posts, captions, hashtags, promotional images, and posting schedule recommendations using Artificial Intelligence.

The system supports both single-product and multi-product campaigns, allowing users to promote individual products or product collections within a single campaign. This document serves as a reference for developers, testers, project supervisors, and stakeholders, ensuring that the system is designed and implemented according to the specified requirements and project objectives.

The system focuses on the following core functionalities:

- Automated generation of marketing campaigns for products or services.
- Support for single-product and multi-product campaign creation.
- Creation of AI-generated posts tailored for different platforms.
- Generation of platform-specific captions, hashtags, and promotional images.
- Recommendation of optimal posting schedules to improve engagement.
- AI-powered chatbot assistance for marketing consultation and user support.
- User profile management and campaign history tracking.

- Campaign saving, reviewing, and management through a campaign library.

The SRS defines the scope of all system modules including:

1. Frontend Mobile Application:
The frontend is developed using Flutter and provides a mobile interface that allows users to input campaign data, add products, select platforms, view generated content, interact with the AI chatbot, manage campaigns, and update user profiles.

2. Backend (.NET Web API):
The backend is developed using ASP.NET Core Web API and is responsible for handling authentication, business logic, campaign processing, database communication, API routing, AI integration, and campaign management.

3. Database:
Microsoft SQL Server is used to store user accounts, product data, campaign data, generated captions, hashtags, promotional images, chatbot interactions, posting schedules, and campaign history.

4. AI Engine:
The AI engine includes AI models and services responsible for campaign text generation, hashtag generation, promotional image generation, posting schedule recommendations, and chatbot-based marketing assistance.

This document serves as a reference for developers, testers, project managers, stakeholders, and future maintainers to ensure that **AD GENERATOR AI** is implemented according to user needs and project objectives

3.1 Intended Audience and Reading Suggestions

This Software Requirements Specification document is intended for different stakeholders involved in the development, testing, deployment, and usage of **AD GENERATOR AI**. Each audience group can use this document according to its role and responsibilities.

3.1.1 Developers

Developers use this document to understand the technical requirements, system architecture, backend services, AI integration, database structure, and functional workflows of the system.

Relevant Sections:

- Functional Requirements
- System Features
- External Interface Requirements
- Analysis Phase
- System Design and Implementation

Suggested Reading Sequence:

- Software Requirements Specification
- System Design
- Implementation
- Project Managers

3.1.2 Project Managers

Project managers use this document to understand the system scope, project objectives, required resources, development methodology, and expected outcomes.

Relevant Sections:

- Product Scope
- Product Functions
- User Classes
- System Features
- Resource Requirements
- Testing and Evaluation

Suggested Reading Sequence:

- Introduction
- Software Requirements Specification Overview
- Testing and Evaluation
- Testers

3.1.3 Testers

Testers use this document to verify that the system meets its functional and non-functional requirements.

They use it to prepare test cases, validate system behavior, and identify errors.

Relevant Sections:

- Functional Requirements
- Non-Functional Requirements
- System Features
- External Interface Requirements
- Testing and Evaluation

Suggested Reading Sequence:

- Software Requirements Specification
- System Features
- Testing and Evaluation
- Marketing Reviewers

3.1.4 Marketing Reviewers

Marketing reviewers use this document to understand the marketing-related features of the system, including campaign generation, caption creation, hashtag generation, promotional image generation, and AI chatbot assistance.

Relevant Sections:

- Product Scope.
- Product Functions.
- Proposed System Features.
- AI Campaign Generation Module.
- AI Chatbot Module.

Suggested Reading Sequence:

- Introduction.

- Proposed System.
- Software Requirements Specification.
- End Users.

3.1.5 End Users

End users include small business owners, entrepreneurs, marketers, and content creators who use the application to generate professional marketing campaigns.

Relevant Sections:

- Product Scope.
- Product Functions.
- User Interfaces
- User Documentation
- System Features

Suggested Reading Sequence:

- Introduction.
- Product Scope
- System Features
- Documentation Writers

3.1.6 Documentation Writers

Documentation writers use this document to prepare user manuals, help guides, FAQs, and training materials.

Relevant Sections:

- User Documentation
- Product Functions.
- User Interfaces
- System Features
- Analysis Phase

Suggested Reading Sequence:

- Software Requirements Specification
- System Features
- User Documentation

This document is organized sequentially to allow readers to move from high-level system concepts to detailed technical requirements. Readers should prioritize sections according to their roles within the project.

3.2 Product Scope

AD GENERATOR AI is a comprehensive AI-powered mobile platform designed to simplify and automate the creation of digital marketing campaigns for small business owners, marketers, entrepreneurs, and content creators.

The system enables users to generate complete marketing campaigns that include captions, hashtags, promotional images, post variations, and posting schedule recommendations across multiple digital platforms such as Instagram, Facebook, TikTok, and Websites/Ads.

The primary purpose of the system is to reduce the complexity of digital marketing content creation by using Artificial Intelligence to produce high-quality, platform-specific, and customized marketing outputs in a short time.

The system also supports multi-product campaign creation. Users can add one or more products within the same campaign, which allows them to promote product collections, bundles, seasonal offers, or related products in one organized workflow.

Key Objectives and Benefits:

1. **Streamline Campaign Creation:**
   The system automates the generation of complete marketing campaigns, including text content, hashtags, promotional images, and scheduling recommendations in one workflow.

2. **Single and Multi-Product Campaign Management**
   The system allows users to create a campaign for a single product or multiple products within the same campaign, reducing repetitive work and improving campaign management.

3. **Multi-Platform Content Generation:**
   The system supports campaign generation tailored for different platforms such as Instagram, Facebook, TikTok, and Websites/Ads with platform-specific optimization.

4. **AI-Powered Content Personalization:**
   The system provides customized marketing outputs based on product details, campaign goals, target audience, campaign style, emotion, occasion, and call-to-action preferences.

5. **Bilingual Campaign Support:**
   The system allows users to generate marketing campaigns in both English and Arabic based on the selected output language.

6. **Enhanced Marketing Assistance:**
   The system provides intelligent marketing recommendations through an AI chatbot to help users improve campaign quality,customer engagement, branding, and sales strategies.

7. **Automated Scheduling Recommendations:**
   The system suggests suitable posting times for each platform to help users improve audience reach and engagement.

8. **Creative Visual Generation:**
   The system generates AI-based promotional images for each marketing post to enhance visual appeal and campaign effectiveness.

9. **Campaign Management:**
   The system allows users to save generated campaigns, review previous campaigns, manage campaign content, and reuse campaign outputs through a campaign library.

3.3 Overall Description

Product Perspective

AD GENERATOR AI is a standalone mobile-based platform designed to simplify and automate the creation of digital marketing campaigns. It is not part of an existing system and has been developed as a new solution to address the challenges of manual and time-consuming campaign creation.

Unlike traditional marketing tools that focus on isolated tasks such as content writing, scheduling, image editing, or hashtag generation, **AD GENERATOR AI** integrates multiple AI-powered functionalities into one unified mobile application.

The system combines campaign generation, promotional image generation, platform-specific hashtag generation, bilingual content generation, posting schedule recommendations, chatbot-based marketing assistance, and campaign management.

Context and Origin:

AD GENERATOR AI was developed as a new AI-powered marketing solution for small business owners, entrepreneurs, marketers, and content creators. The system addresses common challenges such as limited marketing experience, lack of creative resources, time-consuming content preparation, and difficulty in adapting content to different platforms.

The system integrates multiple AI services and external APIs to enhance its functionality, including AI text generation models, AI image generation services, and chatbot-based RAG systems. It also supports communication between the mobile application, backend system, database, and AI services through secure RESTful APIs.

3.3.1 System Components and Interactions:

1. **Frontend Mobile Application:**

   The Flutter mobile application provides a user-friendly interface that allows users to create marketing campaigns, enter product details, add single or multiple products, select platforms, choose output language, interact with the AI chatbot, view generated results, and manage saved campaigns.

2. **Backend System:**

   The ASP.NET Core Web API handles user authentication, campaign processing, product management, multi-product campaign handling, posting schedule recommendations, database operations, and communication between the mobile application and AI services.

3. **Database:**

   Microsoft SQL Server stores user profiles, product data, campaign records, generated captions, hashtags, promotional images, chatbot conversations, posting schedules, and campaign library data.

4. **AI Engine:**

The AI engine is responsible for generating marketing captions, hashtags, promotional images, bilingual content in English and Arabic, chatbot responses, and posting schedule recommendations.

5. **External Interfaces :**

The system integrates with external AI APIs, image generation services, cloud storage services, and hosting services to support content generation, media handling, and system deployment.

3.4 Product Functions

AD GENERATOR AI provides the following core functions:

3.4.1 User Authentication:

- Allow users to sign up using email and password.
- Allow users to sign in securely.
- Support login using Google account.
- Allow users to reset forgotten passwords.
- Protect user accounts through secure authentication and authorization.

3.4.2 Campaign Creation

- Allow users to create customized marketing campaigns.
- Allow users to enter campaign name, campaign goals, target audience, platform, style, emotion, occasion, and call-to-action preferences.
- Allow users to select the campaign output language, either English or Arabic.
- Allow users to add a single product or multiple products within the same campaign.
- Support product collections, bundles, seasonal offers, and related products.

3.4.3 AI Campaign Generation

- Generate complete AI-powered marketing campaigns based on user inputs.
- Generate three different campaign variations: Product-Focused, Campaign-Focused, and Lifestyle-Focused posts.
- Generate content in the selected language, either English or Arabic.

- Generate content tailored to selected platforms such as Instagram, Facebook, TikTok, and Websites/Ads.

3.4.4 Hashtag Generation

- Generate AI-powered hashtags based on campaign details, product category, selected platform, and audience preferences.
- Generate platform-specific hashtags according to each platform's requirements.
- Support different hashtag quantities for Instagram, Facebook, TikTok, and Websites/Ads.

3.4.5 Promotional Image Generation

- Generate AI-powered promotional images for campaign posts.
- Align generated images with campaign style, emotion, product details, and marketing context.
- Support image generation for different campaign variations.
- Store generated images for future access and reuse.

3.4.6 Posting Schedule Recommendations

- Recommend suitable posting times for selected platforms.
- Suggest optimal publishing schedules to improve reach and engagement.
- Display posting recommendations after campaign generation.

3.4.7 AI Marketing Chatbot

- Provide AI-based marketing guidance and recommendations.
- Answer questions related to branding, digital marketing, customer engagement, advertising strategies, and sales improvement.
- Support bilingual chatbot interaction in English and Arabic.
- Use RAG-based knowledge retrieval to provide more relevant marketing responses.

3.4.8 Campaign Management

- Save generated campaigns.
- View and review previous campaigns.
- Edit or delete saved campaigns when needed.
- Store campaign components including captions, hashtags, images, products, and schedules.

3.4.9 Campaign Library

- Store all generated campaigns in one organized place.
- Allow users to retrieve and reuse previous campaign content.
- Support better organization of campaign history.

3.4.10 User Profile Management

- Allow users to view and update profile information.
- Maintain personalized user settings.
- Display user-related campaign statistics when required.

3.5 User Classes and Characteristics

3.5.1 End Users :

End users include small business owners, entrepreneurs, marketers, and content creators who use the system to generate marketing campaigns for their products or services.

Role:

- Create AI-powered marketing campaigns.
- Add product details and campaign preferences.
- Generate captions, hashtags, promotional images, and posting schedules.
- Interact with the AI marketing chatbot.
- Save and manage campaign history.

Charactaristics:

- Technical experience may vary from beginner to intermediate.
- May have limited marketing knowledge.
- Need a simple and user-friendly application.
- Interested in saving time, improving engagement, and increasing sales.

3.5.2 System Administiration:

The system administrator is responsible for managing the system environment, monitoring backend services, and ensuring smooth operation.

Role:

- Manage user accounts when required.

- Monitor backend services and APIs.
- Maintain system stability and performance.
- Manage system configuration and deployment settings.Save and manage campaign history.

Charactaristics:

- High technical experience.
- Strong knowledge of backend systems, APIs, databases, and hosting environments.
- Limited interaction with campaign generation features.

3.5.3 AI Services

AI services act as system actors responsible for generating intelligent outputs.

Role:

- Generate marketing captions.
- Generate hashtags.
- Generate promotional images.
- Provide chatbot responses.
- Recommend posting schedules.
- Process bilingual content in English and Arabic.

Charactaristics:

- Fully automated system actor.
- Uses AI models, NLP techniques, image generation services, and RAG-based chatbot processing.
- Processes user inputs and returns optimized marketing outputs.

3.5.4 System Testers:

System testers are responsible for verifying system functionality, usability, security, performance, and reliability.

Role:

- Test campaign generation features.
- Test authentication and user profile functions.

- Test chatbot responses.
- Test image generation and hashtag generation.
- Test campaign saving, retrieval, and deletion.
- Identify and report bugs.

Charactaristics:

- Fully automated system actor.
- Uses AI models, NLP techniques, image generation services, and RAG-based chatbot processing.
- Processes user inputs and returns optimized marketing outputs.

3.5.5 Priority Users:

End users represent the most important user class because they directly interact with the system's main features and depend on it to generate marketing campaigns and improve business performance.

3.6 Operating Environment

AD GENERATOR AI is designed to operate in the following environment:

3.6.1 Hardware Platform:

The system is designed as a mobile-first application and can be accessed through devices with internet connectivity, including:

- Smartphones and tablets running Android.
- Laptops or desktop computers for development, system administration, backend monitoring, and database management.
- Cloud or server infrastructure for hosting backend services and APIs.

3.6.2 Operating System:

The system supports modern operating environments, including:

- Android for the mobile application(latest versions)
- Windows (for backend/admin usage)
- macOS (for backend/admin usage)
- Linux or cloud-based server environments for deployment when supported.

3.6.3 Mobile Application Framework:

Flutter is used to develop the mobile application interface and application logic, providing a modern and responsive user experience.

3.6.4 Database

Microsoft SQL Server is used as the relational database management system for storing user data, product data, campaign records, generated content, hashtags, images, chatbot conversations, and posting schedules.

3.6.5 Backend Framework:

ASP.NET Core Web API is used to develop backend services. The backend is responsible for authentication, campaign management, product management, AI request routing, scheduling logic, and database operations.

3.6.6 AI and Processing Service:

The system integrates AI services to support marketing content generation, promotional image generation, hashtag generation, chatbot assistance, and bilingual campaign generation.

3.6.6.1 Arabic Content generation Model:

The Arabic marketing content generation model is deployed using Google Gemini API, and is responsible for generating Arabic captions, marketing text, and campaign content based on user inputs.

3.6.6.2 English Content Generation Model:

The English marketing content generation model is based on a T5 Transformer model, hosted on Hugging Face, and is used to generate optimized English marketing captions and campaign content.

3.6.6.3 AI Image Generation Service:

The image generation module is integrated using OpenRouter API, which is responsible for generating AI-powered promotional images based on campaign descriptions and style inputs.

3.6.6.4 AI Chatbot Service:

The chatbot system is deployed on Hugging Face, and provides intelligent responses for marketing assistance, campaign improvement suggestions, and user queries using a conversational AI model.

3.6.6.5 AI Processing Role:

All AI components collectively handle natural language processing, multilingual content generation, image creation, and conversational assistance to support end-to-end campaign generation within the system.

3.6.7 Cloud Hosting/Server:

- The backend services are deployed on a cloud-based hosting environment using Monster ASP.NET Hosting, which provides remote accessibility and ensures continuous availability of the system.
- This deployment enables the mobile application to communicate with backend APIs over the internet in real time.
- The system was initially developed and tested in a local environment to ensure correctness and stability before being deployed to the hosting server.
- The hosting environment supports scalability and allows multiple users to access the system simultaneously without interruption.

3.6.8 Communication Layer:

RESTful APIs are used for communication between the Flutter mobile application, backend system, database, and AI modules. Data exchange is performed in JSON format over secure HTTPS connections.

3.6.9 External Software Components:

The system may use external services such as:

- AI generation APIs for enhanced text and image generation capabilities.
- Hosting services for backend deployment.
- RAG-based chatbot system for intelligent marketing assistance and user support.

3.7 Design and Implementation Constraints

Several constraints affect the design and implementation of **AD GENERATOR AI**:

3.7.1 Technology Constraints:

The system must be developed using the selected technologies to ensure compatibility and maintainability. The mobile application is developed using Flutter, the backend is implemented using ASP.NET Core Web API, and Microsoft SQL Server is used as the primary database. AI-related functionalities are implemented using AI services and models for content generation, chatbot responses, and image generation.

3.7.2 Security Considerations:

The system must follow security best practices to protect user data and campaign

information. Secure authentication and authorization must be implemented. All communication between system components must be encrypted using HTTPS. Users must only be able to access their own accounts, campaigns, products, and generated content.

3.7.3 Hardware Constraints:

The application must operate efficiently on smartphones with different hardware capabilities. Heavy AI processing should be handled by backend services or external AI services instead of the mobile device to reduce client-side load.

3.7.4 Performance Constraints:

The system must provide acceptable response times for campaign generation, chatbot responses, hashtag generation, and image generation. Since some AI operations may require external services, the system must handle delays efficiently and provide a smooth user experience.

3.7.5 User Interface Constraints:

The user interface must be simple, responsive, and optimized for mobile usage. Complex screens and heavy UI elements should be minimized to maintain usability and performance.

3.7.6 Communication Constraints:

The system must use RESTful APIs for communication between the mobile application, backend services, and AI modules. Data must be exchanged in JSON format over secure connections.

3.7.7 AI And External Service Constrains:

The system depends on multiple external AI services including Google Gemini API (Arabic content generation), Hugging Face models (English content generation and chatbot services), and OpenRouter API (image generation). The availability, response time, and limitations of these external services may directly affect system performance and functionality.

3.7.8 Deployment Constrains:

The backend services are deployed on Monster ASP.NET Hosting, which defines certain limitations related to server resources and configuration. The system must be optimized to operate efficiently within the constraints of the hosting environment while ensuring

scalability and availability.

3.7.9 Parallel Operations:

The system must support multiple concurrent users interacting with the platform at the same time. The backend services must handle concurrent API requests, AI processing tasks, and database operations without conflicts or data inconsistency.

3.7.10 Data Management Constraints:

The system must manage relationships between users, campaigns, products, generated content, images, hashtags, chatbot records, and posting schedules accurately. Multi-product campaign relationships must be handled correctly to avoid data inconsistency.

3.8 User Documentation

The following user documentation will be provided with **AD GENERATOR AI**:

3.8.1 User Manual:

A user manual will explain how to use the mobile application, including registration, login, campaign creation, product input, multi-product campaign creation, platform selection, language selection, AI-generated campaign results, chatbot interaction, campaign saving, and campaign library management.

3.8.2 On-line Help:

Online help will be integrated into the application to provide real-time guidance for users. This may include tooltips, FAQs, campaign generation instructions, chatbot guidance, and feature explanations.

3.8.3 Administrator Guide:

An administrator guide will explain how to manage backend services, monitor system performance, maintain APIs, manage database operations, and handle system configuration.

3.8.4 Developer Documentation:

Developer documentation will describe APIs, database structure, AI integration, system architecture, and technical workflows to support future maintenance and development.

3.9 External Interface Requirements

3.9.1 User Interfaces

- Sign up page.
- Sign in page.
- Forgot password and reset password page.
- Home page.
- Campaign creation interface.
- Product information input interface.
- Multi-product input interface.
- Platform selection interface for Instagram, Facebook, TikTok, and Websites/Ads.
- Language selection interface for English and Arabic.
- Campaign style, emotion, occasion, and call-to-action selection interface.
- AI-generated campaign results page.
- Captions and hashtags display interface.
- Promotional image display interface.
- Posting schedule recommendation page.
- AI marketing chatbot interface.
- Campaign library page.
- Saved campaigns page.
- User profile management page.

3.9.2 Software Interfaces

The system interacts with the following software components:

- Flutter mobile application.
- ASP.NET Core Web API backend.
- Microsoft SQL Server database.

- AI content generation services.
- AI image generation services.
- RAG-based chatbot services.
- Cloud storage services.
- Hosting platform.

3.10 System Features

3.10.1 User Authentication Module

a. Description and Priority

This module allows users to create accounts, sign in, sign out, use Google login, and reset forgotten passwords securely.

- **Priority:** High
    - **Benefit:** Protects user accounts and allows personalized campaign storage.
    - **Penalty:** Without authentication, users cannot securely save and manage campaigns.
    - **Cost:** Moderate due to authentication and security implementation.
    - **Risk:** Weak authentication may expose user data to security risks.

b. Stimulus/Response Sequences

Stimulus: User creates a new account.

Response: The system registers the user and stores account information securely.

Stimulus: User enters login credentials.

Response: The system verifies credentials and grants access.

Stimulus: User requests password reset.

Response: The system starts a secure password reset process.

c. Functional Requirements

REQ-1: The system shall allow users to sign up using email and password.

REQ-2: The system shall allow users to sign in securely.

REQ-3: The system shall support Google login.

REQ-4: The system shall allow users to reset forgotten passwords.

REQ-5: The system shall protect user sessions and account data.

3.10.2 AI Campaign Generation Module

a. Description and Priority

This module enables users to generate complete AI-powered marketing campaigns based on structured inputs such as product details, campaign goal, target audience, selected platform, language, style, emotion, occasion, and call-to-action preferences.

The module supports both single-product and multi-product campaign generation. It generates different campaign variations, including Product-Focused, Campaign-Focused, and Lifestyle-Focused posts.

- **Priority:** High
    - **Benefit:** Enables users to create professional campaigns quickly without advanced marketing experience.
    - **Penalty:** Without this feature, users must create all campaign content manually.
    - **Cost:** Moderate to high due to AI integration and prompt engineering.
    - **Risk:** Inaccurate AI output may affect campaign quality.

b. Stimulus/Response Sequences

Stimulus: User enters campaign details and product information.

Response: The system generates AI-powered campaign content.

Stimulus: User adds multiple products to one campaign.

Response: The system processes all selected products and generates one organized campaign.

Stimulus: User selects a platform.

Response: The system customizes generated content according to platform requirements.

Stimulus: User selects English or Arabic.

Response: The system generates campaign content in the selected language.

C. Functional Requirements

REQ-1: The system shall generate AI-powered marketing campaigns based on user inputs.

REQ-2: The system shall support single-product and multi-product campaign creation.

REQ-3: The system shall support bilingual campaign generation in English and Arabic.

REQ-4: The system shall generate captions based on product details, marketing style, and selected platform.

REQ-5: The system shall generate three post variations for each campaign.

REQ-6: The system shall allow users to regenerate campaign content when required.

3.10.3 Hashtag Generation Module

a.Description and Priority

This module generates platform-specific hashtags based on product category, campaign type, target platform, and marketing context.

- **Priority:** High
    - **Benefit:** Improves content discoverability and audience reach.
    - **Penalty:** Without hashtag generation, users may struggle to select effective hashtags manually.
    - **Cost:** Moderate due to AI processing and platform rules.
    - **Risk:** Irrelevant hashtags may reduce campaign effectiveness.

b.Stimulus/Response Sequences

Stimulus: User generates a campaign.
Response: The system generates hashtags suitable for the selected platform.
Stimulus: User changes the selected platform.
Response: The system adjusts hashtag quantity and style according to the platform.

c.Functional Requirements

REQ-1: The system shall generate AI-powered hashtags.

REQ-2: The system shall generate platform-specific hashtags.

REQ-3: The system shall generate up to 20 hashtags for Instagram.

REQ-4: The system shall generate up to 6 hashtags for Facebook.

REQ-5: The system shall generate up to 5 hashtags for TikTok and Websites/Ads.

3.10.4 Promotional Image Generation Module

a.Description and Priority

This module generates AI-powered promotional images for marketing campaigns using campaign context, product details, and selected creative style.

- **Priority:** High
    - **Benefit:** Enhances visual engagement and campaign attractiveness.
    - **Penalty:** Without image generation, campaigns may lack visual impact.
    - **Cost:** High due to AI image generation service dependency.
    - **Risk:** External API failure or slow response may affect user experience.

b.Stimulus/Response Sequences

Stimulus: User requests image generation.

Response: The system generates a promotional image.

Stimulus: User regenerates campaign content.

Response: The system generates updated visual content based on the new inputs.

c.Functional Requirements

REQ-1: The system shall generate AI-powered promotional images.

REQ-2: The system shall generate images based on campaign context.

REQ-3: The system shall store generated images for future access.

REQ-4: The system shall integrate with external AI image generation services.

3.10.5 Campaign Optimization and Posting Recommendation Module

a. Description and Priority

This module provides posting schedule recommendations for different platforms to help users publish content at suitable times.

- **Priority:** High
    - **Benefit:** Improves campaign performance by suggesting optimal posting times that increase audience engagement and reach. (Rating: 9)
    - **Penalty:** Without this feature, users may publish content at

ineffective times, reducing engagement and marketing efficiency. (Rating: 8)

- **Cost:** Moderate due to rule-based or AI-based scheduling analysis.(Rating: 6)
- **Risk:** Incorrect or generalized recommendations may reduce campaign effectiveness. (Rating: 5)

b. Stimulus/Response Sequences

Stimulus: User generates a marketing campaign.

Response: The system displays recommended posting schedules.

Stimulus: User changes the selected platform.

Response: The system updates posting recommendations.

c. Functional Requirements

REQ-1: The system shall provide posting time recommendations for each platform.

REQ-2: The system shall display posting recommendations after campaign generation.

REQ-3: The system shall support platform-based scheduling suggestions.

3.10.6 User Profile Management Module

a. Description and Priority

This module allows users to view and update their account information and manage personal settings.

- **Priority:**Medium
  - **Benefit:** Improves personalization and user account control.
  - **Penalty:** Without this feature, users cannot manage account details easily.
  - **Cost:** Low to moderate.
  - **Risk:**Incorrect profile updates may affect user experience.

b. Stimulus/Response Sequences

Stimulus: User opens profile page.

Response: The system displays user information.

Stimulus: User updates profile details.

Response: The system saves updated information.

c. Functional Requirements

REQ-1: The system shall allow users to view profile information.

REQ-2: The system shall allow users to update profile details.

REQ-3: The system shall maintain user-related settings securely.

3.10.7 AI Chatbot Module(RAG-Based)

a. Description and Priority

This module provides a Retrieval-Augmented Generation (RAG) based AI Marketing Chatbot Assistant that helps users with marketing strategies, branding, content ideas, customer engagement, sales improvement, and campaign optimization.

- **Priority:** High
    - **Benefit:** Provides instant intelligent marketing assistance.
    - **Penalty:** Without the chatbot, users may lack guidance in improving campaigns.
    - **Cost:** Moderate to high due to AI and RAG integration.
    - **Risk:** Incorrect or hallucinated responses may lead to misleading marketing decisions.

b. Stimulus/Response Sequences

Stimulus: User asks for marketing advice.
Response: System generates AI-based recommendation using RAG.

Stimulus: User requests hashtags.

Response: System generates optimized hashtags.

Stimulus: User asks for content ideas.

Response: The chatbot generates relevant marketing ideas.

c. Functional Requirements

REQ-1: The system shall provide an AI chatbot for marketing assistance.
REQ-2: The chatbot shall support English and Arabic interaction.
REQ-3: The chatbot shall answer questions related to marketing, branding, sales, and customer engagement.
REQ-4: The chatbot shall provide campaign improvement suggestions.
REQ-5: The chatbot shall retrieve relevant knowledge before generating responses when using RAG.

3.10.8 Campaign Library Module

a. Description and Priority

This module allows users to save, retrieve, manage, and reuse previously generated marketing campaigns including captions, hashtags, promotional images, and posting schedules.

- **Priority:** Medium
    - **Benefit:** Enables users to reuse successful campaigns and maintain structured marketing history. (Rating: 8)
    - **Penalty:** Without this feature, users lose previous campaign data and must regenerate content repeatedly. (Rating: 6)
    - **Cost:** Low to moderate due to database storage and retrieval mechanisms. (Rating: 4)
    - **Risk:** Data inconsistency or synchronization issues may affect saved campaign integrity. (Rating: 4)

b. Stimulus/Response Sequences

Stimulus: User saves campaign.
Response: System stores campaign data in database.

Stimulus: User opens campaign library.

Response: System displays stored campaign history.

Stimulus: User deletes campaign.

Response: The system removes the campaign from the library.

c. Functional Requirements

REQ-1 :The system shall allow saving generated campaigns.
REQ-2: The system shall store captions, hashtags, images, and schedules.
REQ-3: The system shall allow users to retrieve saved campaigns.
REQ-4: The system shall allow deletion of campaigns.

3.10.9 Non-functional Requirements

Performance Requirements

- The system shall support multiple users generating campaigns at the same time.
- AI text generation shall respond within acceptable time limits.
- Image generation shall complete within acceptable latency limits.
- The mobile application shall remain responsive during backend and AI processing.

Safety Requirements

- The system shall protect user data from loss or unauthorized access.

- Regular backups shall be performed to prevent data loss.
- AI outputs should be reviewed or filtered to avoid unsafe, irrelevant, or inappropriate content.

Security Requirements

- The system shall implement secure authentication and authorization.
- All communications shall be encrypted using HTTPS/TLS.
- Users shall only access their own campaigns, products, and generated data.
- Passwords shall be stored securely using hashing techniques.
- API requests shall be protected against unauthorized access.

Usability Requirements

- The system shall provide a simple and intuitive mobile interface.
- The system shall guide users through campaign creation steps clearly.
- The application shall be suitable for users with limited marketing experience.

Reliability Requirements

- The system shall handle errors gracefully.
- The system shall provide meaningful messages when AI services are unavailable.
- Saved campaign data shall remain consistent and accessible.

Maintainability Requirements

- The system shall use modular architecture to simplify updates and maintenance.
- Backend, frontend, database, and AI services should be separated into manageable components.

Scalability Requirements

- The system shall support future growth in users, campaigns, products, and AI requests.
- The system architecture shall allow future integration with more platforms and AI models.

3.10.10 Software Quality Attributes

- **Usability:** The system shall provide a simple, and user-friendly mobile interface (Flutter-based).
- **Reliability:** AI services (text, image, chatbot) shall operate consistently and accurately.
- **Maintainability:** The modular architecture shall support future updates and improvements.
- **Scalability:** The system shall handle increasing numbers of users, campaigns,

products, and AI requests.

- **Performance**: System responses shall be optimized for real-time user interaction.
- **Security:** The system shall protect user accounts, campaign data, and API communication
- **Availability:** The system shall be accessible through the internet when backend services are running.

3.10.11 Business Rules

- Users must authenticate before creating or saving campaigns.
- Campaign generation requires campaign details and at least one product.
- Each campaign belongs to a single user account.
- Each campaign may include one product or multiple products.
- AI-generated outputs depend strictly on user-provided inputs.
- Each campaign may include multiple generated variations.
- Posting recommendations depend on the selected platform.
- Users can only access their own saved campaigns.
- Generated campaigns may include captions, hashtags, images, and schedules.

3.11 Analysis Phase

3.11.1 Use Case Diagram

The diagram will include:

3.11.1.1 Actors

a. **User** – Registers, logs in, creates campaigns through the campaign creation wizard, uploads product images,generates AI-powered marketing campaigns, views generated campaign results, accesses the campaign library, views campaign details, deletes campaigns, uses the AI Chatbot, and manages personal profile information including editing information and changing password.

b. **AI Service** – Generates AI-powered marketing campaign content including advertisement text, hashtags, images, and posting schedule suggestions.

c. **AI Chatbot Service** – Processes marketing-related questions submitted through the chatbot interface and generates AI-powered responses in Arabic or English based on general marketing knowledge.

3.11.1.2 Use Cases

1. **UC-01: Register / Login / Manage Authentication** *(User)*
2. **UC-02: View Home Page** *(User)*
3. **UC-03: Create Campaign (Main Wizard)** *(User)*

Includes the following 6 internal UCs:

- **UC-03.1: Enter Basic Info** *(User)*
- **UC-03.2: Select Platform (Audience)** *(User)*
- **UC-03.3: Enter Creative Info** *(User)*
- **UC-03.4: Enter Product Info** *(User)*
- **UC-03.5: Upload Product Images** *(User)*
- **UC-03.6: Confirm & Submit Campaign** *(User)*

4. **UC-04: Generate AI Content** *(AI Service / User)*
5. **UC-05: View Generated Campaign Results** *(User)*
6. **UC-06: View Posting Schedule** *(User)*
7. **UC-07: View Content Library** *(User)*
8. **UC-08: View Campaign Details** *(User)*
9. **UC-09: Delete Campaign** *(User)*
10. **UC-10: Use AI Chatbot** *(AI Chatbot Service / User)*
11. **UC-11: View User Profile** *(User)*
12. **UC-12: Edit Profile** *(User)*
13. **UC-13: Edit Password** *(User)*

Use Case Tables

UC-01: Register / Login / Manage Authentication

Use Case ID	**UC-01**
Use Case Name	Register / Login / Manage Authentication
Actors	User
Goal	To allow the user to securely create an account, authenticate, and gain authorized access to the system.
Preconditions	1. The user has access to the application interface. 2. For login, the user must already have a registered account and provide valid credentials (email & password) for login. 3. For registration, the user must provide a unique email address and the required profile information.
Postconditions	**Success:** – User is successfully authenticated. – A valid session token is created. – User is redirected to the Home Page. **Failure:** – User remains unauthenticated. – Error message displayed (invalid email, incorrect password, already registered email, etc.)
Main Success Scenario	1. **User opens the application** and selects "Login" or "Register". 2. System displays the appropriate authentication form. 3. User enters required information: - For login: Email and Password - For registration: First Name, Last Name, Email, Password 4. System validates the submitted information. 5. 5. If login: - System checks if email exists. - System verifies password against stored hash.

	6. If registration: - System checks email uniqueness - System encrypts password and stores new user data 7. System generates a secure session token. User is redirected to the **Home Page** with authenticated privileges.
Alternative Scenarios	**A. Invalid Login Credentials:** 1. User enters wrong password or non-existing email. 2. System displays: *"Incorrect email or password."* 3. User can retry login. **B. Email Already Registered (During Registration):** 1. User enters an email that already exists in the database. 2. System displays: *"Email already in use."* 3. User chooses to login instead. **C. Missing Required Fields:** 1. User tries to submit empty fields. 2. System highlights missing required inputs. 3. User must complete all mandatory data before proceeding. **D. System Error / Authentication Service Down:** 1. System cannot validate login due to server issue. 2. System displays: *"Authentication service unavailable. Please try again later."* 3. User cannot proceed.

UC-02: View Home Page

Use Case ID	**UC-02**

Use Case Name	View Home Page
Actors	User
Goal	To allow the authenticated user to access the main dashboard where they can create campaigns, access the Campaign Library, use the AI Chatbot, and manage their profile.
Preconditions	1. User must be authenticated (successful login). 2. The application is available and operational. 3. User profile information is stored in the system. 4. Application must be running and able to retrieve initial user data.
Postconditions	**Success:** – Home Page is displayed successfully – Navigation options are available to the user. – User can access campaign creation, campaign library, AI Chatbot, and profile management features. **Failure:** – If system error occurs → fallback error screen is displayed
Main Success Scenario	1. The user logs in successfully. 2. System retrieves user profile data (name, join date). 3. System loads the Home Page layout. 4. System displays the main navigation bar including: - **Home** - **Content Library** - **Create New Campaign** - **AI Chatbot** - **Profile** 5. System displays recently created campaigns (if any). 6. User selects the next action : - **Create New Campaign** - **Open Campaign Library** - **Use AI Chatbot** - **View Profile**

Alternative Scenarios	**A. No Campaigns Found:** 1. User has no created campaigns. 2. System displays: *"You have no campaigns yet – Create your first campaign!"* 3. User may select **Create Campaign**. **B. System Fails to Load User Data:** 1. System fails to retrieve user information from the database. 2. System displays: *"Unable to load user data."* 3. User remains on an error fallback screen. **C. Slow Loading / Partial Loading:** 1. Home Page loads but campaign list takes time. 2. System displays loading placeholders until data arrives.

UC-03: Create Campaign (Main Campaign Wizard)

Use Case ID	**UC-03**
Use Case Name	Create Campaign (Main Campaign Wizard)
Actors	User
Goal	To allow the user to create a full marketing campaign through a structured multi-step wizard that collects all required campaign, audience, creative, product, and media inputs.
Preconditions	1. User must be successfully authenticated and logged into the system. 2. User must have access to the "Create Campaign" feature from the Home Page. 3.System must be able to load the 6-step wizard interface.

Postconditions	**Success:** – Campaign information is successfully collected and validated. – The campaign is submitted for AI content generation. – Campaign data is validated and stored, and The system proceeds to UC-04 Generate AI Content. **Failure:** – No campaign is saved if the user cancels or exits before completing Step 6. System displays validation errors if required fields are missing.
Main Success Scenario	**This Use Case includes 6 internal steps (<<include>> relationships).** – The user proceeds step-by-step until completing the wizard: 1. User selects **Create Campaign** from the Home Page. 2. System opens the Campaign Wizard (Step 1). 3. **UC-03.1 – Enter Basic Info** is included. 4. User presses **Next**. 5. **UC-03.2 – Select Platform (Audience)** is included. 6. User presses **Next**. 7. **UC-03.3 – Enter Creative Info** is included. 8. User presses **Next**. 9. **UC-03.4 – Enter Product Info** is included. 10. User presses **Next**. 11. **UC-03.5 – Upload Images** is included. 12. User presses **Next**. 13. **UC-03.6 – Confirm Summary** is included. 14. User reviews the final summary screen. 15. User clicks **Generate AI Campaign** on the Summary page. 16. System validates all collected campaign information. 17. System stores the campaign data. 18. System initiates UC-04 Generate AI Content.
	A. User Cancels Wizard: 1. At any step, the user selects **Cancel**. 2. Wizard closes without saving.

Alternative Scenarios	3. User is returned to the Home Page **B. Missing Required Fields:** 1. User attempts to click **Next** with incomplete information. 2. System shows validation errors (highlighted fields). 3. User corrects the inputs. 4. User proceeds successfully. **C. Technical Error During Input Saving:** 1. System fails to temporarily store step data (e.g., storage error). 2. Error message appears: *"Unable to save progress."* 3. User may retry or exit to Home Page. **D. User Navigates Back:** 1. User selects **Back** from any step except Step 1. 2. System returns to previous step **with preserved inputs**. **E. Session Timeout:** 1. User is inactive for long time while in wizard. 2. Session expires → user is logged out. 3. System displays message: *"Your session has expired. Please log in again."* 4. All unsaved campaign data is discarded.

UC-03.1: Enter Basic Info (Step 1 of Campaign Wizard)

Use Case ID	**UC-03.1**
Use Case Name	Enter Basic Info
Actors	User
Goal	To collect the essential campaign-level information such as campaign name, and campaign goal.

Preconditions	1. User is authenticated and inside the **Create Campaign Wizard** (UC-03). 2. The system has successfully loaded **Step 1 – Basic Info** interface. 3. All mandatory input fields must be displayed to the user.
Postconditions	**Success:** – Step 1 data is validated and temporarily stored in session memory / draft state. – The wizard progresses to Step 2 (UC-03.2 – Select Platform). **Failure:** – If validation fails, user remains on Step 1. – No data is saved if the user cancels the wizard.
Main Success Scenario	1. The user views **Step 1 – Basic Info** screen. 2. System displays required input fields: - Campaign Name - Campaign Goal - Target Language 3. User enters the required data into the fields. 4. User clicks **Next**. 5. System validates the inputs. 6. If all fields are valid, system stores the data in draft memory 7. System navigates the user to **Step 2 – Select Platform (Audience)**.
	A. Missing or Invalid Fields: 1. User clicks **Next** without completing all required fields. 2. System highlights missing / invalid fields (e.g., empty name, unsupported characters). 3. System displays validation message: *"Please complete all required fields before continuing."* 4. User corrects errors and proceeds successfully.

Alternative Scenarios	**B. User Cancels Campaign Creation:** 1. User clicks **Cancel** on Step 1. 2. System asks for confirmation: *"Are you sure you want to cancel campaign creation?"* 3. If user confirms → wizard closes with no data saved. 4. User is redirected back to Home Page. **C. Temporary System Failure:** 1. User completes fields and clicks **Next**. 2. System attempts to store Step 1 data. 3. Storage failure occurs (e.g., database write issue or session error). 4. System displays message: *"An unexpected error occurred. Please try again."* 5. User retries or cancels the wizard. **D. Session Timeout During Step 1:** 1. User stays inactive for extended time. 2. Session expires automatically. 3. Upon action, the system redirects user to Login Page with a notification: *"Your session has expired. Please log in again."*

UC-03.2: Select Platform (Audience) (Step 2 of Campaign Wizard)

Use Case ID	**UC-03.2**
Use Case Name	Select Platform (Audience)
Actors	User
Goal	To allow the user to define the campaign audience by selecting the target marketing platform(s) (e.g., Facebook, Instagram, TikTok), which will be used to guide AI-generated content and campaign recommendations.

Preconditions	1. User successfully completed **Step 1 – Basic Info (UC-03.1)**. 2. Step 2 Audience interface is displayed with available platforms loaded from the system. 3. Available target platform options are loaded by the system.
Postconditions	**Success:** – Selected platforms are stored in the campaign draft. – System navigates to **Step 3 – Creative Info (UC-03.3) Failure:** – If no platform is selected, the user cannot proceed to next step. – No draft data is saved if user cancels the wizard.
Main Success Scenario	1. System displays the Audience step 2. System displays all available marketing platforms. 3. User selects one platforms (e.g., Facebook + Instagram, TikTok). 4. User clicks **Next**. 5. System validates the selection. 6. The system validates the input. 7. System stores platform selection in campaign draft. 8. System loads **Step 3 – Creative Info**.
	A. No Platform Selected: 1. User clicks Next without choosing a platform. 2. System shows error: *"Please select at least one platform to continue."* 3. User selects platform(s) and proceeds **B. Invalid / Unsupported Platform:** 1. User selects a platform temporarily disabled by the system.

Alternative Scenarios	2. System shows message: *"Selected platform is currently unavailable."* 3. User selects an alternative platform and proceeds. 4. User is redirected back to Home Page. **C. User Cancels Campaign Creation:** 1. User clicks **Cancel** on Step 2. 2. System asks for confirmation: *"Are you sure you want to cancel campaign creation?"* 3. If user confirms → wizard closes with no data saved. 4. User is redirected back to Home Page. **D. Platform Loading Failure:** 1. System fails to load platform options. 2. System shows message: *"Unable to load platform options. Please try again."* 3. User may retry or cancel the wizard.

UC-03.3: Enter Creative Info (Step 3 of Campaign Wizard)

Use Case ID	**UC-03.3**
Use Case Name	Enter Creative Info
Actors	User
Goal	To collect creative preferences that guide AI-generated campaign content, including campaign tone, Call-to-Action, seasonality, occasion, and emotional appeal.
Preconditions	1. User successfully completed **Step 2 – Select Platform (UC-03.2)**. 2. Creative templates or style presets are loaded by the system.

Postconditions	**Success:** – Selected Campaign Tone, Call-to-Action, Seasonality, Occasion, and Emotion are stored in the campaign draft. – System navigates to **Step 4 – Product Info (UC-03.4)**. **Failure:** – User remains on Step 3 until all required selections are valid.
Main Success Scenario	1. System displays Step 3 creative fields (dropdowns): - **Campaign Tone(Friendly /Luxury/ Formal/ etc.)** - **Call-to-Action (Shop Now / Discover More / Buy Today / Limited Offer)** - **Seasonality (Winter, Summer, Spring / etc.)** - **Occasion (Daily /Party / Wedding /etc.)** - **Emotion (Confidence / Joy / Desire / etc.)** 2. User selects one value from each dropdown list. 3. User clicks Next. 4. System validates the selected values. 5. System stores the creative information in the campaign draft. 6. System navigates to **Step 4 – Product Info**.

Alternative Scenarios	**A. Missing Required Selections:** 1. User attempts to proceed without selecting one or more required creative options. 2. System highlights the missing fields. 3. System displays: *"Please complete all required creative selections before continuing."* 4. User completes the missing selections and proceeds successfully. **B. Invalid Characters / Excessive Length:** – System displays validation warning. **C. User Cancels Wizard:** 1. User clicks **Cancel** on Step 3. 2. System asks for confirmation: *"Are you sure you want to cancel campaign creation?"* 3. If user confirms → wizard closes with no data saved. 4. User is redirected back to Home Page.

UC-03.4: Enter Product Info (Step 4 of Campaign Wizard)

Use Case ID	**UC-03.4**
Use Case Name	Enter Product Info
Actors	User
Goal	To allow the user to enter detailed information for One or more products, including product name, category, style, material, target audience attributes, and unique selling points to support AI-generated campaign content.
Preconditions	– User successfully completed **Step 3 – Enter Creative Info (UC-03.3)**. – Step 4 – Product Info interface is successfully loaded.
Postconditions	**Success:** - The following information is stored in the campaign draft for each entered product: • **Product Name** • **Product Category**

	• **Product Sub-Category** • **Product Material** • **Product Style** • **Product Description** • **Target Gender** • **Unique Selling Point (USP)** • **Target Persona** - The system stores all entered products in the campaign draft - System navigates to **Step 5 – Upload Product Images (UC-03.5)**. **Failure:** - Validation errors prevent transition to the next step.
Main Success Scenario	1. System displays Step 4 – Product Info. 2. System displays the following product fields: - **Product Name (text field)** - **Product Category (dropdown list) Examples: Apparel, Footwear, Accessories, etc.** - **Product Sub-Category (dropdown list) Examples: Dresses, Abayas, Pants, Outerwear, Heels, Sandals, etc.** - **Product Material (text field)** - **Product Style (dropdown list) Examples: Casual, Formal, Sporty, Luxury, Modest, Trendy, etc.** - **Product Description (text area)** - **Target Gender (dropdown list) Examples: Women, Men, Unisex** - **Unique Selling Point (USP) (text field)** - **Target Persona (dropdown list) Examples: Sweet, Elegant, Athleisure, Minimalist, Bohemian, Sustainable, etc.** 3. User enters the required information for the first product. 4. User may click **Add New Product**. 5. The system displays a new empty Product Information form. 6. User enters the required information for the additional

	product. 7. Steps **4–6** are repeated until the user finishes adding all desired products. 8. User clicks Next 9. System validates the entered data. 10. System stores the product information in the campaign draft. 11. System navigates to **Step 5 – Upload Product Images (UC-03.5)**.
Alternative Scenarios	**A. Missing Required Product Information:** 1. User attempts to proceed without completing the required fields for one or more products. 2. System highlights missing fields. 3. System displays: *"Please complete all required product information before continuing."* 4. User completes the missing information and proceeds successfully. **B. Unsupported Characters:** 1. User enters unsupported characters in one or more product fields. 2. System displays a validation warning. 3. User corrects the invalid input and proceeds successfully. **C. User Adds Multiple Products** 1. User clicks **Add New Product**. 2. System displays another empty Product Information form. 3. User enters the product information. 4. User repeats this process until all desired products have been added. 5. System includes all entered products in the campaign draft. **D. User Cancels Wizard:** 1. User clicks **Cancel** on Step 4.

	2. System asks for confirmation: *"Are you sure you want to cancel campaign creation?"* 3. If user confirms → wizard closes with no data saved. 4. User is redirected back to Home Page. **E. User Navigates Back:** 1. User clicks Back. 2. System returns to Step 3 – Creative Info. 3. Previously entered information for **all products** remains available.

UC-03.5: Upload Product Images (Step 5 of Campaign Wizard)

Use Case ID	**UC-03.5**
Use Case Name	Upload Product Images
Actors	User
Goal	To allow the user to upload **one or more images for each entered product** that will be used by the AI to generate promotional campaign content.
Preconditions	1. User successfully completed **Step 4 – Enter Product Info (UC-03.4).** 2. At least **one product** has been added to the campaign draft. 3. The image upload component is available. 4. Supported image formats are configured by the system.
Postconditions	**Success:** – Images are uploaded and associated with their corresponding products in the campaign draft. – The system stores the uploaded image references for all products. – System navigates to **Step 6 – Confirm & Submit Campaign (UC-03.6)**. **Failure:** – Validation errors prevent transition to the next step.
	1. System displays Step 5 – Upload Product Images.

Main Success Scenario	2. System displays each previously entered product with its own image upload section. 3. For each product, the system displays: - Product Name - Image upload area - Supported formats and upload instructions 4. User uploads one or more images for the first product. 5. System validates the uploaded images (JPG/PNG only). 6. System uploads the images and displays image previews. 7. The system automatically moves to the next product. 8. Steps **4–7** are repeated until images have been uploaded for all entered products. 9. User reviews all uploaded images. 10. User clicks Next. 11. System stores the image references for all products in the campaign draft. 12. System navigates to **Step 6 – Confirm & Submit Campaign (UC-03.6).**
Alternative Scenarios	**A. Unsupported File Type:** 1. User uploads an unsupported file type. 2. System reject and shows: *"Only JPG and PNG formats are allowed."* 3. User selects a supported image and proceeds successfully. **B. File Too Large:** 1. User uploads an image exceeding the allowed size. 2. System rejects the upload. 3. System displays the maximum allowed file size. 4. User uploads a smaller image. **C. Upload Failure:** 1. System fails to upload one or more images due to a network or server error. 2. System displays: *"Image upload failed. Please try again."* 3. User retries the upload successfully.

	D. Missing Product Images: 1. User clicks Next without uploading an image for one or more products. 2. System highlights the product(s) missing images. 3. System displays: *"At least one product image is required."* 4. User uploads one or more images. 5. User proceeds successfully. **E. User Cancels Wizard:** 1. User clicks **Cancel** on Step 5. 2. System asks for confirmation: *"Are you sure you want to cancel campaign creation?"* 3. If user confirms → wizard closes with no data saved. 4. User is redirected back to Home Page. **F. User Navigates Back** 1. User clicks **Back**. 2. System returns to **Step 4 – Enter Product Info**. 3. Previously entered product information and uploaded images remain available.

UC-03.6: Confirm & Submit Campaign (Step 6)

Use Case ID	**UC-03.6**
Use Case Name	Confirm & Submit Campaign
Actors	User
Goal	To allow the user to review all campaign information entered in Steps 1–5, including one or more products and their uploaded images, optionally return to any step for modifications, and generate the AI campaign content.
Preconditions	1. User successfully completed All previous steps. 2. Campaign draft contains all required information. 3. Summary page is successfully loaded.
Postconditions	**Success:** **– If Generate pressed:** - System triggers **UC-04 Generate AI Content.**

	- AI-generated campaign content is created successfully. - Campaign is automatically saved to the Content Library. - User is redirected to View Generated Campaign Results (UC-05). **Failure:** – Campaign generation is not completed. – Appropriate error message is displayed.
Main Success Scenario	1. System displays a campaign summary containing : - Basic Information - Audience / Target Platforms - Creative Information - Information for each entered product, including: - Product Details - Uploaded Images - Edit option 2. The Basic Information, Audience, Creative Information, and each Product section contain an Edit option. 3. User reviews the campaign information. 4. User may select Edit beside any summary section or any product. 5. System navigates directly to the corresponding wizard step while preserving previously entered data. 6. User updates information and returns to the Summary page. 7. User clicks Generate AI Campaign. 8. System validates all campaign data, including all entered products and their uploaded images. 9. System stores the complete campaign information, including all products and their associated images. 10. System triggers UC-04: Generate AI Content. 11. AI-generated campaign content is successfully created. 12. System automatically saves the generated campaign to the Content Library. 13. System redirects the user to View Generated Campaign Results (UC-05).
	A. Edit Campaign Information: 1. User clicks Edit beside a summary section or a product. 2. System opens the corresponding wizard step. 3. Previously entered values remain available. 4. User updates the information. 5. User returns to the Summary page. 6. Updated campaign or product information is displayed on the Summary page.

Alternative Scenarios	**B. AI Generation Failure:** 1. System fails to generate campaign content. 2. System displays: *"Only JPG and PNG form Unable to generate campaign content. Please try again."* 3. User may retry generation. **C. User Cancels Campaign Creation:** 1. User leaves the Summary page before generation. 2. System warns the user that progress may be lost. 3. User confirms cancellation. 4. Campaign draft is discarded. 5. User is returned to the Home Page.

UC-04: Generate AI Content

Use Case ID	**UC-04**
Use Case Name	Generate AI Content
Actors	User, AI Service
Goal	To generate three AI-powered marketing advertisement variants for each entered product based on the campaign information provided by the user, including one or more products and their associated images. Each generated advertisement includes marketing text, hashtags, and an AI-generated promotional image tailored to a specific marketing focus.

Preconditions	1. The user must be authenticated and logged in. 2. A campaign must be created and all 6 steps must be completed successfully: - Basic Info entered - Audience selected - Creative Info added - One or more products have been added. - Images have been uploaded for all entered products. - User is currently viewing Step 6 – Campaign Summary. 3. The system must have internet access to communicate with the AI Generation Service. 4. All required fields used for AI generation must be present and valid.
Postconditions	**Success:** – Three AI-generated advertisement variants are created successfully for each entered product: - Product-Focused Advertisement - Lifestyle-Focused Advertisement - Campaign-Focused Advertisement – Each generated advertisement contains: - Marketing Caption - Hashtags - AI-Generated Image – The generated campaign, including all products and their generated advertisements, is automatically saved to the Content Library. – The user is redirected to View Generated Campaign Results (UC-05). **Failure:** – No campaign content is generated. – The system displays an appropriate error message.

Main Success Scenario	1. User completes Step 6 – **Campaign Summary.** 2. The user clicks **"Generate AI Content"**. 3. The system validates all campaign information. 4. The system sends a generation request to the **AI Service** containing: - Basic campaign Information - Target Platforms - Creative Preferences - Information for all entered products - Uploaded images associated with each product 5. AI Service analyzes the complete campaign data, including all products and their associated images. 6. AI Service generates three advertisement variants: - Product-Focused Advertisement - Lifestyle-Focused Advertisement - Campaign-Focused Advertisement 7. For each advertisement variant, the AI Service generates: - Marketing Caption - Relevant Hashtags - Promotional Image 8. System receives the generated campaign content. 9. System stores the generated campaign and advertisements associated with all entered products in the database. 10. System automatically saves the campaign to the Content Library. 11. System redirects the user to **View Generated Campaign Results (UC-05)**.
	A. Missing or Invalid Data:

Alternative Scenarios	1. The system detects incomplete or invalid campaign information. 2. The system shows an error: ***"Campaign information incomplete. Please review the creation steps."*** 3. The system redirects the user back to the missing step. **B. AI Service Timeout:** 1. AI service does not return a response within the allowed time 2. The system shows a message: ***"AI generation is taking longer than expected. Please try again later."*** 3. User may reattempt generation. **C. AI Service Error:** 1. AI Service returns an error (e.g., API failure, rate limit, invalid format). 2. The system logs the error. 3. The system displays: ***"AI generation failed. Please try again."*** **D. Campaign Save Failure:** 1. AI content is generated successfully. 2. System fails to save the generated campaign. 3. System displays: ***"Generated content could not be saved. Please try again."*** 4. User may retry the generation process.

UC-05: V iew Generated Campaign Results

Use Case ID	**UC-05**

Use Case Name	View Generated Campaign Results
Actors	User
Goal	To allow the user to view the generated campaign results, review the AI-generated advertisements created for each product, copy advertisement text, save advertisement images, view the posting schedule, or return to the Content Library.
Preconditions	1. The user must be logged into the system. 2. Generate AI Content must exist for the campaign (UC-04 completed successfully). 3. The AI Service must have successfully generated three advertisement variants for each entered product. 4. The generated campaign must be automatically saved to the Content Library.
Postconditions	**Success:** – The generated campaign results for all entered products are displayed successfully. – User may: - Copy advertisement text for any generated advertisement. - Save any generated advertisement image. - View Posting Schedule Suggestions – The campaign remains stored in the Content Library. – **Failure:** – The user's action fails due to system errors; an appropriate error message is displayed. – Results cannot be displayed; an appropriate error message is shown.
	1. System receives the generated campaign from UC-04. 2. System automatically saves the generated campaign to the Content Library. 3. System displays the Generated Campaign Results page. 4. System displays the generated advertisements grouped by product. 5. For each entered product, the system displays three

Main Success Scenario	advertisement variants: • **Product-Focused Advertisement** • **Lifestyle-Focused Advertisement** • **Campaign-Focused Advertisement** 6. For each generated advertisement, the system displays: • **Generated Caption** • **Generated Hashtags** • **Generated Promotional Image** 7. For each advertisement, the system displays: • **Copy Text button** • **Save Image button** 8. System displays: • **View Posting Schedule button** • **Back to Library button** 9. User reviews the generated advertisements for all entered products. 10. User may: • **Copy text from any generated advertisement.** • **Save any generated advertisement image.** • **Click View Posting Schedule to generate and view posting schedule suggestions.** • **Return to the Content Library** 11. The use case ends when the user leaves the results page.
	A. Copy Content: 1. User clicks **Copy Text**. 2. The system copies the advertisement caption and hashtags to the clipboard. 3. A confirmation toast appears: **"Content copied successfully."** **B. Save Image:** 1. User clicks **Save Image**.

Alternative Scenarios	2. The system downloads the generated advertisement image. 3. The image is saved to the user's device. 4. A confirmation message appears: **"Image Saved Successfully."** **C. View Posting Schedule:** 1. User clicks View Posting Schedule 2. System generates posting schedule suggestions based on the generated campaign. 3. System displays the Posting Schedule page. 4. UC-06: View Posting Schedule is triggered. **D. Return to Content Library:** 1. User clicks **back to Library.** 2. The system opens the Content Library page. 3. The generated campaign is displayed in the library because it was automatically saved after generation. **E. Results Loading Failure:** 1. The system fails to retrieve the generated campaign results. 2. The system displays: **"Unable to load campaign results. Please try again later."** 3. The user remains on the current page.

UC-06: View Posting Schedule

Use Case ID	**UC-06**
Use Case Name	View Posting Schedule
Actors	User
Goal	To allow the user to view AI-generated posting schedule suggestions for the three advertisement variants generated for each entered product before returning to the Generated Campaign Results page.

Preconditions	1. The user must be authenticated and logged into the system 2. UC-04 (Generate AI Content) must be completed successfully. 3. UC-05 (View Generated Campaign Results) must be currently displayed. 4. The AI Service must have generated posting schedule suggestions for the campaign.
Postconditions	**Success:** – The posting schedule suggestions are displayed successfully. – The user can review the recommended publishing dates and times for the three advertisement variants generated for each entered product. – The user may return to the Generated Campaign Results page. – Posting schedule suggestions are available only during the current generated campaign session. **Failure:** – If Posting schedule information cannot be displayed. – An appropriate error message is shown.

Main Success Scenario	1. The user is viewing the Generated Campaign Results page (UC-05). 2. The user clicks the View Posting Schedule button. 3. The system opens the Posting Schedule page. 4. The system requests posting schedule recommendations from the AI Service. 5. The AI Service generates posting schedule recommendations for the **three advertisement variants of each entered product**. 6. The system displays the posting schedule recommendations **grouped by product**. 7. The system displays posting recommendations for: - Product-Focused Advertisement - Lifestyle-Focused Advertisement - Campaign-Focused Advertisement 8. For each advertisement, the system displays: - Recommended posting date - Recommended posting time - Suggested platform 9. The system does not store the posting schedule recommendations in the Content Library. 10. The user reviews the schedule recommendations. 11. The user clicks the Back Arrow. 12. The system returns the user to the Generated Campaign Results page.
	A. Schedule Data Unavailable: 1. U The user clicks View Posting Schedule 2. The system cannot retrieve schedule recommendations. 3. System displays: *"Posting schedule is currently unavailable".* 4. The user returns to the Generated Campaign Results page

Alternative Scenarios	**B. AI Service Error:** 1. The system requests schedule data. 2. AI Service fails to return the schedule. 3. The system displays: *"Unable to load posting schedule. Please try again later".* 4. The user remains on the current page. **C. User Returns Without Viewing Entire Schedule** 1. The user opens the Posting Schedule page. 2. The user clicks the Back Arrow immediately. 3. The system returns to the Generated Campaign Results page. 4. No campaign data is modified.

UC-07: View Campaign Library

Use Case ID	**UC-07**
Use Case Name	View Campaign Library
Actors	User
Goal	To allow the user to access and view all previously generated campaigns stored in the Content Library, open campaign details, or delete existing campaigns.
Preconditions	1. The user must be authenticated (logged in). 2. The user must have generated at least one campaign that has been automatically stored in the Campaign Library, or the system displays an empty state. 3. The system must have access to stored campaigns in the Campaign Library entity/table.

Postconditions	**Success:** – The user successfully views the campaign library – The system displays all stored campaigns with their associated metadata. – The user may: • Open a campaign and view its details (UC-08). • Delete a campaign (UC-09). • Return to other system pages. **Failure:** – If the system encounters an error, it displays a message: **"Unable to load library content. Please try again."**
Main Success Scenario	1. The user navigates to the **Content Library** page via the navigation bar. 2. The system retrieves all campaigns associated with the authenticated user. 3. The system displays the Campaign Library in list/grid format showing: • Campaign Name • Creation Date • Campaign Status • Delete Icon 4. The user browses the available campaigns. 5. The user selects a campaign from the Campaign Library. 6. The system opens the selected campaign. 7. UC-08 (View Campaign Details) is triggered. 8. The use case ends.
Alternative Scenarios	**A. Empty Library:** 1. The user opens the Campaign Library. 2. The system finds no generated campaigns associated with the user. 3. The system displays the message: ***"No campaigns available yet. Create your first campaign to get started."***

	4. The user may navigate to Create Campaign. **B. System Error:** 1. The system fails to retrieve campaigns from the Campaign Library. 2. The system displays the message: *"Unable to load campaign library. Please try again later."* 3. The user remains on the Campaign Library page. **C. View Campaign Details:** 1. The user clicks on a campaign entry. 2. The system opens the selected campaign. 3. The system displays the campaign details page. 4. UC-08 (View Campaign Details) is triggered. **D. Delete Campaign:** 1. The user clicks the Delete icon next to a campaign. 2. The system displays a confirmation dialog: *"Are you sure you want to delete this campaign?This action cannot be undone."* 3. The user confirms deletion. 4. The system permanently deletes: •Campaign information •Generated advertisements •Generated images 5. The system refreshes the Campaign Library. 6. The system displays: *"Campaign deleted successfully."*

UC-08: View Campaign Details

Use Case ID	**UC-08**
Use Case Name	View Campaign Details
Actors	User

Goal	To allow the user to view the details of a selected campaign, including all entered products and their AI-generated advertisements, captions, hashtags, and generated images, with options to copy advertisement text and save advertisement images.
Preconditions	1. The user must be authenticated (logged in). 2. The selected campaign must exist in the Campaign Library. 3. The campaign must be selected from UC-07 (View Campaign Library). 4. The system must have access to the campaign data and generated advertisement results.
Postconditions	**Success:** – The user successfully views all relevant details of the selected campaign. – The system displays the campaign metadata (name, goal, audience, dates, platform), all entered products, and all generated advertisements associated with each product. – The user may copy advertisement text or save advertisement images. **Failure:** – If the campaign does not exist or cannot be retrieved, the system shows: **"Campaign details not available. Please try again."**
Main Success Scenario	1. The user opens the Campaign Library (UC-07). 2. User selects a campaign to view its details. 3. The system retrieves the campaign information, including: - Campaign Name - Campaign Goal - Audience & Platform - Creative Settings - Products Information, including for each product: (Name, Description, Gender, USP, Target Persona) - Upload Images 4. The system displays the generated campaign results grouped by product.

	5. The system displays three advertisement variants for each entered product: - Product-Focused Advertisement - Lifestyle-Focused Advertisement - Campaign-Focused Advertisement 6. For each generated advertisement, the system displays: - Generated Caption - Generated Hashtags - Generated Promotional Image 7. For each generated advertisement, the system provides: - Copy Text button - Save Image button 8. The user reviews the generated advertisements for all entered products. 9. The user may copy text or save images for any generated advertisement. 10. The use case ends when the user leaves the campaign details page.
Alternative Scenarios	**A. Campaign Not Found:** 1. User selects a campaign that was deleted or does not exist. 2. System displays error: **"Campaign details not available."** 3. User returns to the previous page. **B. System Error During Retrieval:** 1. User selects campaign. 2. System fails to retrieve campaign data due to a server/database error. 3. System displays: **"Unable to load campaign details. Try again later."** **C. Copy Advertisement Text:** 1. The user clicks the Copy Text button on an advertisement. 2. The system copies the advertisement caption and hashtags to the clipboard.

	3. The system displays: ***"Content copied successfully."*** **D. Save Advertisement Image:** 1. The user clicks the Save Image button. 2. The system downloads the selected advertisement image. 3. The image is saved to the user's device. 4. The system displays: ***"Image saved successfully."***

UC-09: Delete Campaign

Use Case ID	**UC-09**
Use Case Name	Delete Campaign
Actors	User
Goal	To allow the user to permanently delete an existing campaign and all associated generated content from the Campaign Library.
Preconditions	1. The user must be authenticated (logged in). 2. The selected campaign must exist in the Campaign Library. 3. The user must own the campaign being deleted. 4. The system must have access to the campaign and associated generated content records.

Postconditions	**Success:** – The selected campaign is permanently removed from the database. – All associated generated advertisements, generated images, and campaign-related records are removed. – The Campaign Library is updated automatically. – System shows confirmation message: **"Campaign deleted successfully."** **Failure:** – If campaign deletion fails, the system displays: **"Unable to delete campaign. Please try again later."**
Main Success Scenario	1. User navigates to the **Campaign Library (UC-07)**. 2. System displays all campaigns belonging to the authenticated user. 3. User locates the desired campaign. 4. User clicks the **Delete** icon next to the campaign. 5. System displays a confirmation popup: ***"Are you sure you want to delete this campaign?*** ***This action cannot be undone."*** 6. User clicks **Confirm Delete**. 7. System validates that the user owns the selected campaign. 8. System permanently deletes: - Campaign record - All generated variants under that campaign - Any associated images - Associated campaign data 9. System updates the Campaign Library. 10. The deleted campaign no longer appears in the library. 11. System shows success message: **"Campaign deleted successfully."**

Alternative Scenarios	**A. User Cancels Deletion:** 1. User clicks **Delete icon.** 2. Confirmation dialog appears. 3. User clicks **Cancel**. 4. System closes the dialog. 5. Campaign remains unchanged. 6. User stays on the **Campaign Library** page. **B. System Error During Deletion:** 1. User confirms deletion. 2. System encounters a database or server error. 3. Campaign is not deleted. 4. System displays: ***"Unable to delete campaign. Please try again later."*** 5. Campaign remains unchanged. 6. User remains on the Campaign Library page. **C. Campaign Not Found** *1. User attempts to delete a campaign.* 2. System cannot find the selected campaign record. 3. System displays: ***"Campaign not found."*** 4. No data is deleted. **D. Unauthorized Deletion Attempt** 1. User attempts to delete a campaign they do not own. 2. System rejects the request. 3. System displays: ***"You do not have permission to delete this campaign."*** 4. Campaign remains unchanged.

UC-10: Use AI Chatbot

Use Case ID	**UC-10**

Use Case Name	Use AI Chatbot
Actors	User / Ai Chatbot Service
Goal	To allow the user to interact with the AI Chatbot by asking marketing-related questions in Arabic or English and receiving AI-generated responses based on general marketing knowledge.
Preconditions	1. The user must be authenticated (logged in). 2. The AI Chatbot page must be accessible from the application. 3. AI Chatbot Service must be available. 4. Internet connectivity must be available.
Postconditions	**Success:** – User receives a response from the AI Chatbot. – Chat history is displayed in the conversation window. – User may continue asking additional questions in Arabic or English. **Failure:** – No chatbot response is generated. – An appropriate error message is displayed. – Chat session remains available for retry.
Main Success Scenario	1. User navigates to the AI Chatbot page. 2. System displays the chatbot interface. 3. System displays: - Message input field - Send button 4. User enters a marketing-related question in Arabic or English. 5. User clicks Send. 6. System validates the question. 7. System forwards the request to the AI Chatbot Service. 8. AI Chatbot Service processes the user's question and

	generates a response in the same language. 9. AI Chatbot Service generates a response. 10. System receives the generated response. 11. System displays the chatbot reply in the conversation window. 12. User reviews the response. 13. User may continue the conversation by submitting additional questions in Arabic or English. 14. Use case ends when the user leaves the chatbot page.
Alternative Scenarios	**A. Empty Question** 1. User clicks Send without entering a question. 2. System displays: ***"Please enter a question."*** 3. No request is sent to the AI Chatbot Service. **B. AI Chatbot Service Unavailable** 1. User submits a question. 2. AI Chatbot Service cannot process the request. 3. System displays: ***"The chatbot service is currently unavailable. Please try again later."*** 4. User may retry later. **C. Response Generation Failure** 1. User submits a question. 2. AI Chatbot Service encounters an unexpected error. 3. System displays: ***"Unable to generate a response. Please try again."*** 4. Chat session remains active.

	D. Network Connection Lost 1. User submits a question. 2. Network connection is interrupted. 3. System displays: ***"Connection lost. Please check your internet connection."*** 4. User may retry after reconnecting.

UC-11: View User Profile

Use Case ID	**UC-011**
Use Case Name	View User Profile
Actors	User
Goal	To allow the user to view their personal account information, including basic profile details and membership data.
Preconditions	1. User must be logged in. 2. The system must have access to the user's stored profile information in the **User** table.
Postconditions	**Success:** – The system displays the profile information correctly. – Profile page displayed with first name, last name, email, join date, and sub-actions. – User can proceed to edit their profile or change password. **Failure:** – If an error occurs, system shows: **"Unable to load your profile information. Please try again later."**

Main Success Scenario	1. The user clicks the **Profile** icon in the navigation bar OR selects “View Profile” from the home page. 2. The system retrieves the user’s account details from the database, including: - First Name - Last Name - Email - Join Date 3. The system displays the profile page clearly with all personal information. 4. The user reviews their profile. 5. The profile page also shows two buttons: - **Edit Profile** - **Change Password** 6. The use case ends successfully.
Alternative Scenarios	**A. System Error:** 1. User selects “View Profile.” 2. System fails to fetch profile details. 3. System displays error message. 4. User stays on the current page. **B. Missing Data:** 1. Some profile fields (like last name) are missing. 2. System displays available information and marks missing fields as “Not provided.”

UC-12: Edit Profile

Use Case ID	**UC-12**
Use Case Name	Edit Profile

Actors	User
Goal	To allow the user to update their personal information, such as first name, last name, and email.
Preconditions	1. User must be logged in. 2. User must be on the **User Profile** page. 3. The system must be able to retrieve and update user data in the **User** table.
Postconditions	**Success:** – Updated profile data is saved to the system. – The user receives a confirmation message. – The updated information appears immediately in the profile page. **Failure:** – System fails to save changes and displays an error message. – No data is changed in the database.
Main Success Scenario	1. The user clicks **Edit Profile** from the Profile page. 2. The system displays an editable form populated with the current profile information: - First Name - Last Name - Email 3. The user updates one or more fields. 4. The user clicks **Save Changes**. 5. The system validates the new data: - Email must be unique and valid format. - Required fields must not be empty. 6. The system updates the profile information in the database. 7. System displays: **"Profile updated successfully."**

	8. Updated profile page is shown.
Alternative Scenarios	**A. Validation Error:** 1. User enters invalid email format / leaves required fields empty. 2. System displays a validation message: - "Invalid email format." - "First name cannot be empty." 3. User corrects data and retries. **B. Email Already Exists:** 1. User enters an email used by another account. 2. System displays: **"This email is already registered."** 3. User must enter a different one. **C. System Error During Save:** 1. User clicks **Save Changes**. 2. System error occurs while updating data. 3. System displays: **"Failed to update profile. Please try again."** 4. No changes are saved.

UC-13:Edit Password

Use Case ID	**UC-13**
Use Case Name	Edit Password
Actors	User
Goal	To allow the user to securely change their account password to maintain account security.

Preconditions	1. User must be logged in. 2. User must be on the **User Profile** page. 3. The system must be able to validate the current password and update the new password in the database.
Postconditions	**Success:** – The user successfully updates the account password. – The system stores the new password securely (hashed). – A confirmation message is displayed: **"Password updated successfully."** **Failure:** – If the current password is incorrect, system shows an error. – If the new password does not meet security requirements, system rejects it. – No changes are applied to the account password on failure.
Main Success Scenario	1. The user clicks **Edit Password** on the Profile page. 2. The system displays a form requesting: - Current password - New password - Confirm new password 3. The user enters the required information. 4. The user clicks **Save Changes**. 5. The system validates the input: - Checks if the current password matches the stored password. - Ensures the new password meets security rules (e.g., minimum length, uppercase, number). - Confirms that the new password matches the confirmation field. 6. If validation passes, the system updates the password in the database securely (hashed). 7. The system displays a success message: **"Password updated successfully."**

	8. Updated profile page is shown.
Alternative Scenarios	**A. Incorrect Current Password:** 1. User enters the wrong current password. 2. System displays: **"Current password is incorrect."** 3. User retries **B. New Password Validation Fails:** 1. New password does not meet requirements (length, complexity). 2. System displays: **"New password does not meet security requirements."** 3. User corrects password and retries. **C. New Password and Confirmation Do Not Match:** 1. User enters mismatching new password and confirmation 2. System displays: **"New password and confirmation do not match."** 3. User corrects and retries **D. System Error During Update:** 1. User submits valid password change. 2. System encounters an internal error. 3. System displays: **"Password update failed. Please try again."** No changes are applied

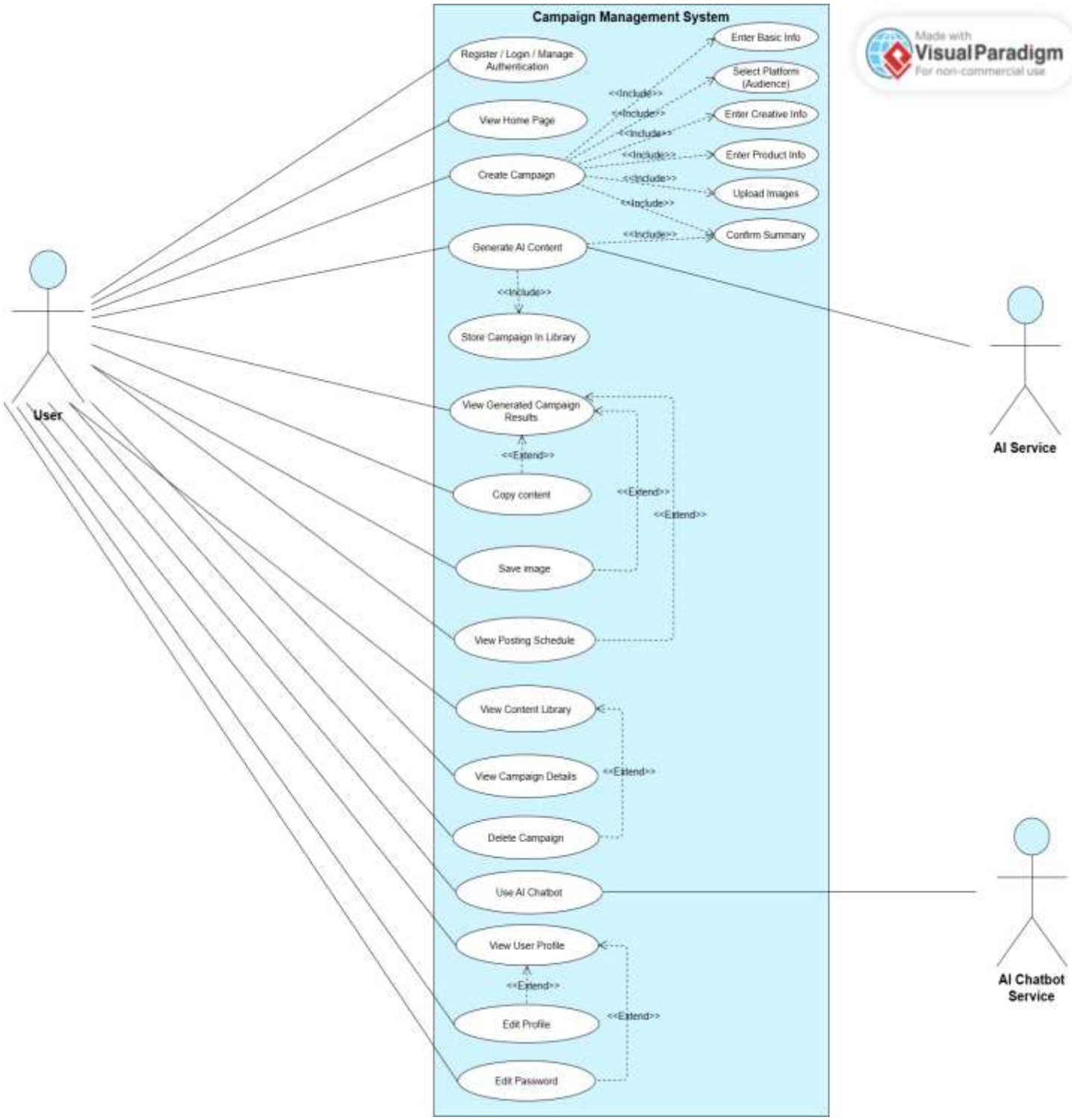


Figure 2 UseCase Diagram

3.11.2 Class Diagram

This class diagram illustrates the object-oriented structure, core entities, attributes, and operational methods within the **AD GENERATOR AI** system. It maps the visual behaviors defined in the Activity and Sequence diagrams into concrete software classes, displaying how data is encapsulated and manipulated across the system layers (Mobile UI, API Backend, AI Engine, and Database).

3.11.2.1 Classes and Their Attributes/Methods

1.User

- **Attributes:**
  - UserID: int (PK)
  - FirstName: string
  - LastName: string
  - Email: string (Unique)
  - PasswordHash: string
  - JoinDate: DateTime
- **Methods:**
  - SignUp(firstName, lastName, email, password): void — POST /auth/register
  - SignIn(email, password): string — POST /auth/login
  - Logout(): void — POST /auth/logout
  - DeleteAccount(): bool — DELETE /user

2.Dashboard

- **Attributes:**
  - TotalCampaigns: int
  - ActiveCampaigns: int
  - EndedCampaigns: int
  - TotalProducts: int
- **Methods:**
  - LoadCampaignStats(): void — GET /dashboard
  - DisplayDashboardOverview(): void

3. Product

- **Attributes:**
  - ProductID: int (PK)
  - ProductName: string
  - Category: string
  - SubCategory: string
  - Material: string
  - Style: string

- ProductDescription: text
- TargetGender: string
- USP: string
- TargetPersona: string
- ImageURL: string

- **Methods:**
  - EnterProductDetails(): void
  - UploadProductImages(imageFile): string — POST /upload/images

4. Campaign

- **Attributes:**
  - CampaignID: int (PK)
  - CampaignName: string
  - CampaignGoal: string
  - TargetLanguage: string
  - Platform: string
  - Status: string
  - GeneratedAt: DateTime
  - EndDate: DateTime
- **Methods:**
  - SaveCampaignToDatabase(): bool
  - DisplayCampaignDetails(): void
  - DeleteCampaign(campaignId): bool — DELETE /campaign/{id}

5. AI Module

- **Attributes:**
  - EngineVersion: string
  - PromptTemplate: string
- **Methods:**
  - GenerateCampaignVariants(campaignParams, productData, imageUrls): List
  - CalculateBestPostingTimes(campaignId): List

6.AI_Generated_Content

- **Attributes:**
  - ContentID: int (PK)
  - VariantType: string
  - ContentText: text
  - Ideal_Hashtags: List<string>
  - GeneratedImageURL: string
  - Ad_Tone: string

- Season: string
- GeneratedDate: DateTime

- **Methods:**
  - CopyText(): void
  - SavePost(): bool
  - PostingSchedule

7.PostingSchedule

- **Attributes:**
  - ScheduleID: int (PK)
  - BestTimes: List<string>
  - Tips: text
- **Methods:**
  - ViewPostingSchedule(): void

8.AIChatbot

- **Attributes:**
  - ChatSessionID: string
  - MarketingKnowledgeBase: string
- **Methods:**
  - OpenAIChat(): void
  - TypeMarketingQuestion(message): void
  - SendToRAGSystem(message): string
  - DisplayStructuredResponse(): void

System Relationships + Paths

User → Product (1:N)
Path: /user → /product (create, manage products)

1. User → Campaign (1:N)
   - Path: /user → /campaign (create, view campaigns)
2. Product → Campaign (1:N)
   - Path: /product → /campaign (assign product to campaign)
3. Campaign → AI_Generated_Content (1:N)
   - Path: /campaign/generate → /generated-content (3 variants output)
4. Campaign → PostingSchedule (1:1)
   - Path: /campaign/{id}/schedule
5. AIModule (API Engine)
   - Path: POST /campaign/generate
   - Path: GET /schedule/{campaignId}
6. AIChatbot (RAG System)
   - Path: POST /chat/message

Database Layer

- Path: SQL Server (Users, Products, Campaigns, Content, Schedule)

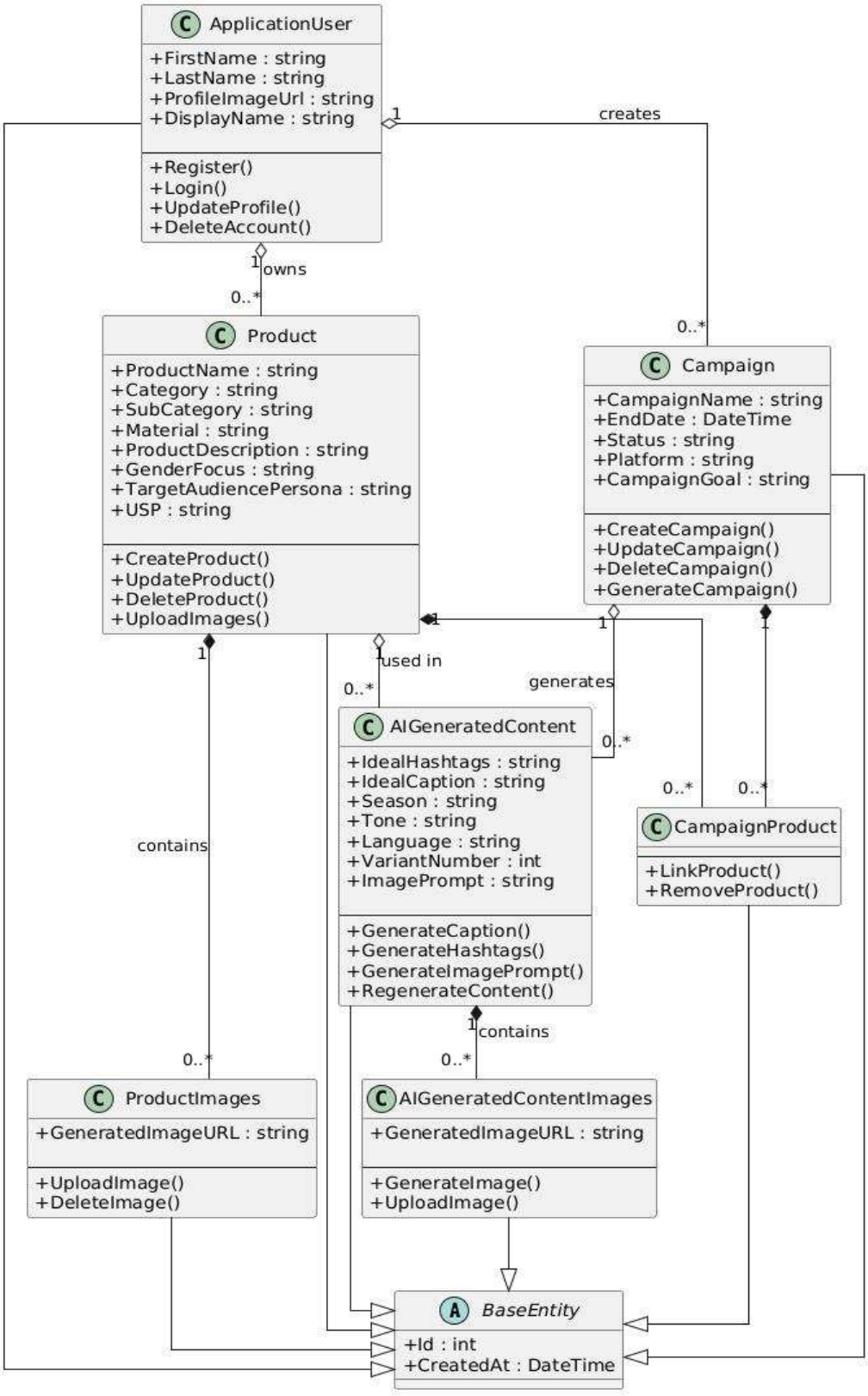


Figure 3 Class Diagram

3.11.3 Sequence Diagram

This sequence diagram depicts the chronological workflow, message exchanges, and dynamic interactions among the end user, (mobile application) frontend, backend API, AI processing engine, and the database within the **AD GENERATOR AI** System. It maps out how structural requests are routed, authorized, persisted, and processed through generative models.

3.11.3.1 Actors and Lifelines

1**. User**: The marketing client or business owner who triggers operational requests through the mobile view.

2**. Mobile App (Flutter):**The client-side application frontend responsible for capturing inputs, managing state, executing local logic, and rendering visual results.

3. **ASP.NET Core API:** The centralized backend server acting as a secure gateway for application routing, transaction orchestration, database access, and token validation.

4. **AI Module (Python):** The external analytical and generative core executing Large Language Model (LLM) prompting, content multi-variant orchestration, and Retrieval-Augmented Generation (RAG) lookups.

5. **Database (SQL Server):** The relational persistence layer handling permanent reads and writes for configurations, user profile states, and transactional logs.

3.11.3.2 Processes and Interactions

1. Authentication Process

- Sign Up Flow: The User submits registration metrics via the interface. The Mobile App routes a `POST /auth/register` payload to the ASP.NET Core API, which persists the client parameters directly into SQL Server and issues a validation confirmation message.
- Sign In Flow: The User inputs active credentials. The Mobile App dispatches a `POST /auth/login` request. The API executes verification lookups against SQL Server records; upon identity match, a secure, cryptographically signed JWT Token is passed back to authorize subsequent device communication channels.
- Reset Password Flow: In the event of failed Sign In credentials, the User is presented with a "Forgot Password" option as a recovery path. Tapping this redirects the Mobile App to a dedicated Reset Password screen, where the User inputs a New Password and Confirm Password. The Mobile App dispatches a POST /auth/reset-password request carrying the updated credential payload to the ASP.NET Core API. The API validates the new password against the same security requirements, then commits the updated credentials into SQL Server, overwriting the prior stored hash. Upon confirmation of the successful update, the API returns a success response, and the Mobile App routes the User back to the Sign-In screen to authenticate with the newly reset password.
- Google Sign In/Sign Up Flow: The User initiates the alternative authentication path by tapping "Continue with Google." The Mobile App redirects to the Google OAuth consent layer, where the User authorizes profile data sharing. Google returns a signed token

alongside profile metrics (Name, Email) to the Mobile App, which forwards this token via a POST /auth/google request to the ASP.NET Core API. The API validates the token's authenticity directly against Google's verification servers, then queries SQL Server to determine account existence — provisioning a new account from the Google profile data if no match is found, or retrieving the existing record otherwise. A JWT Token is then issued back to authorize subsequent device communication channels, identical in function to the standard Sign In flow.

2. Loading Dashboard Metrics

- Upon successful authentication landing, the Mobile App automatically fetches tracking stats using `GET /dashboard`.
- The API inquiries the Database to aggregate statistics including *Total Campaigns, Active Campaigns, Ended Campaigns,* and *Total Products*.
- This packed payload returns down the stack to render the centralized multi-metric visual overview directly on the User's dashboard display.

3. Multi-Step Campaign Configuration & Asset Uploading

- The User navigates through sequential input steps within the creation wizard (Base Setup, Target Platform selection, Creative Setting definitions, and Product specifications).
- Image Upload Phase: When digital media assets are selected, the Mobile App executes a multipart `POST /upload/images` request. The API saves metadata paths into SQL Server, and streams secure, operational Image URLs back to the mobile view context.

4. Core AI Content Generation Pipeline

- The User initiates final generation processing by tapping the execution button. The Mobile App issues a unified `POST /campaign/generate` call to the ASP.NET Core API layer.
- The API proxies parameters to the Python AI Module engine.
- The AI Engine leverages specialized prompt structures to execute inference loops, generating exactly three specialized marketing variants: Post 1 (Product Focused), Post 2 (Lifestyle Focused), and Post 3 (Campaign Focused).
- The generated text blocks, tag structures, and asset paths return to the API layer, which writes the operational structures permanently to SQL Server and reflects successful multi-variant results on the User's device interface for clipboard coping or layout preservation actions.

5. Optimal Posting Schedule Lookups

- The User requests strategic placement metric recommendations. The Mobile App performs a targeted `GET /schedule/{campaignId}` lookup.
- The API triggers processing functions within the Python AI Module, which calculates channel peak interaction times to deliver an engagement grid divided into ranked priorities (Priority 1, 2, 3) coupled with execution advice.

6. Campaign Library Maintenance & Local Processing

- Accessing the library triggers a `GET /campaigns` call down to SQL Server, pulling the stored marketing histories categorized by lifecycle status flags.

- Local Filtering:Searching variations by string queries occurs strictly within the Flutter application container state, removing structural latency burdens from back-end pipelines.
- Campaign Deletion Lifecycle: Activating the deletion option commands Flutter to invoke a `DELETE /campaign/{id}` request. The API executes drop actions across linked relational tables inside SQL Server, returning a success notification to drop the element view instantly.
- Multi-Product Addition: During the Product specification step, the User completes the standard product fields (Name, Category, Sub-Category, Material, Style, Description, Gender, USP, Persona) for a single item. Rather than exiting the wizard, the User may tap "Add Another Product," which appends the current entry to an in-memory products array and re-surfaces a blank product form for the next item — avoiding the need to repeat the Basic Info, Audience, and Creative steps for each individual product. This loop repeats for as many products as the User wishes to group under the same campaign. Only once the User declines to add further products and proceeds through Upload and Summary does the complete products array get bundled into a single POST /campaign/generate request, ensuring the AI Module generates variants for all products together under one unified campaign rather than scattering them across multiple separate campaigns.

7. RAG Marketing Chatbot Conversational Streams

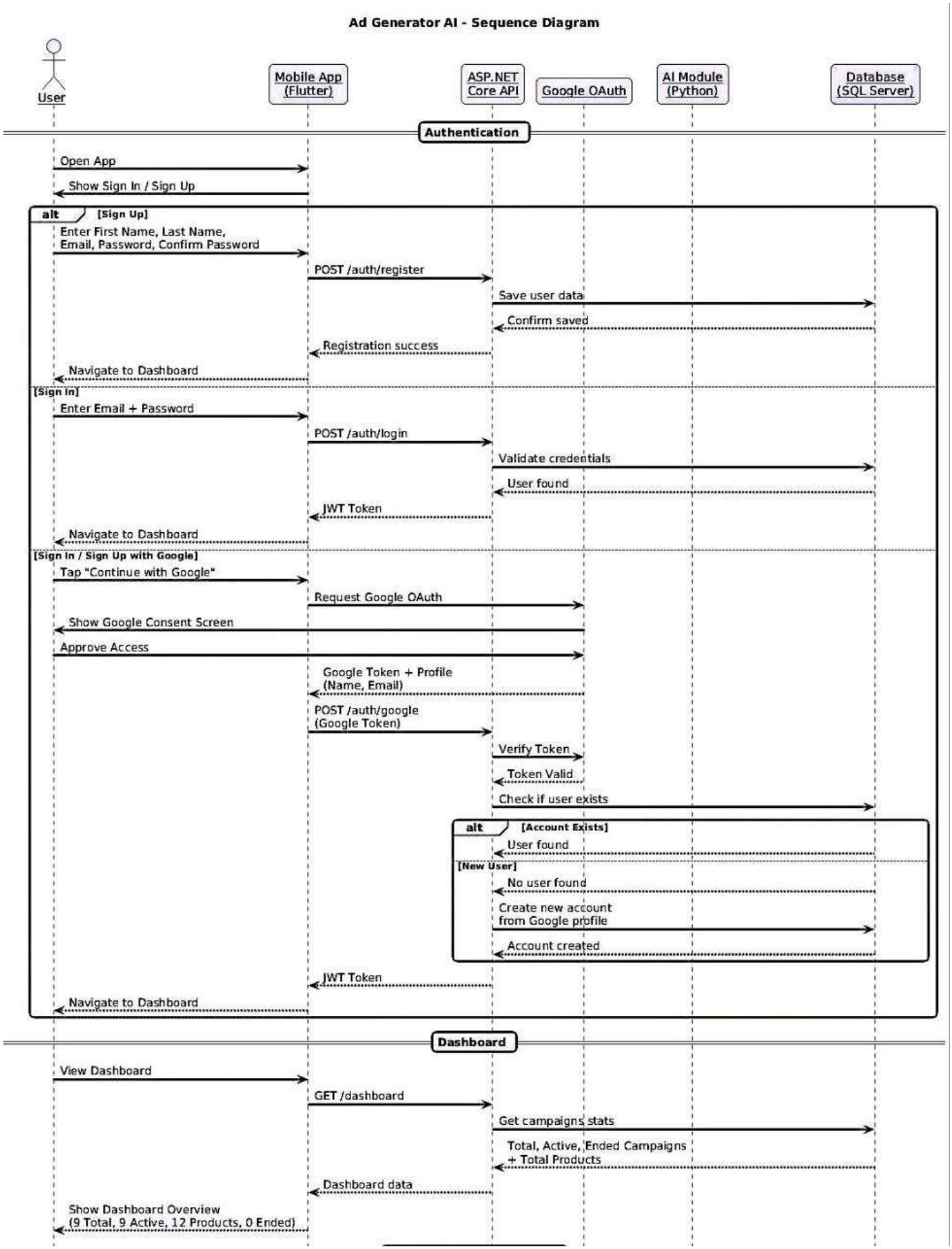
- Opening the conversational section instantiates the interaction layer, returning an automated system-wide welcoming prompt to the interface.
- The User submits targeted operational textual questions. The Mobile App forwards data packages through a `POST /chat/message` gateway request.
- The API invokes the semantic indexing framework inside the Python AI Module, which routes contextual search embeddings into its local Marketing Knowledge Base.
- The system yields an advice block structurally categorized into *Target Audience, Ad Objective, Call to Action,* and *Testing* parameters, which the backend formats for unified rendering inside the Chat UI stream.

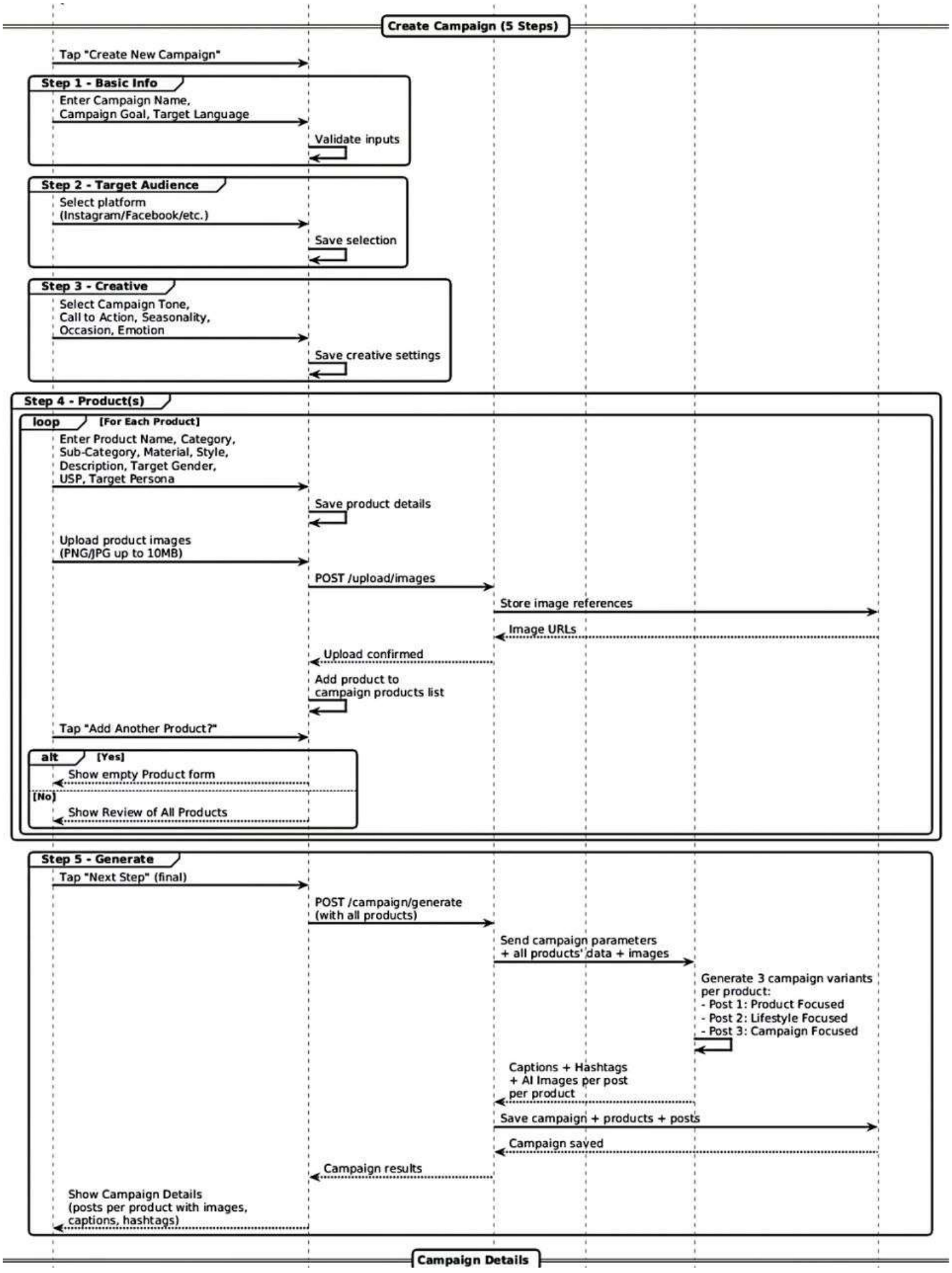
8. Profile Discovery and Session Termination

- Accessing account configuration sections invokes a `GET /profile` inquiry to populate basic verified administrative metrics (Name and Email) from the database layout.
- Tapping logout pushes a standard `POST /auth/logout` structural cleanup request, dismantling active JWT operational tokens and shifting UI states safely back onto initial login screen structures.

Key Alternate Flows

- Local UI Optimization: Basic campaign list searches and text filtering bypass backend routing completely, managing lookup parameters locally inside Flutter memory states to optimize response times.
- Image Reference Decoupling: Heavy asset media handling uses an isolated multi-part transaction sequence (`/upload/images`) to separate slow file system operations from rapid core relational data processing.


Ad Generator AI - Sequence Diagram
User
Mobile App (Flutter)
ASP.NET Core API
Google OAuth
AI Module (Python)
Database (SQL Server)
Authentication
Open App
Show Sign In / Sign Up
alt [Sign Up]
Enter First Name, Last Name, Email, Password, Confirm Password
POST /auth/register
Save user data
Confirm saved
Registration success
Navigate to Dashboard
[Sign In]
Enter Email + Password
POST /auth/login
Validate credentials
User found
JWT Token
Navigate to Dashboard
[Sign In / Sign Up with Google]
Tap "Continue with Google"
Request Google OAuth
Show Google Consent Screen
Approve Access
Google Token + Profile (Name, Email)
POST /auth/google (Google Token)
Verify Token
Token Valid
Check if user exists
alt [Account Exists]
User found
[New User]
No user found
Create new account from Google profile
Account created
JWT Token
Navigate to Dashboard
Dashboard
View Dashboard
GET /dashboard
Get campaigns stats
Total, Active, Ended Campaigns + Total Products
Dashboard data
Show Dashboard Overview (9 Total, 9 Active, 12 Products, 0 Ended)


Create Campaign (5 Steps)
Tap "Create New Campaign"
Step 1 - Basic Info
Enter Campaign Name,
Campaign Goal, Target Language
Validate inputs
Step 2 - Target Audience
Select platform
(Instagram/Facebook/etc.)
Save selection
Step 3 - Creative
Select Campaign Tone,
Call to Action, Seasonality,
Occasion, Emotion
Save creative settings
Step 4 - Product(s)
loop
[For Each Product]
Enter Product Name, Category,
Sub-Category, Material, Style,
Description, Target Gender,
USP, Target Persona
Save product details
Upload product images
(PNG/JPG up to 10MB)
POST /upload/images
Store image references
Image URLs
Upload confirmed
Add product to
campaign products list
Tap "Add Another Product?"
alt
[Yes]
Show empty Product form
[No]
Show Review of All Products
Step 5 - Generate
Tap "Next Step" (final)
POST /campaign/generate
(with all products)
Send campaign parameters
+ all products' data + images
Generate 3 campaign variants
per product:
- Post 1: Product Focused
- Post 2: Lifestyle Focused
- Post 3: Campaign Focused
Captions + Hashtags
+ AI Images per post
per product
Save campaign + products + posts
Campaign saved
Campaign results
Show Campaign Details
(posts per product with images,
captions, hashtags)
Campaign Details

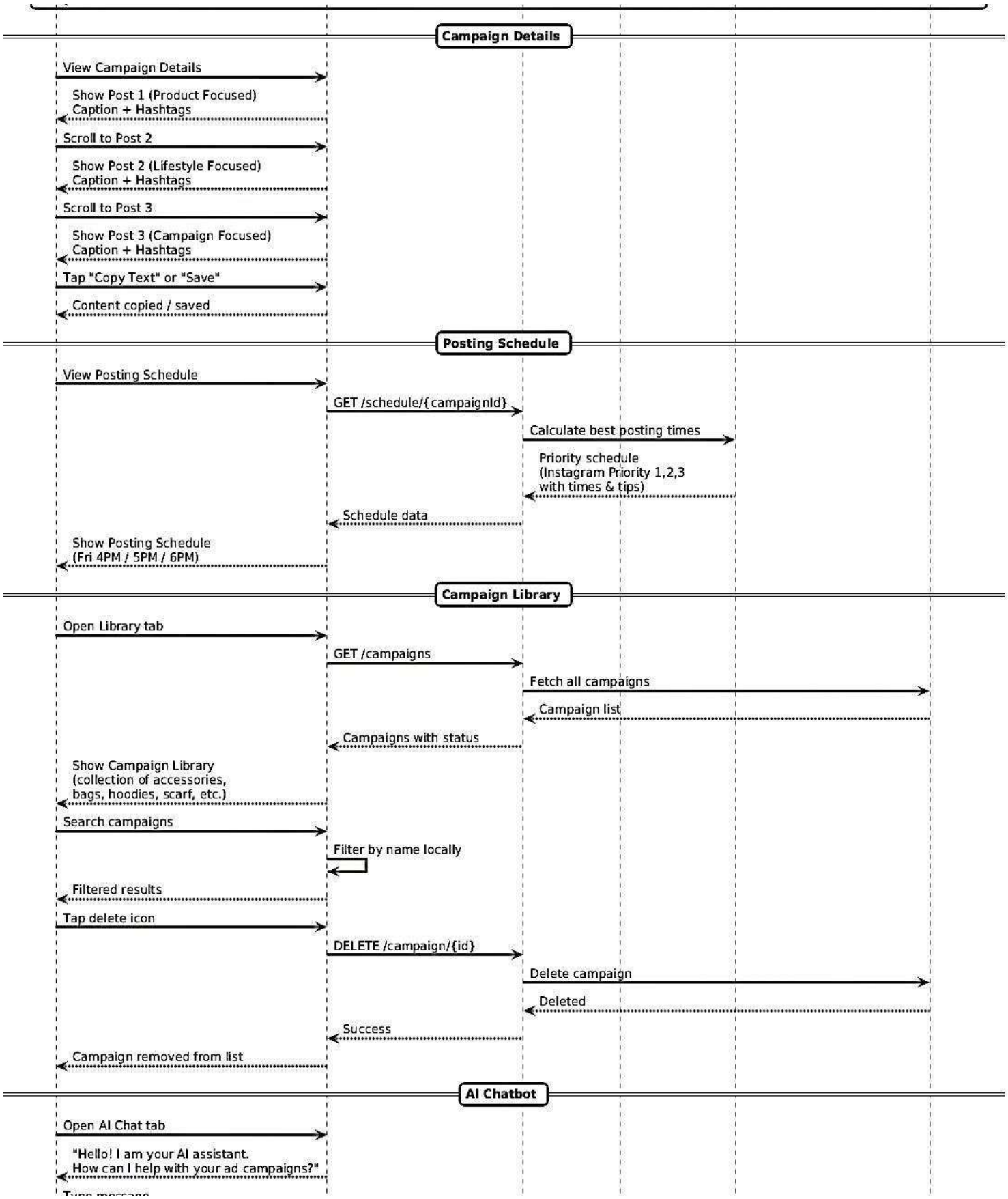


Figure4 SequenceDiagram

3.11.4 Activity Diagram

This Activity Diagram outlines the workflow of the (**Ad Generator AI** Campaign Generator)system, detailing how users authenticate, navigate the dashboard, create AI-driven marketing campaigns, manage their campaign library, interact with an AI chatbot, and manage their profile settings. Here's a detailed explanation:

1. Start of the Process

- *The black circle at the top center represents the entry point where the user opens the application ("Open App").

2. Initial Steps (Authentication)

- The workflow splits into two primary paths for user access:
- Sign In: The user enters their Email and Password. The system validates the credentials; if valid, a JWT Token is generated to grant access. If invalid, an error message is displayed.
- If the entered credentials are invalid, the user may alternatively tap "Forgot Password" to enter and confirm a new password, resetting their credentials before being routed back to the Sign-In screen.s
- Sign Up: New users enter their First Name, Last Name, Email, Password, and Confirm Password. The system checks if the password meets security requirements (8 characters, uppercase, lowercase, number, special char). If it passes, the account is created; otherwise, a validation error is shown.
- Sign In with Google: As a third authentication path, the user may tap "Continue with Google" to be redirected to the Google OAuth consent screen. Upon successful authorization, the system retrieves the user's Google profile (Name, Email) and checks whether an account already exists; if so, the user is logged into their existing account, otherwise a new account is automatically created from the Google profile data. Either case concludes with a JWT Token being generated, just as in the standard Sign In flow.
- All three successful paths (Sign In, Sign Up, and Google Authentication) merge to proceed to the main system.

3. Dashboard Navigation

- Once authenticated, the user is navigated to the Dashboard, which automatically loads campaign statistics, including Total, Active, Ended, and Products metrics. From this central hub, the workflow forks into four main parallel modules.
- Create Campaign (Step-by-Step Generation)

This is the core multi-step wizard of the system:

- Step 1 - Basic Info: The user enters the Campaign Name, Goal, and Target Language.
- Step 2 - Target Audience:The user selects the target social media platform (e.g., Instagram,

Facebook, etc.).

- Step 3 - Creative The user defines the creative direction by selecting Tone, CTA (Call to Action), Seasonality, Occasion, and Emotion.
- Step 4 - Product: The user inputs comprehensive product details (Product Name, Category, Sub-Category, Material, Style, Description, Gender, USP, Persona) and may tap "Add Another Product" to repeat this entry process for multiple items — grouping several products under the same campaign rather than creating a separate campaign for each one. Once no further products are needed, the user proceeds to the next step.
- Step 5 - Upload: The user uploads product images (supporting PNG/JPG up to 10MB).
- Step 6 - Generate: All collected data is sent to the AI Module, which concurrently processes and generates three distinct post types:
  - Post 1 (Product Focused): Caption + Hashtags + Image
  - Post 2 (Lifestyle Focused): Caption + Hashtags + Image
  - Post 3 (Campaign Focused): Caption + Hashtags + Image
- Post-Generation Actions:The generated campaign is saved to the database and details are displayed. The user can then perform actions such as Copy Text, Save Post, or View Posting Schedule (which shows priority 1, 2, 3 timings with best times & tips).

5. Campaign Library Management

- Users can view their historical data by loading all saved campaigns.
- Search/Filter: The system checks if the user is searching; if yes, campaigns are filtered by name before displaying the list along with their current status (Active/Ended).
- Management Actions: For any selected campaign, the user can either:
- View the specific posts and variations.
- Delete the campaign, which prompts a confirmation before permanently removing it from the database and updating the library view.

6. AI Chatbot Interaction

- Users can open a dedicated AI Chat interface where they receive an automated welcome message.
- The user types a marketing-related question, which is sent to the RAG AI System to retrieve contextual answers from the Marketing Knowledge Base.
- The system displays a structured AI response. The user can choose to continue chatting (looping back to type another message) or exit the chat module.

7. Profile Settings

- Users can view their full account name and email.
- From here, two distinct account control paths are available:
- Logout: Clears the session token and redirects the user back to the Sign-In screen.
- Delete Account:Prompts for a confirmation; if confirmed, the system removes all user data from the database and terminates the session.

8. End of the Process

- The workflow safely concludes at the final termination node (the encircled black dot) at the very bottom of the diagram when an active user session is intentionally ended via logout, account deletion, or completion of management tasks.

General Observations

- Parallel Processing: The diagram highlights the platform's multi-functional capability by allowing users to switch dynamically between creating campaigns, browsing libraries, chatting with AI, or modifying settings.
    - AI-Driven Automation: It clearly maps how raw user inputs and uploaded assets are structurally handed over to an external AI module to produce multi-perspective marketing outputs simultaneously.
    - Data Integrity:Explicit decision diamonds ensure security tokens, validation parameters, and database deletion steps are executed conditionally.

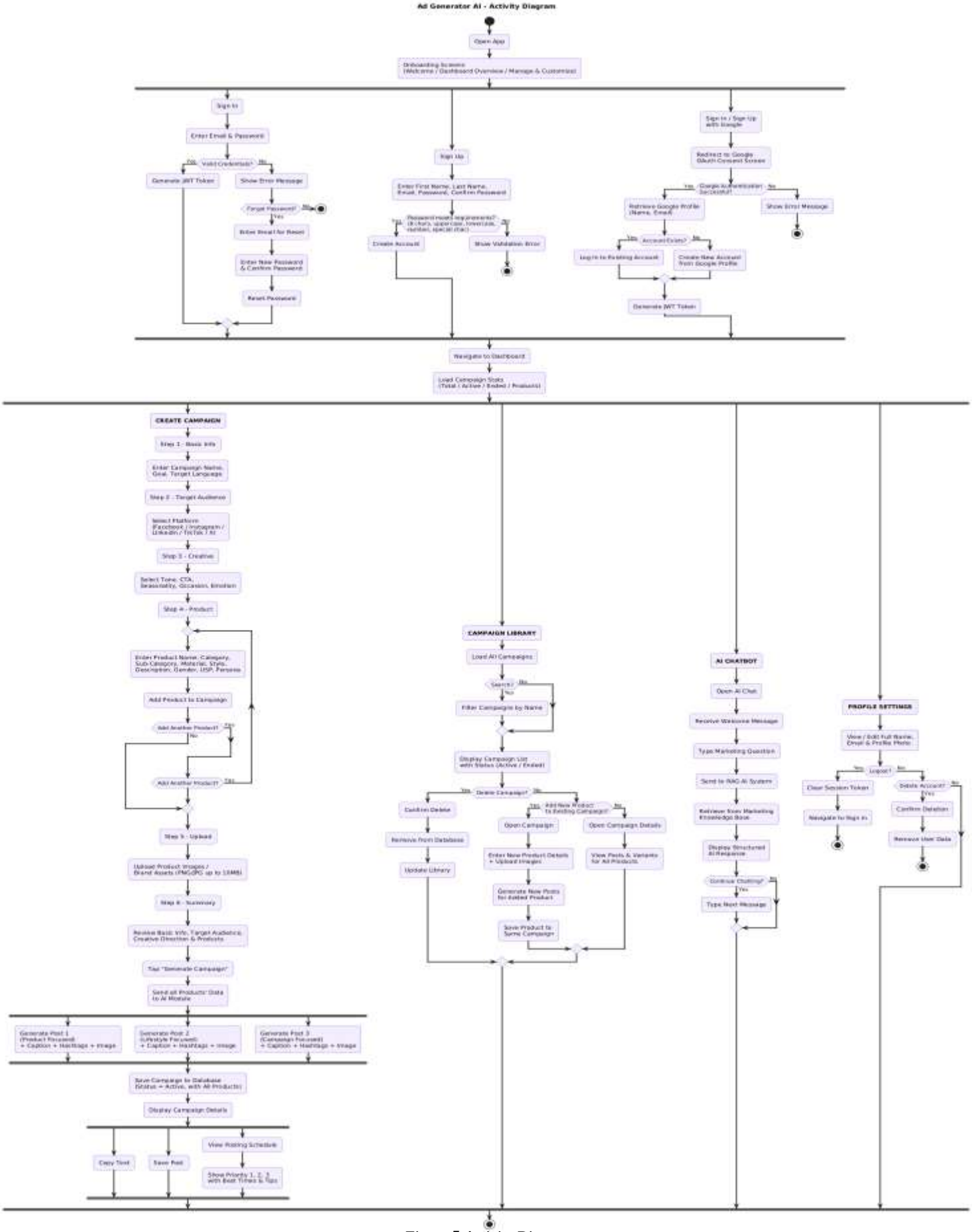


Figure 5 Activity Diagram

3.11.5 Entity Relationship Diagram (ERD)

This **Entity-Relationship Diagram (ERD)** visualizes the database schema for the Ad Generator AI Campaign Generator system. It describes the data structures required to manage users, products, marketing campaigns, and AI-generated content, along with their attributes and operational cardinalities.

3.11.5.1 Entities and Attributes

1.1. User

Description: Represents the registered users or marketers utilizing the platform.

- Attributes:
  - UserID (Primary Key): Unique identifier for each user.
  - Name (Composite Attribute): Composed of Fname (First Name) and Lname (Last Name).
  - Email (Unique Key / Derived Multi-line Oval): The user's unique authentication email address.
  - PasswordHash: Secured hash of the user's password.
  - join_Date: The timestamp when the user registered.

1.2. Product

Description: Represents the commercial item, service, or brand for which advertisements are created.

- **Attributes:**
  - ProductId (Primary Key): Unique identifier for each product.
  - Product_Name: The commercial name of the product.

Product_Description: Detailed text describing the product.

  - Target_Audience_Persona: Profiles or demographics targeted by the product.
  - ImageURL (Multi-valued Attribute / Double Oval): References for product source images.
  - Gender_Focus: Specifies the gender demographic target.
  - USP (Unique Selling Proposition): The distinct feature that sets the product apart.
  - CreatedAt: Timestamp when the product profile was added.

1.3. Campaign

Description: Represents a specific marketing initiative set up by a user for a specific product.

- Attributes:
  - CampaignID (Primary Key): Unique identifier for the campaign.
  - Campaign_Name: Title assigned to the marketing campaign.
  - Campaign_Goal: The objective of the campaign (e.g., Brand Awareness, Conversions).

- Platform: Targeted social network distribution channels (e.g., Instagram, Facebook).
- GeneratedAt: Timestamp indicating when the campaign assets were compiled.
- EndDate: The scheduled or actual expiration date of the campaign.
- Status: Indicates the operational state of the campaign (e.g., Active, Ended).

Note: The Status attribute is a derived value calculated dynamically at run-time by comparing the current system date with the EndDate and is not permanently stored in the database.

1.4. AI_Generated_Content

Description: The creative outputs (captions, hashtags, image prompts) generated by the AI module.

- Attributes:
  - ContentID (Primary Key): Unique identifier for the AI variation asset.
  - Image_Prompt: The text prompt used by the generative model to create visuals.
  - Ad_Tone: The stylistic voice of the content (e.g., Professional, Humorous).
  - ContentText: The generated copywriting or caption text body.
  - GeneratedImageURL (Multi-valued Attribute / Double Oval): Storage links pointing to AI media assets.
  - Ideal_Hashtags (Multi-valued Attribute / Double Oval): Arrays of recommended social tags generated by the AI.
  - GeneratedDate: Creation timestamp of the content blocks.
  - Season: Seasonal or holiday context (e.g., Summer,Autumn).

2. Relationships & Cardinalities

- User - Owns - Product (1:N): A single User can register and manage multiple Products. Each Product belongs to exactly one User.
- User - Launches - Campaign (1:N): A single User can launch multiple Campaigns. Each Campaign is initiated by one User.
- Product - Has/Related to - Campaign (1:N): A single Product can be linked to multiple Campaigns. Each Campaign is associated with one Product.
- Campaign - Generates - AI_Generated_Content (1:N): A single Campaign produces multiple AI-generated content variations. Each content item belongs to one Campaign.

3. Diagram Notation Key

- Strong Entities: Rectangles representing User, Product, Campaign, AI_Generated_Content.
- Attributes: Ovals connected to entities.
- Primary Keys: Underlined identifiers (UserID, ProductId, CampaignID, ContentID).
- Multi-Valued Attributes: Double ovals (Email, ImageURL, GeneratedImageURL, Ideal_Hashtags).

Relationships: Diamonds representing connections between entities.

- Owns:User owns Products
- Lunches:User lunches Campaigns
- Generates:User generates Campigns

Participation Lines: Double lines indicate total participation where applicable.

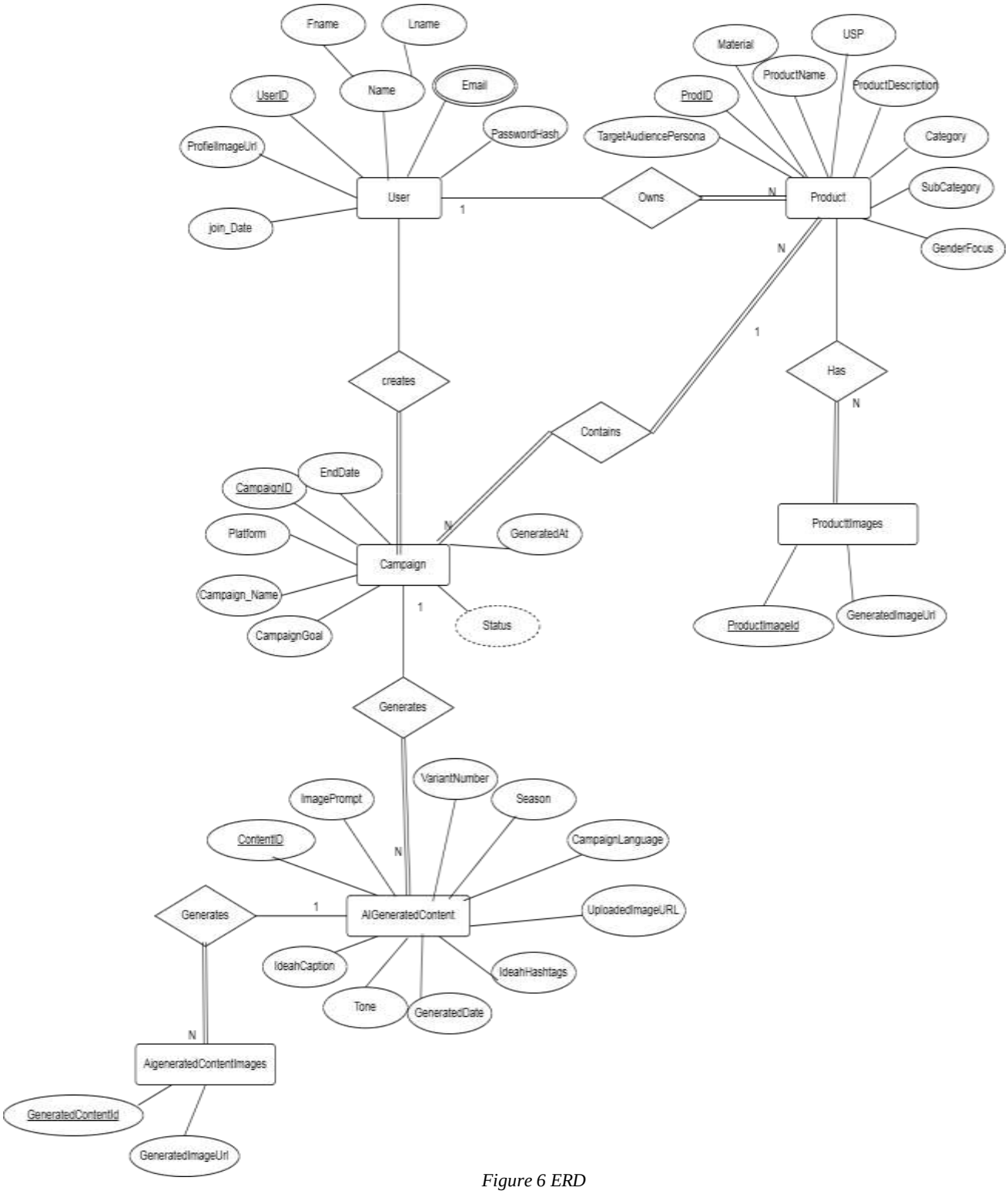


Figure 6 ERD

CHAPTER 4

System Design

System Design

This chapter presents the User Interface (UI) design of the AI-Powered Marketing Campaign Management System. It highlights the main screens and user workflows that support campaign creation, management, and AI-assisted marketing activities. The interface was designed with a focus on simplicity, usability, and accessibility to ensure a smooth user experience for marketers and business owners. Each screen is presented with a brief explanation of its functionality and contribution to the overall system workflow.

Through intuitive navigation, AI-powered assistance, and organized layouts, the application enables users to efficiently create, manage, and optimize marketing campaigns

4.0 Application Logo
4.1 Onboarding Screen – Manage & Customize
4.2 Onboarding Screen – Dashboard Overview
4.3 Onboarding Screen – Welcome to AI Campaign Creator
4.4 Sign Up Interface
4.5 Sing in Interface
4.6 Forgot Password Interface
4.7 Reset Password Interface
4.8 Dashboard Overview
4.9 Campaign Library
4.10 Create Campaign – Basic Information
4.11 Create Campaign – Audience Selection
4.12 Create Campaign – Product Information
4.13 Create Campaign – Creative Requirements
4.14 Create Campaign – Brand Assets Upload
4.15 AI Generated Campaign Details
4.16 Campaign Variants and Generated Content
4.17 AI Marketing Assistant
4.18 Profile Settings
4.19 User Interface Prototype
4.20 Project Banner

4.0 Application Logo

The application logo represents the identity of the AI-Powered Marketing Campaign Management System. It reflects the integration of artificial intelligence and marketing solutions while providing recognizable visual identity for users.



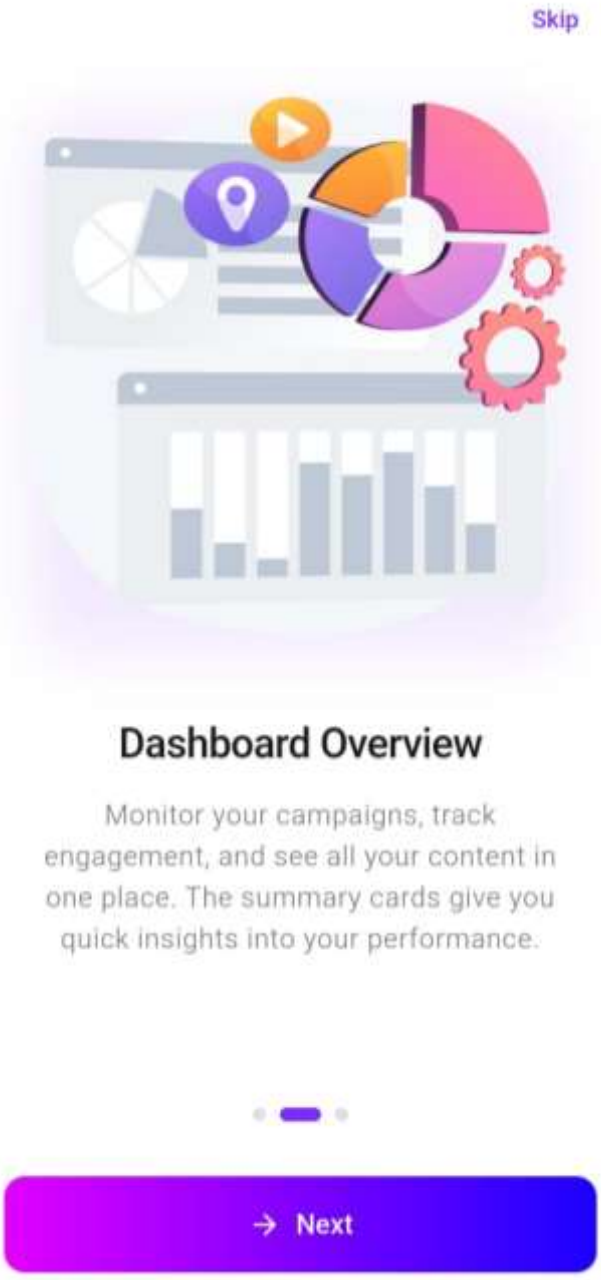
4.1 Onboarding Screen – Welcome Screen

The welcome screen serves as the entry point to the application. It provides users with options To sign in or create a new account before accessing the system.



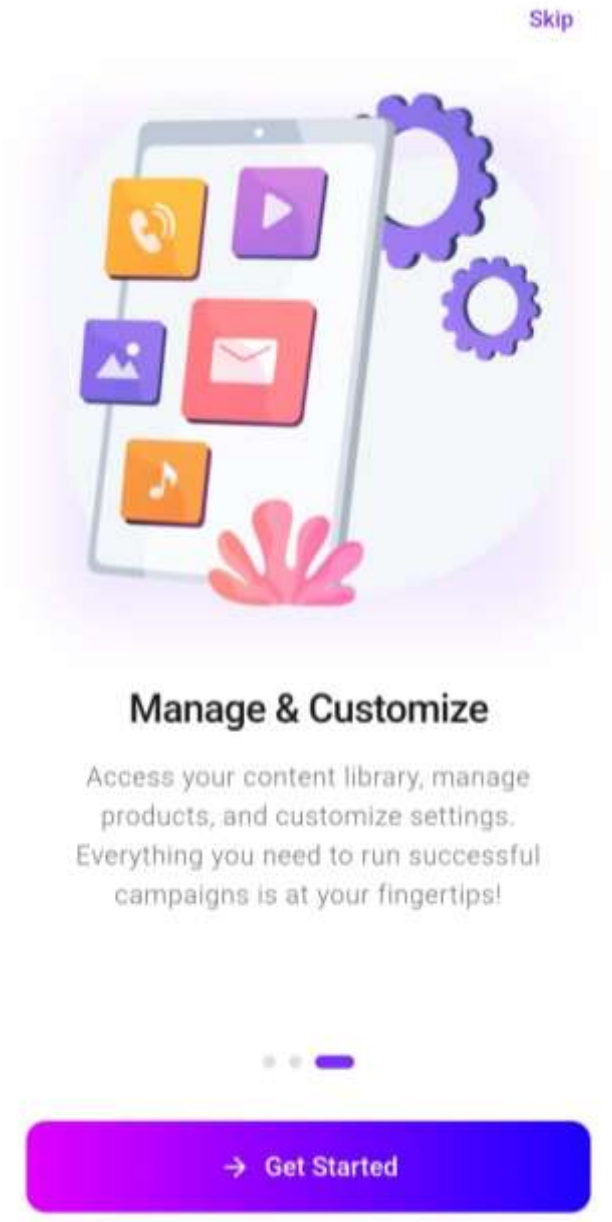
4.2 Onboarding Screen – Dashboard Overview

This screen presents an overview of the dashboard features available within the application. Users are introduced to campaign tracking, management tools, and AI-powered functionalities.



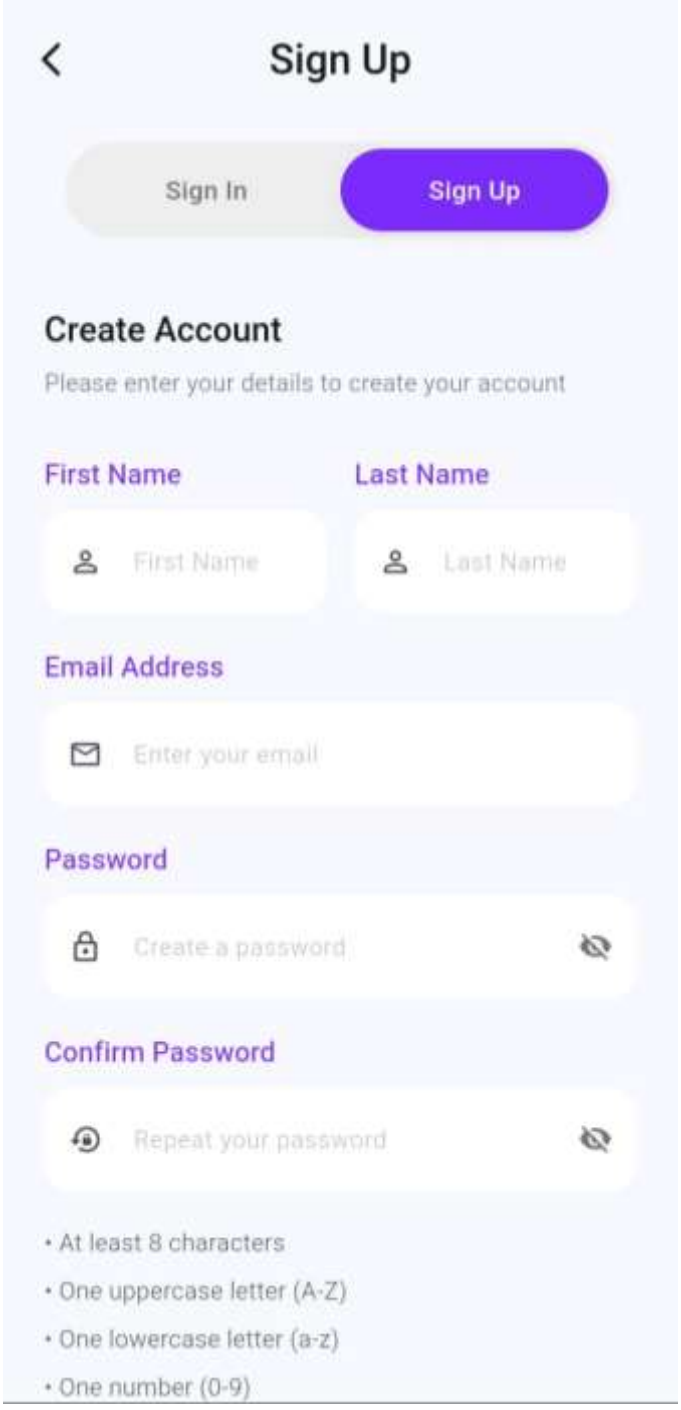
4.3 Onboarding Screen – Manage & Customize

This onboarding screen introduces users to the platform's ability to manage and Customize marketing campaigns. It provides a brief overview of the application's core Functionality and guides users through the initial experience.



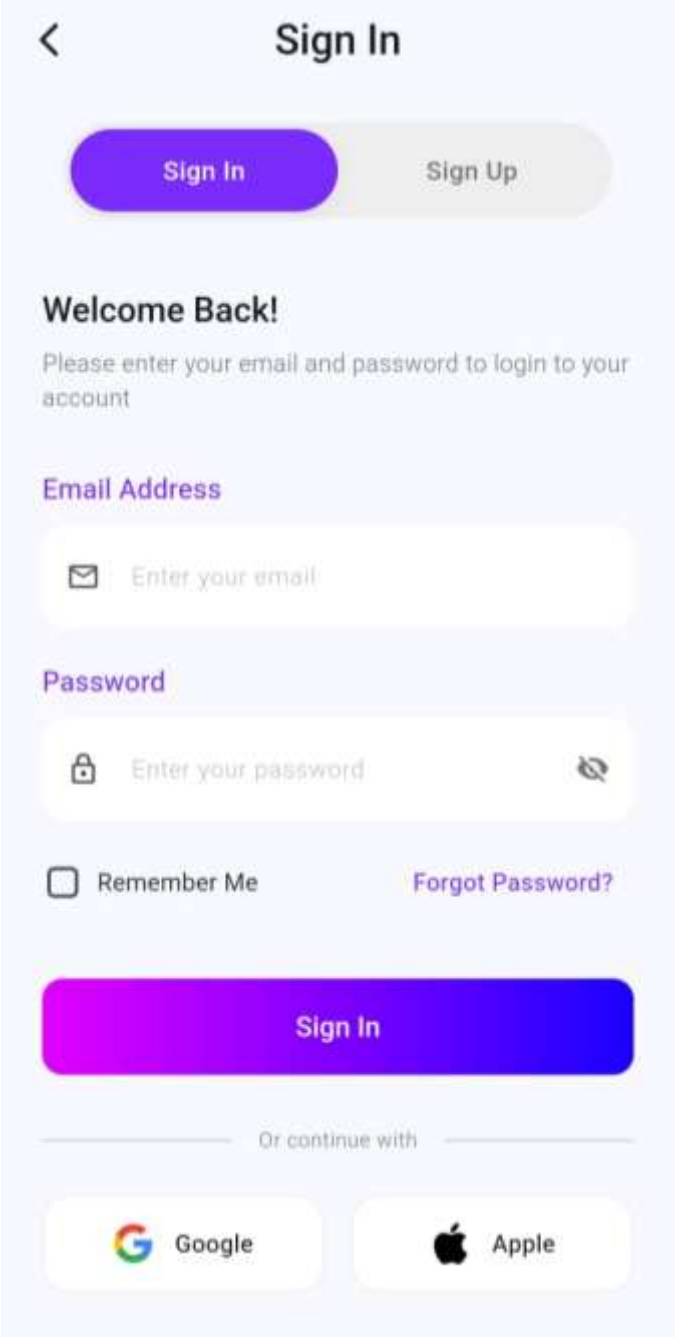
4.4 Sign Up Interface

The sign-up interface allows new users to create accounts by entering the required personal information. Successful registration grants access to the application's services.



4.5 Sign In Interface

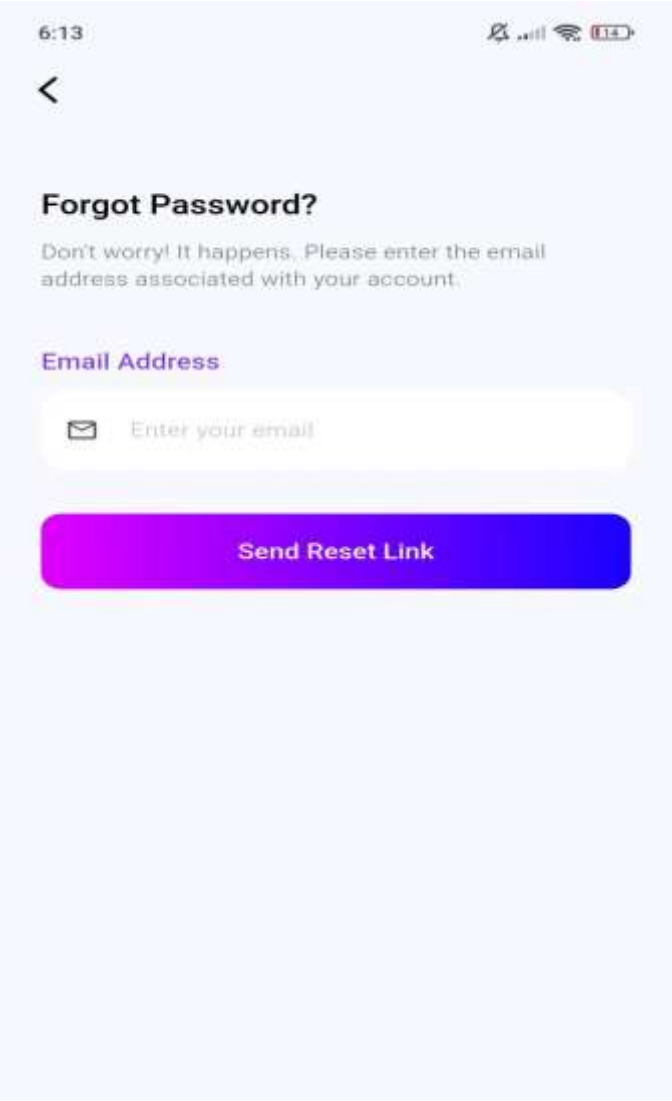
The sign-in interface enables registered users to securely access their accounts using their Credentials . Authentication is performed through secure communication with the backend system.



4.6 Forgot Password Interface

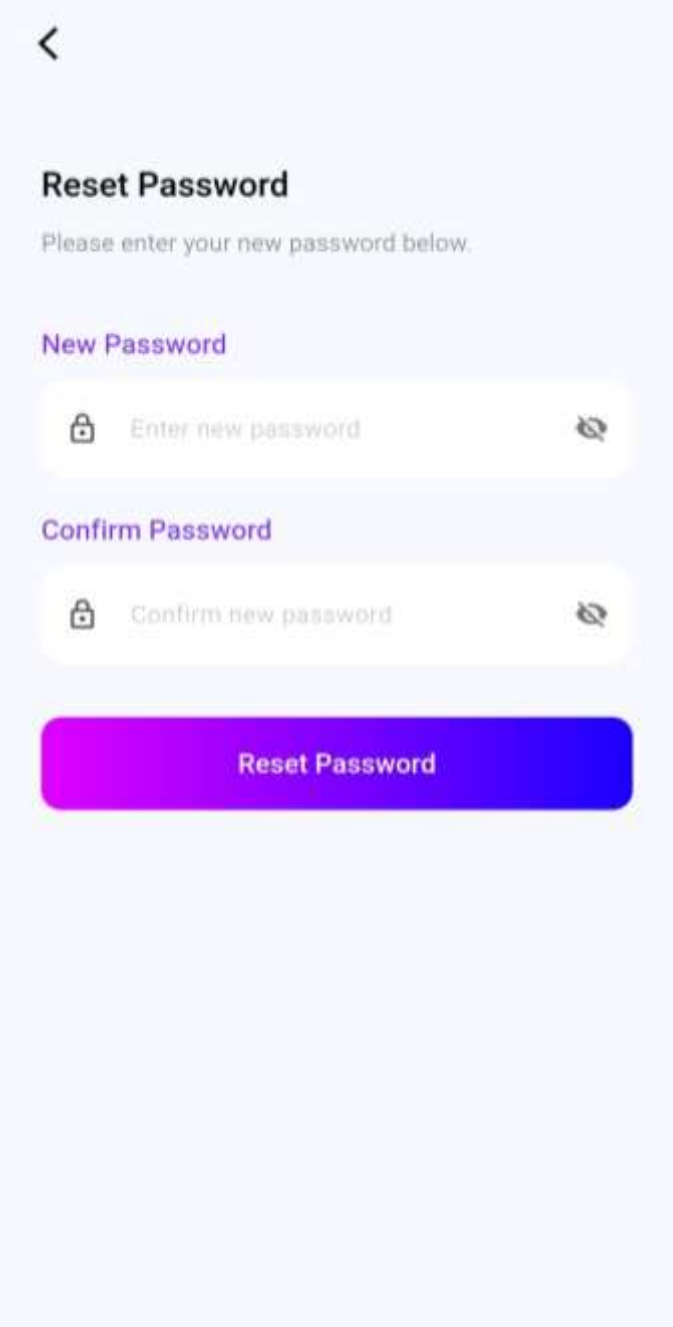
This screen allows users to request a password reset if they forget their login credentials.

A reset link or verification process is initiated through the user's registered email.



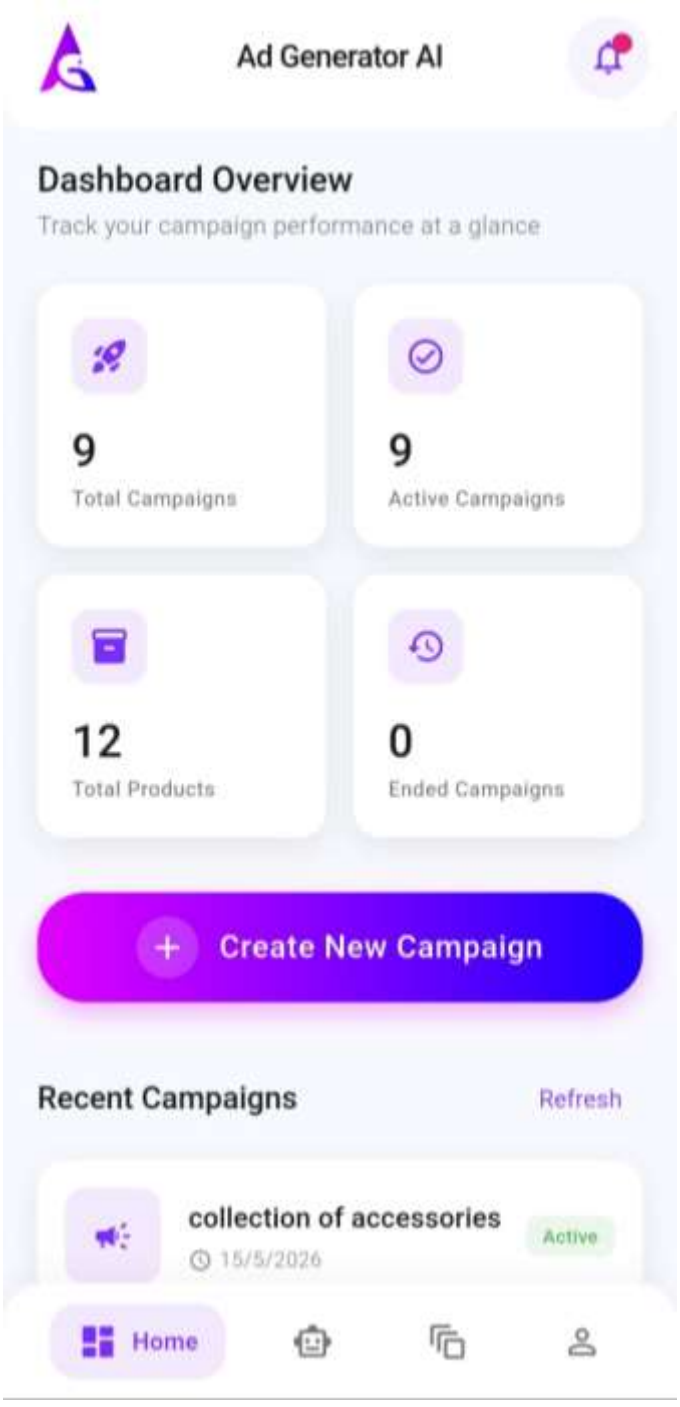
4.7 Reset Password Interface

The reset password interface enables users to create a new password after successful verification, ensuring secure account recovery.



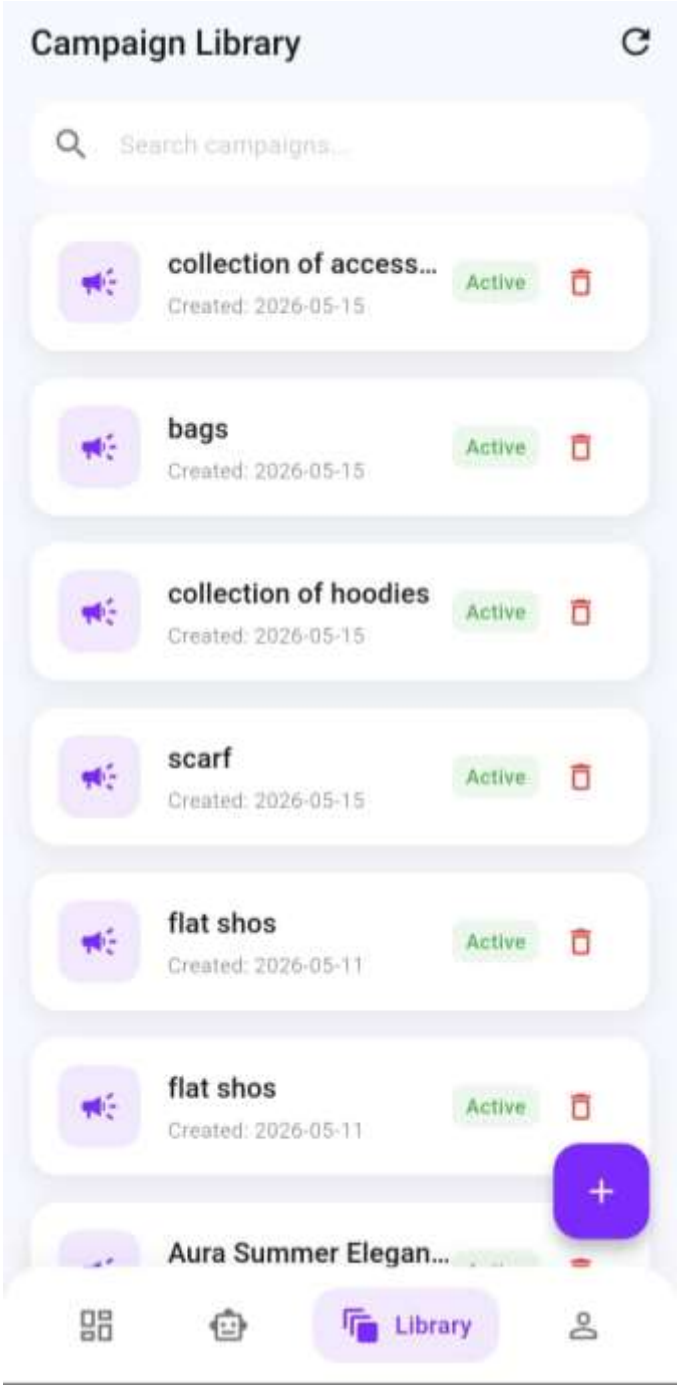
4.8 Dashboard Overview

The dashboard serves as the main control center of the application. It provides users with access to campaigns, AI-generated content, analytics, and other key system functionalities.



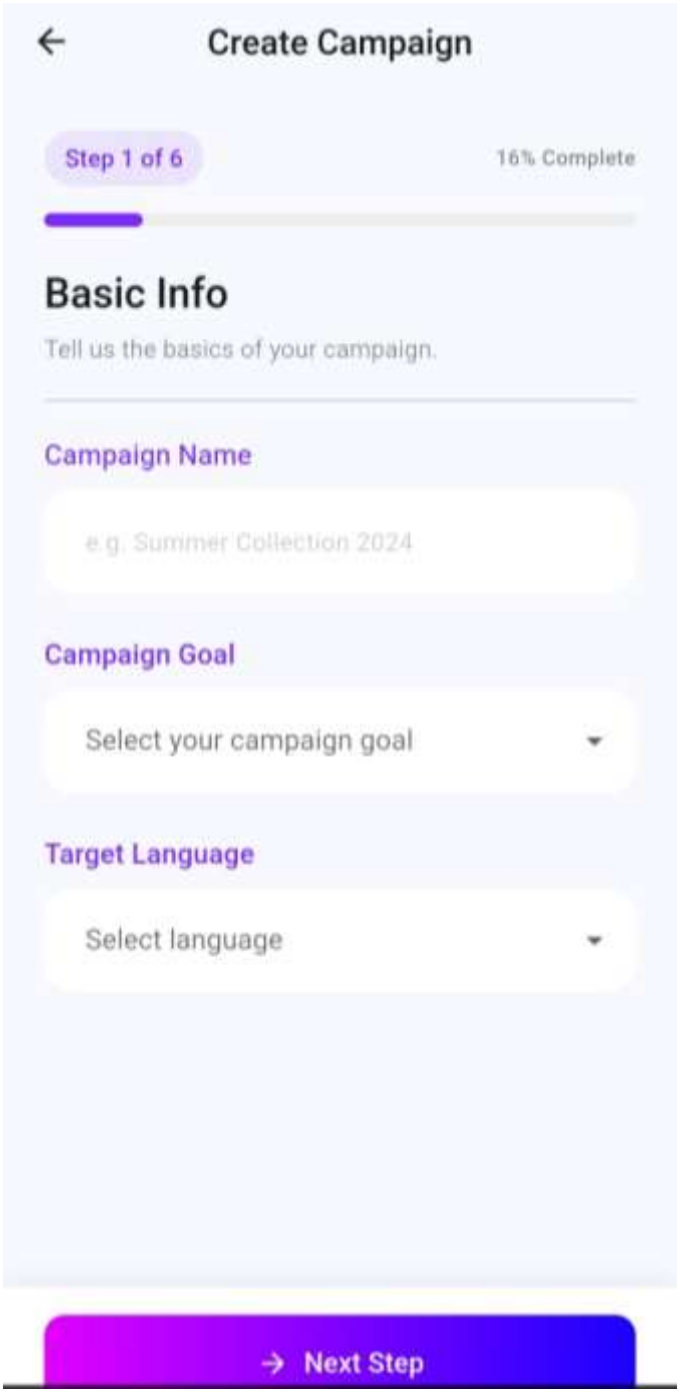
4.9 Campaign Library

The campaign library displays all campaigns created by the user. It allows users to browse, manage, and access campaign information efficiently.



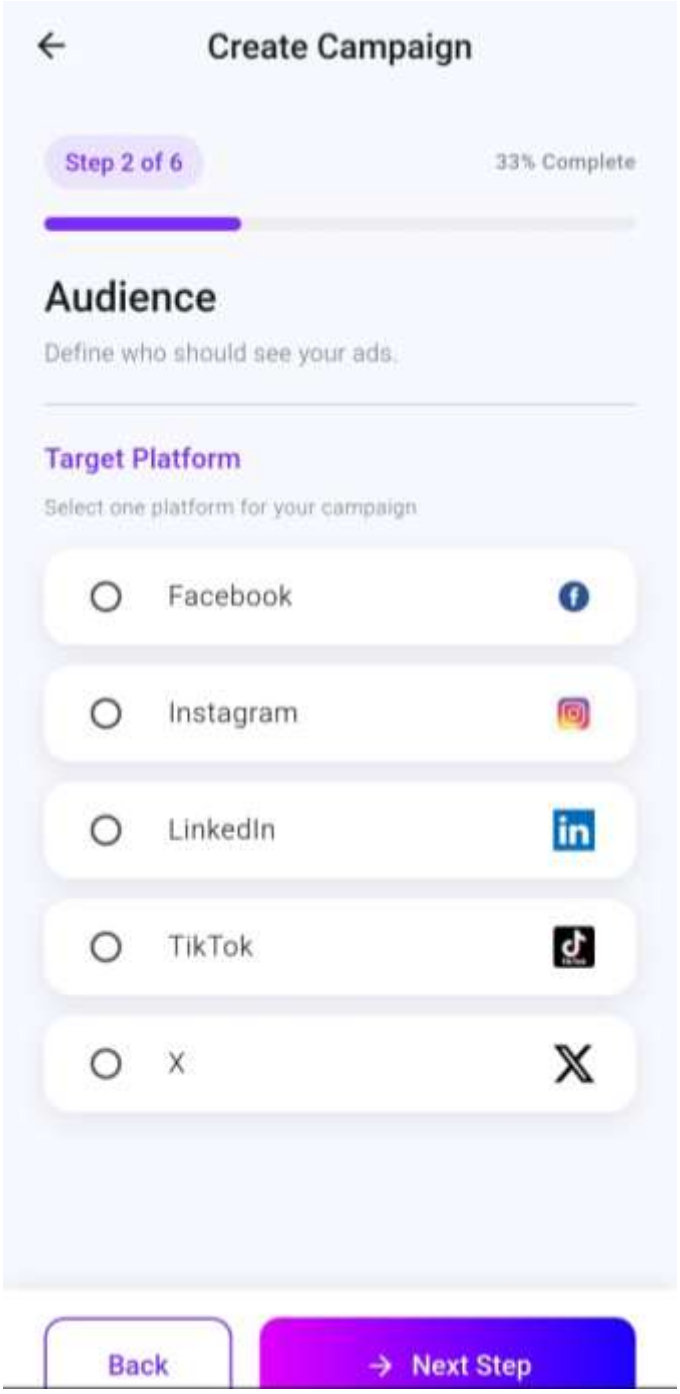
4.10 Campaign Creation – Basic Information

This screen represents the first step of the campaign creation process, where users enter essential campaign information such as campaign name and objectives.



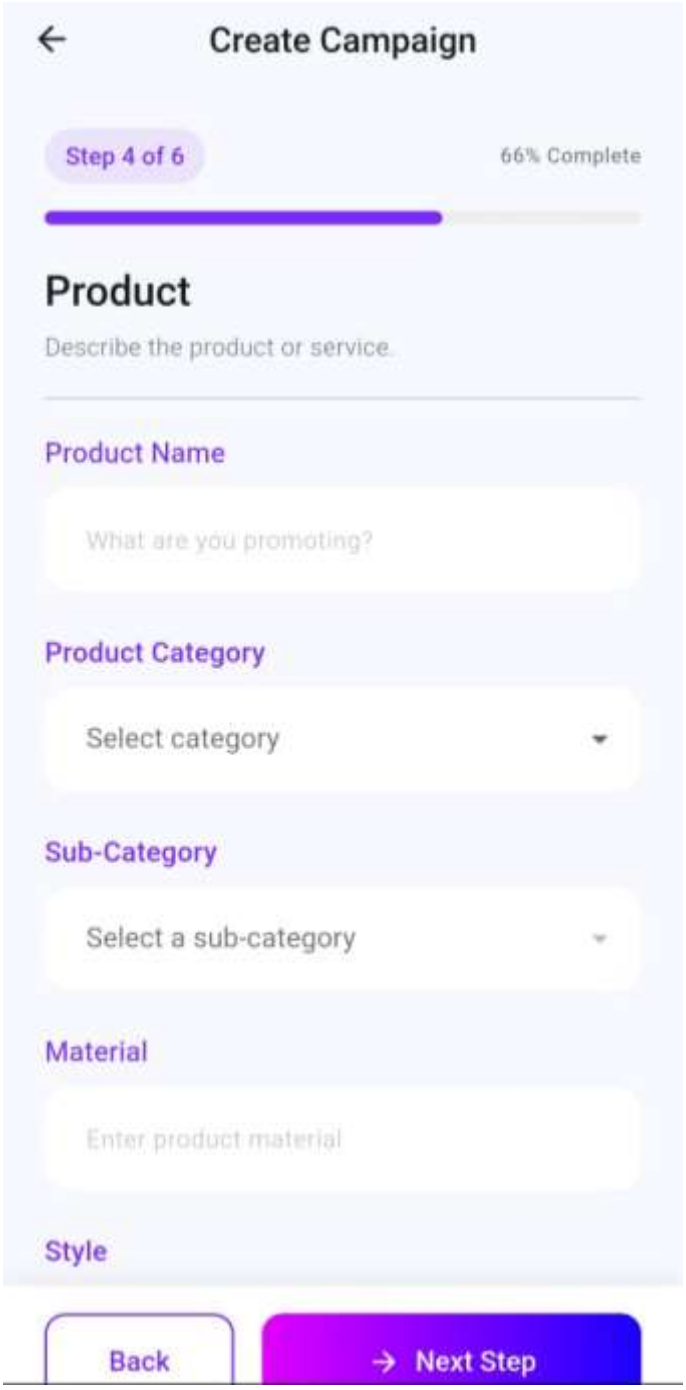
4.11 Campaign Creation – Audience Selection

This step enables users to define their target audience by specifying demographic and marketing related criteria.



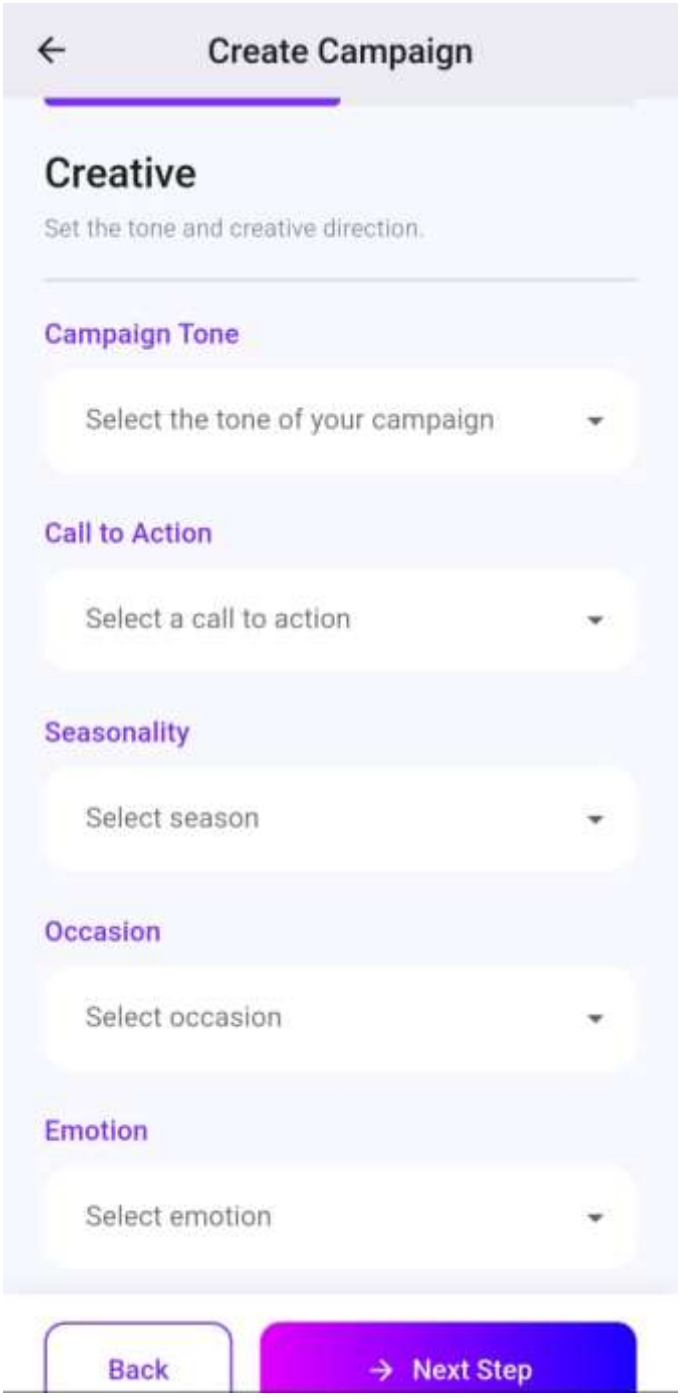
4.12 Campaign Creation – Product Information

Users provide information related to the product or service being promoted, allowing the A system to generate more accurate marketing content.



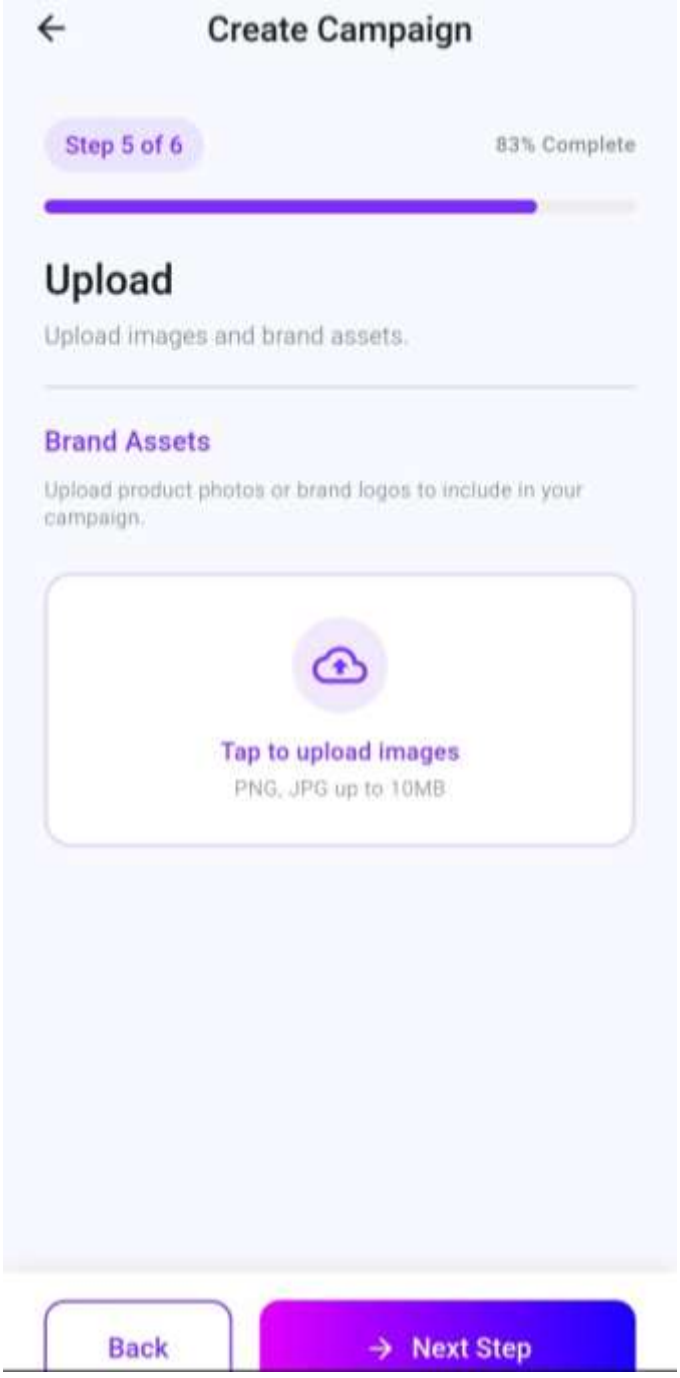
4.13 Campaign Creation – Creative Requirements

This screen collects campaign requirements, promotional goals, and content preferences needed for content generation.



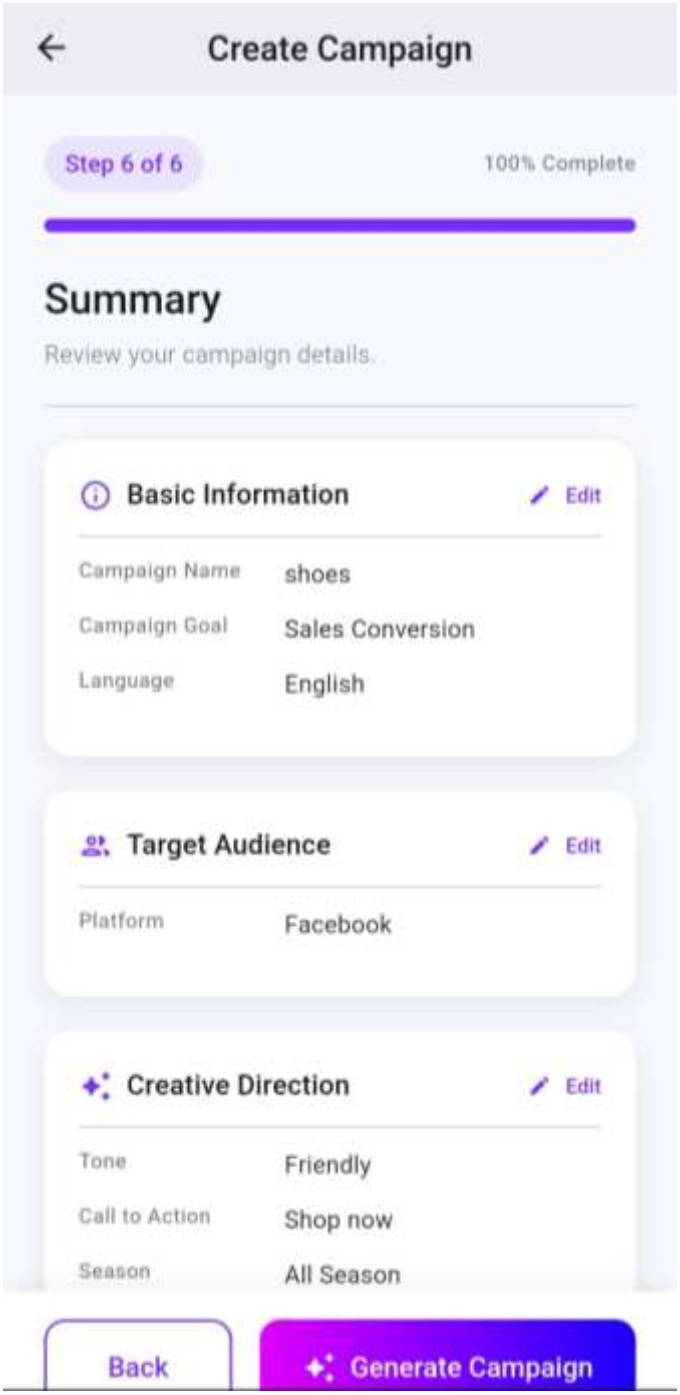
4.14 Campaign Creation – Brand Assets Upload

Users can upload brand assets such as logos, images, and visual materials that will be incorporated into generated marketing content.



4.15 Campaign Summary Screen

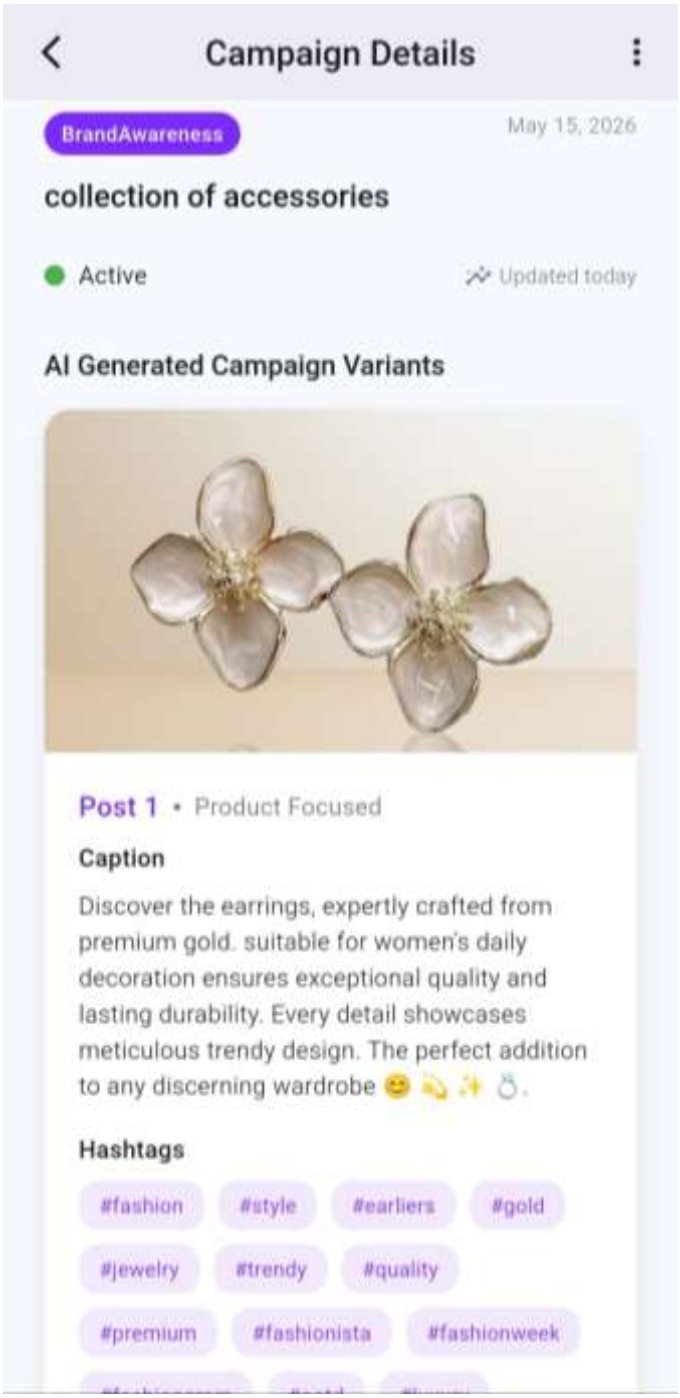
The campaign summary screen displays all information entered by the user throughout the campaign creation process. It allows users to review campaign details, target audience information, product specifications, uploaded brand assets, and selected preferences before generating the final marketing campaign.



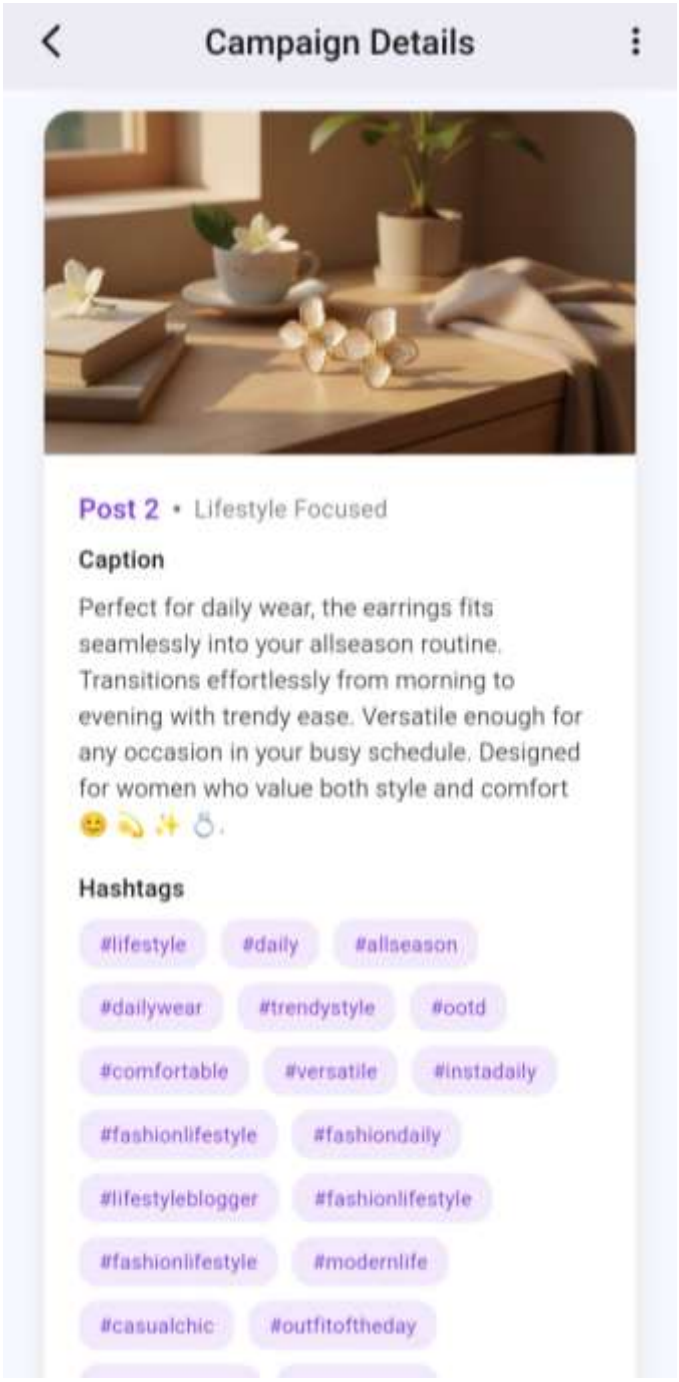
4.16 AI Generated Campaign Details

This screen displays the result of 3 campaigns content generated by the AI engine based on the information provided during campaign creation.

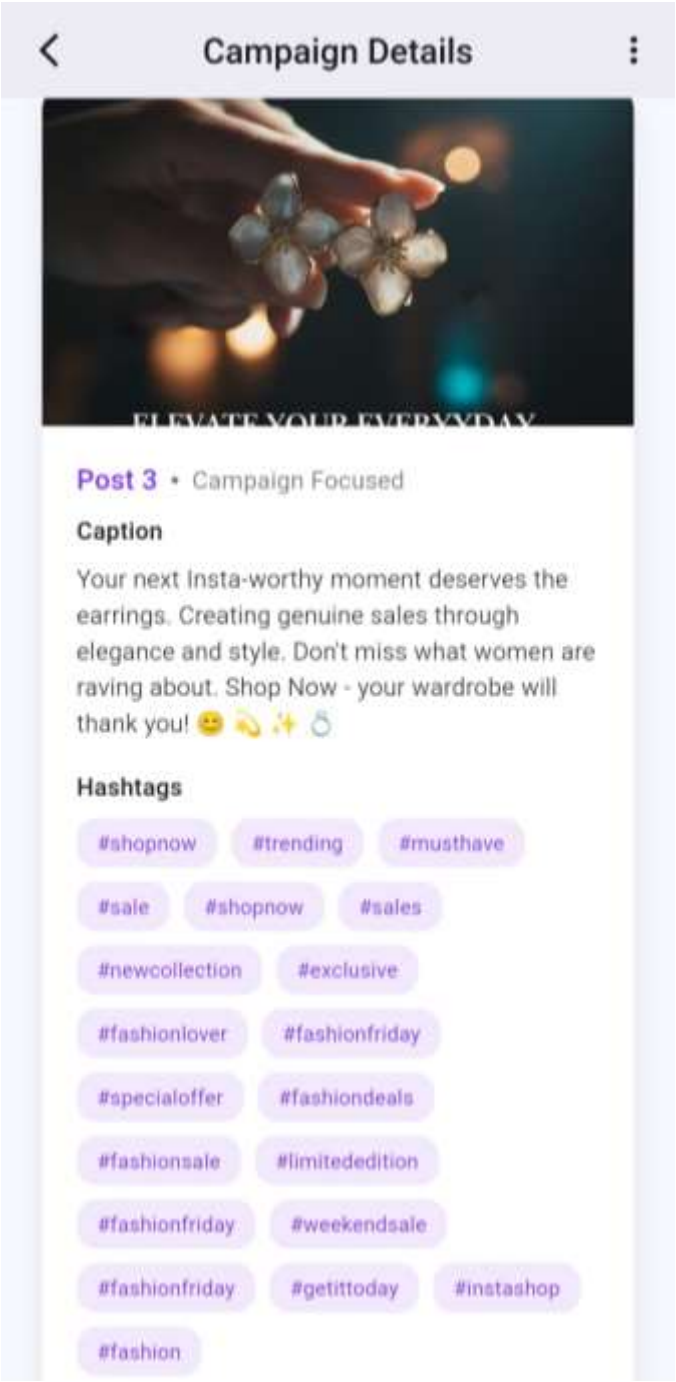
Post 1



Post 2

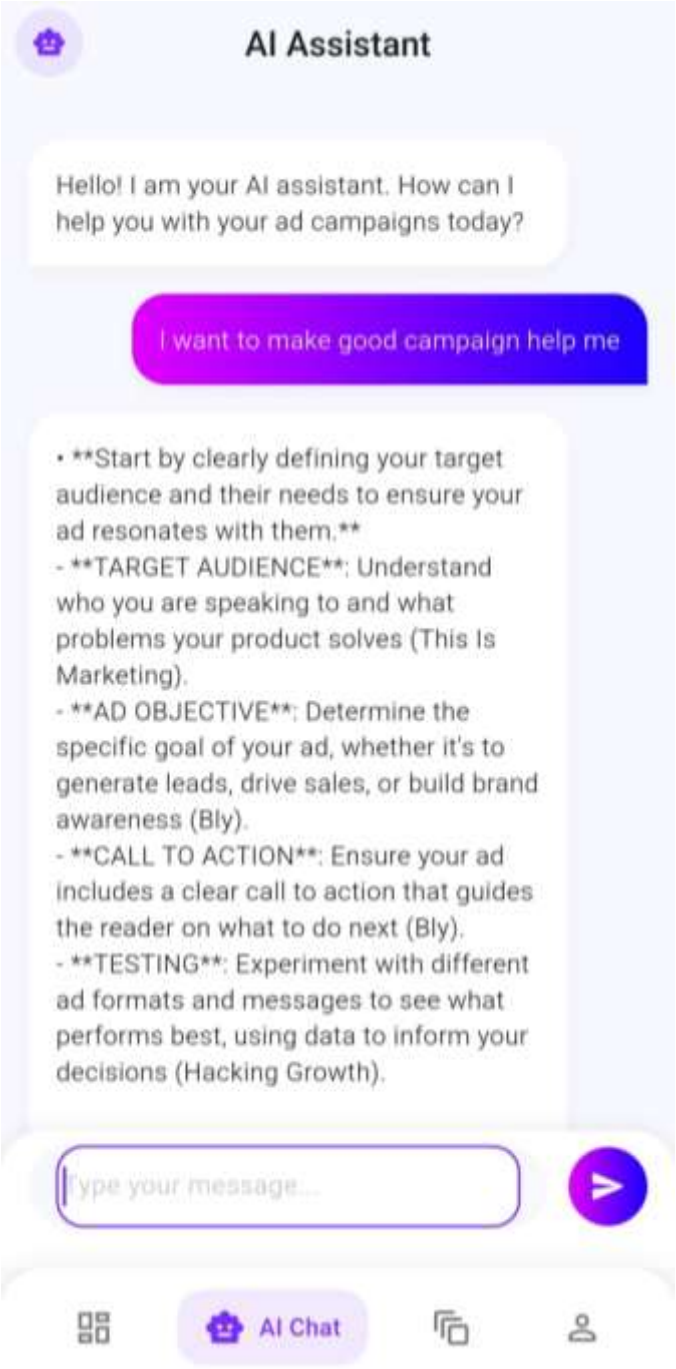

Campaign Details
Post 2 • Lifestyle Focused
Caption
Perfect for daily wear, the earrings fits seamlessly into your allseason routine. Transitions effortlessly from morning to evening with trendy ease. Versatile enough for any occasion in your busy schedule. Designed for women who value both style and comfort
Hashtags
#lifestyle
#daily
#allseason
#dailywear
#trendystyle
#ootd
#comfortable
#versatile
#instadaily
#fashionlifestyle
#fashiondaily
#lifestyleblogger
#fashionlifestyle
#fashionlifestyle
#modernlife
#casualchic
#outfitoftheday

Post 3


Campaign Details
Post 3 • Campaign Focused
Caption
Your next Insta-worthy moment deserves the earrings. Creating genuine sales through elegance and style. Don't miss what women are raving about. Shop Now - your wardrobe will thank you!
Hashtags
#shopnow
#trending
#musthave
#sale
#shopnow
#sales
#newcollection
#exclusive
#fashionlover
#fashionfriday
#specialoffer
#fashiondeals
#fashionsale
#limitededition
#fashionfriday
#weekendsale
#fashionfriday
#getittoday
#instashop
#fashion

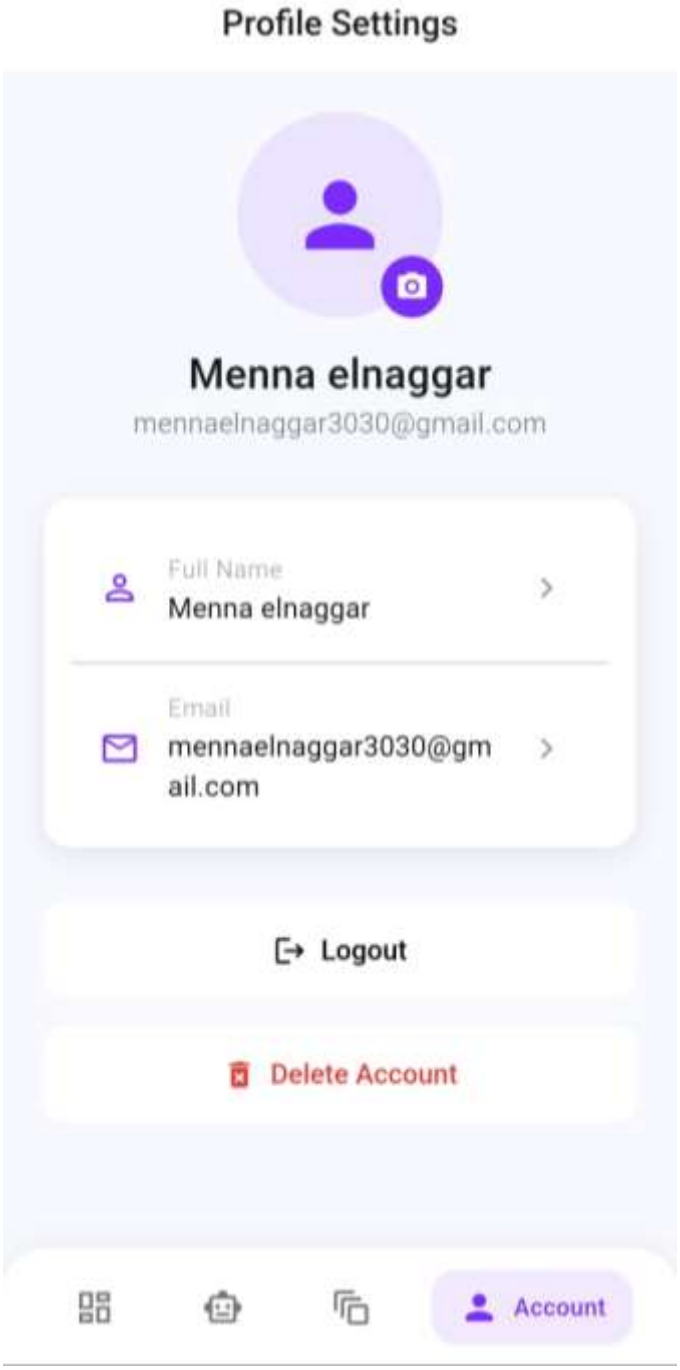
4.17 AI Marketing Assistant

The AI Marketing Assistant provides users with intelligent recommendations, marketing insights, and campaign-related guidance through conversational interaction.



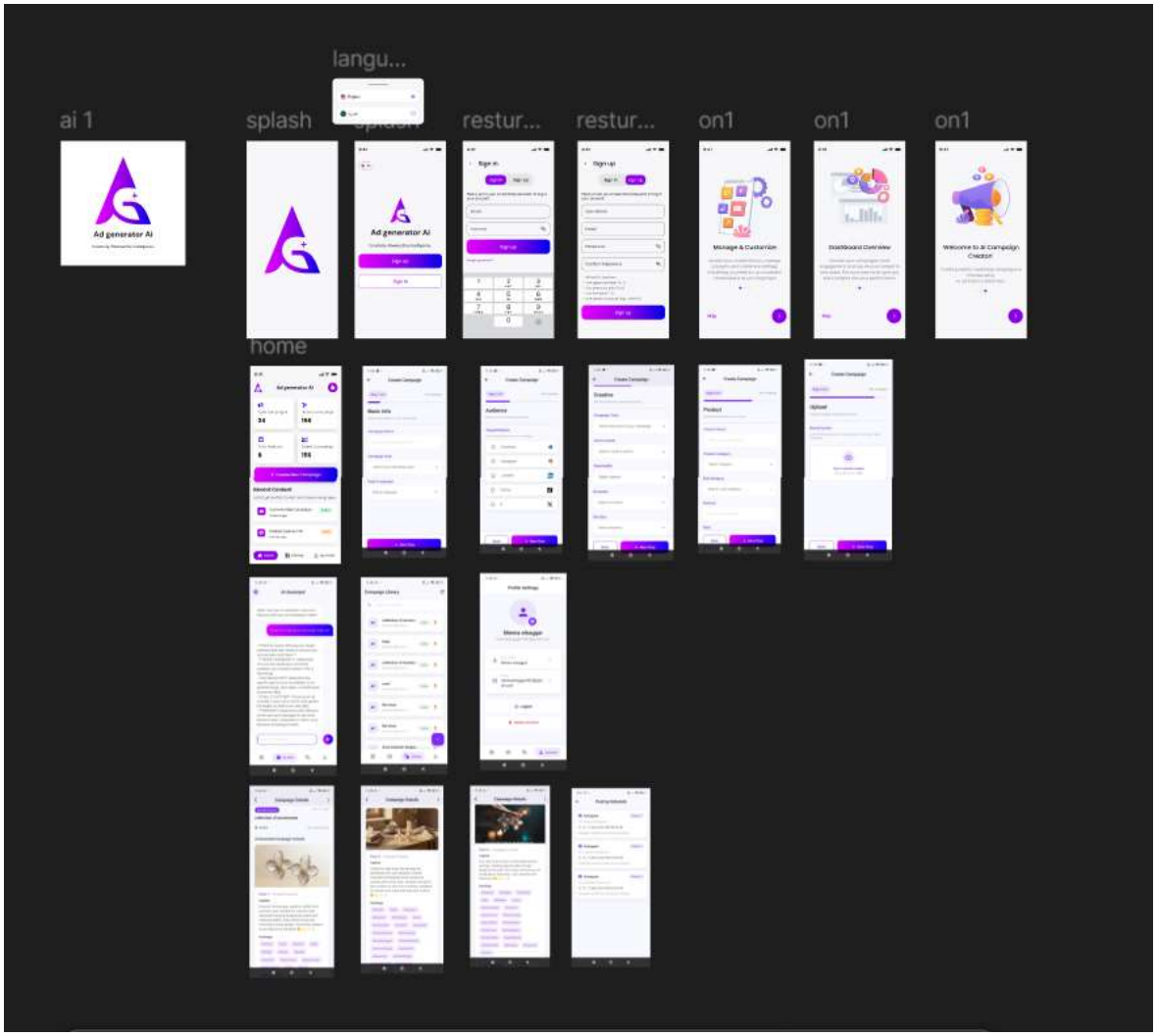
4.18 Profile Settings

This screen allows users to manage personal information, account settings, and application Preferences.



4.19 User Interface Prototype

The user interface prototype presents a complete overview of the application's design and navigation flow before implementation.



4.20 Project Banner

The Ad Generator banner provide a comprehensive visual overview of the platform,highlighting its core mission to revolutionize marketing through artificial intelligence.



CHAPTER 5

System Implementation

System Implementation

5.1 Introduction

In this chapter, we delve into the practical aspects of bringing **AD GENERATOR AI** to life. After defining the system's architecture and design, the next crucial step is implementation, where concepts are translated into a fully functional, AI-powered system.

We will explore the core components and technologies used to develop **AD GENERATOR AI** , including frontend development with Flutter, integration of AI services for automated marketing campaign generation, and the implementation of the chatbot assistant feature.

This chapter also highlights the database integration and API connections that support the system core functionalities, along with the structured approach used to organize the mobile application.

By the end of this chapter, readers will have a clear understanding of how **AD GENERATOR AI** was built, including insights into the development practices, frameworks, and tools that played a vital role in its implementation.

5.1 Introduction

5.2 Frontend Implementation

5.3 Database Implementation

5.4 Backend Implementation

5.5 APIs Implementation

5.6 AI Implementation— Text and Hashtag Generation

5.7 AI Implementation — AI Advertising Image Generation Module

5.8 AI Implementation— AI Marketing Chatbot Module

5.2 Frontend Implementation (Flutter)

The frontend of the **Ad generator AI** application was developed using **Flutter [1]**, for building natively compiled applications for mobile. This choice was driven by Flutter's exceptional performance, rapid development capabilities, and its ability to create beautiful, expressive user interfaces. This section provides an in-depth look at how the frontend was structured, including key components, state management strategies, API integration, and the underlying UI design principles that guided its development.

We will explore:

- The overall structure of the Flutter project, emphasizing modularity and scalability.
- Core Modules and their specific functionalities within the application.
- Seamless API integration with the backend services to ensure robust data flow.
- Effective state management techniques employed to maintain application consistency and responsiveness.

Code snippets and explanations will be provided to illustrate how different parts of the frontend were implemented.

5.2.1 Project Structure

The project follows a well-defined, feature-driven directory structure that is carefully organized to enhance code readability, maintainability, and collaborative development.

This architectural approach ensures that each functional module is self-contained, promoting a clear separation of concerns and simplifying the management of complex application features.

Key Features:

1 **Modular Organization** ➢The lib directory, the heart of the Flutter application, is logically segmented into core for shared utilities and features for distinct functional modules (e.g., auth, campaign, chatbot).
✔This modularity significantly enhances maintainability, scalability, and facilitates team collaboration by isolating concerns.

2 **Centralized Entry Point** ➢The main.dart file serves as the application's primary entry point, responsible for orchestrating the initial setup, dependency injection, and defining the application's root widget and routing.
✔It manages initial deep linking and sets up application-wide navigation, ensuring a smooth user journey from launch.

5.2.2 Authentication Module

The Authentication Module (features/auth) is meticulously designed to manage all aspects of user access within the application, ensuring secure identification and authorization.

✔It encompasses critical functionalities such as user registration, secure login procedures, robust password recovery mechanisms, and efficient session management.

✔This module is paramount for safeguarding user data and protecting application resources from unauthorized access.

Key Features:

3 **User Credential Management**➢Handles user registration, login, and password reset flows.

✔Ensures secure user identification and authorization within the Flutter app.

4 **Dedicated UI for Authentication** ➢Provides specific Flutter pages (AuthPage, LoginPage, SignUpPage) and widgets (SignInForm, SignUpForm).

✔Offers intuitive interfaces for user interaction and input validation using Flutter's widget tree.

5 **API Integration for Security**➢Seamlessly interacts with backend authentication services via Flutter's HTTP capabilities to verify credentials and manage user sessions.

✔Guarantees secure communication and data integrity for user authentication.

Code Highlights:

-**AuthPage Widget**: A central Flutter widget that orchestrates the display of various authentication forms based on the user's state.

-**AuthBloc / AuthCubit**: Manages the authentication state (e.g., loading, authenticated, unauthenticated) using Flutter's state management solutions.

5.2.3 Campaign Creation Module

➢The Campaign Creation Module (features/campaign/presentation/create_campaign) is a core functionality, empowering users to design and generate sophisticated AI-powered marketing campaigns directly within the Flutter application.

✔This module guides users through a multi-step process, systematically collecting diverse inputs such as precise campaign goals, target platforms, desired creative styles, and comprehensive product details.

✔A dedicated controller manages the state and logic across these sequential steps, efficiently aggregating user inputs before their submission to the AI generation service.

Key Features:

6 **Multi-Step User Flow**➢Guides users through a series of distinct steps (e.g., basic info, audience, creative, product, images) to define campaign parameters.

✔Ensures all necessary information is collected systematically through a user-friendly Flutter UI.

7 **Form Data Management**➢A controller aggregates and manages all user inputs across different steps.
✔Facilitates data consistency and prepares data for submission to the AI generation API.

8 **Dynamic UI for Input**➢Each step is implemented as a distinct Flutter UI component, allowing for a guided and intuitive user experience.
✔Utilizes Flutter widgets like`T extField`, `DropdownButton`, and `Checkbox` for rich user input.

Code Highlights:

- **`CampaignCreationController`**

Class: A Dart class responsible for holding and managing

the state of various campaign parameters (`campaignName`, `selectedPlatforms`, `tone`, `productName`).

- **`toEntity()`**

Method: Converts the collected form data into a `Campaign` entity, ready for API calls

5.2.4 Campaign Results Module

➢The Campaign Results Module is specifically designed to present the meticulously generated output of the AI-powered marketing campaigns to the user within the Flutter interface.
✔Upon successful campaign generation, this module furnishes the user with comprehensive

results, including AI-generated captions, intelligently suggested hashtags, and visually

appealing promotional images, all meticulously tailored for various digital platforms.
✔The design prioritizes clarity and readability, enabling users to effortlessly review and effectively utilize the generated content.

Key Features:

9 **Rich Content Display**➢Presents AI-generated captions, relevant hashtags, and promotional images in an organized Flutter layout.
✔Content is clearly displayed and tailored for specific platforms, enhancing user understanding.

10 **Intuitive User Interface**➢Designed for clarity and readability, allowing easy review and utilization of generated content.
✔Enhances user experience by providing organized and intuitive presentation using Flutter widgets.

11 **Post-Generation Actions**➢Offers options to save, share, or further refine the generated campaigns directly from the Flutter app.
✔Provides flexibility and control over campaign deployment and management.

Code Highlights:

- **CampaignResultPage**

Widget: The primary Flutter UI for displaying the generated campaign content, often using ListView.builderfor dynamic lists of posts.

- **PostCard**

Widget: A reusable Flutter widget to display individual campaign posts, including image, caption, and hashtags.

5.2.5 AI Chatbot Module

➢The AI Chatbot Module (features/chatbot) seamlessly integrates an intelligent conversational agent into the application, providing users with on-demand marketing advice and support through a Flutter-based chat interface.

✔Users can engage with the chatbot to pose questions concerning marketing strategies, content ideation, and business expansion.

✔The ChatbotPage serves as the principal interface for these interactions, showcasing the conversation history and facilitating real-time communication with the AI.

Key Features:

12 **Interactive Chat Interface**➢Provides an intuitive chat UI where users can type questions and receive responses from the AI marketing assistant.
✔Built with Flutter widgets for a smooth and responsive conversational experience.

13 **Real-time Communication**➢Displays conversation history and enables real-time interaction with the AI backend.
✔Ensures a dynamic and responsive user experience, crucial for chatbot interactions.

14 **Marketing Guidance**➢Offers on-demand marketing advice, content ideas, and business growth strategies.
✔Enhances the application's utility by providing immediate expert insights.

Code Highlights:

- **ChatbotPage Widget**: The main Flutter UI for chatbot interactions, typically composed of a ListView for messages and a TextField for input.
- **ChatInputField Widget**: A custom Flutter widget for user input, often including a send button.

5.2.6 Campaign Library Module

➢The Campaign Library Module provides users with a centralized and accessible repository

to manage and review all their previously created marketing campaigns directly within the Flutter
✔This module enables users to effortlessly browse through their saved campaigns, delve into their intricate details, and monitor their performance over time.
✔It incorporates functionalities such as advanced searching, efficient filtering, and intuitive organization of campaigns, empowering users to revisit successful past strategies or make informed adjustments to ongoing ones.

Key Features:

15 **Centralized Campaign Storage**➢Stores and organizes all previously created marketing campaigns within the application.
✔Provides easy access to historical campaign data through a Flutter-based list or grid.

16 **Efficient Campaign Management**➢Supports searching, filtering, and organizing campaigns to facilitate efficient retrieval.
✔Utilizes Flutter's UI capabilities to provide intuitive controls for managing saved content.

17 **Performance Review**➢Allows users to view campaign details and track performance over time.

✔Enables informed decision-making and strategy refinement based on past campaign data.

Code Highlights:

- **LibraryPage**

Widget: The primary Flutter UI for displaying the list of saved campaigns, often using BlocBuilder or Consumerfor state management.

- **CampaignGrid / CampaignList**

Widget: Reusable Flutter widgets to display campaigns in an organized layout.

5.2.7 Profile Module

➢The Profile Module is designed to empower users with the ability to manage their personal information and customize application settings directly within the Flutter interface.
✔This module typically encompasses features for viewing and editing user details, managing subscription plans, and tailoring application preferences to individual needs.
✔It plays a crucial role in delivering a personalized user experience, allowing users to maintain control over their data and adapt the application to their specific requirements.

Key Features:

18 **Personal Data Management**➢Allows users to view and edit their personal details, such as name, email, and profile picture.
✔Ensures user data is accurate and up-to-date through Flutter forms.

19 **Application Settings**➢Provides options for managing subscription plans, notification preferences, and other application settings.
✔Empowers users to tailor the app to their specific needs and preferences.

20 **Account Control Center**➢Serves as a central location for all user-specific configurations and account-related actions.
✔Enhances user control and personalization within the Flutter application.

Code Highlights:

- **ProfilePage**

Widget: The main Flutter UI for displaying and editing user profile information and settings.

5.3 Database Implementation

The **Ad Generator AI** system relies on a well-structured relational database to manage user accounts, marketing campaigns, product details, and AI-generated content. The database ensures data integrity, efficient retrieval, and seamless integration between the mobile application, the API layer, and the AI generation module.

Key Components:

- **User Management**: Stores user accounts, authentication credentials, and profile information.
- **Campaign Management**: Stores campaign metadata, including goals, target audience, creative

direction, and lifecycle status (Active/Ended).

- **Product Management**: Stores the details of each product associated with a campaign, allowing multiple products to be grouped under a single campaign.
- **AI-Generated Posts**: Logs the AI-generated captions, hashtags, and images produced for each product, categorized by post type (Product Focused, Lifestyle Focused, Campaign Focused).

Tables and Relationships:

- The database follows a **relational model**, where key entities are connected through
- **foreign keys** to maintain consistency.
- Constraints such as **ON DELETE CASCADE** ensure data integrity by automatically handling deletions.

5.3.1 Users Table

The The AspNetUsers table stores essential information about each registered user in the Ad Generator AI system. It includes personal details, authentication credentials, and links to their products and campaigns.

Table Structure:

- **Id:** A unique identifier for each user (string, primary key, generated by ASP.NET Core Identity).
- **FirstName**: The first name of the user.
- **LastName**: The last name of the user.
- **JoinDate**: The date the account was created, automatically set on insertion using UTC time.
- **ProfileImageUrl**: An optional URL pointing to the user's profile picture.
- **Email**: A unique email address for authentication and communication (inherited from Identity).
- **PasswordHash**: The user's password, stored in a hashed format for security (inherited from Identity).

The table inherits from ASP.NET Core Identity's IdentityUser, which also provides built-in fields such as UserName, NormalizedUserName, SecurityStamp, ConcurrencyStamp, PhoneNumber, EmailConfirmed, Lockout information, Two-Factor Authentication settings, and authentication tokens. The **ON DELETE RESTRICT** constraint prevents deleting a user while they still own products or campaigns, preserving referential integrity.

5.3.2 Products Table

The Products table stores the details of each product that a user wishes to promote within a marketing campaign. It includes descriptive, categorical, and targeting information used later by the AI module

during content generation.

Table Structure:

- **Id:** A unique identifier for each product (auto-incremented).
- **Category**: The product category, stored as a string representation of the ProductCategory **enum** (Footwear, Apparel, or Accessories).
- **SubCategory**: A more specific classification of the product.
- **Material**: The material the product is made from.
- **ProductName**: The name of the product.
- **ProductDescription**: An optional field describing the product in detail.
- **GenderFocus**: The target gender, stored as a string representation of the GenderFocus enum (Men, Women, or Unisex).
- **TargetAudiencePersona**: A description of the intended customer persona.
- **USP**: An optional field containing the product's Unique Selling Point.
- **UserId**: A foreign key linking the product to its owner.
- **CreatedAt**: The date the product was created, automatically generated using UTC time.

The **ON DELETE RESTRICT c**onstraint ensures that a user cannot be deleted while they still own products. Each product can contain multiple product images, can belong to multiple campaigns through the CampaignProducts table, and can be associated with multiple AI-generated content records. The relationships with CampaignProducts and AIGeneratedContents also use ON DELETE RESTRICT to preserve product records and maintain referential integrity.

5.3.3 ProductImages Table

The ProductImages table stores the images uploaded for each product, allowing a single
Product to have multiple associated images,

Table Structure:

- **Id:** A unique identifier for each image (auto-incremented).
- **ImageURL:** The URL of the uploaded product image.
- **ProductId:** A foreign key linking the image to its product.
- **CreatedAt: The date the image record was created automatically using UTC time.**

The **ON DELETE CASCADE** constraint ensures that when a product is deleted, all of its associated

images are automatically removed.

5.3.4 Campaigns Table

The Campaigns table stores the core information of each marketing campaign created by a user, including its goal, target platform, creation date, and lifecycle information.

Table Structure:

- **Id:** A unique identifier for each campaign (auto-incremented).
- **CampaignName:** The name assigned to the campaign.
- **EndDate:** An optional date indicating when the campaign ends.
- **Platform:** The target social media platform, stored as a string representation of the CampaignPlatform enum (TikTok, Facebook, or Instagram).
- **CampaignGoal:** The campaign objective, stored as a string representation of the CampaignGoal enum (Sales, Brand Awareness, Engagement, Launch/Announcement, or Conversion).
- **UserId:** A foreign key linking the campaign to its creator.
- **GeneratedAt:** The date the campaign was created, automatically generated using UTC time.

Status Handling:

The campaign status is intentionally not mapped to the database by using the [NotMapped]

attribute. Instead, it is computed in real time whenever campaign information is retrieved.

Currently, the system supports two status values:

Active: Returned when the campaign has no EndDate specified or when the current

UTC date And time is Earlier than the EndDate .

Ended: Returned when the current UTC date and time is later than the EndDate.

This design ensures that the campaign status is always up to date without requiring database updates. It also provides flexibility for future dashboard enhancements, where additional campaign statuses or monitoring modes can be introduced without modifying the database Schema.

The **ON DELETE RESTRICT** constraint prevents deleting a user while campaigns still exist. The

ON DELETE CASCADE constraint automatically removes related CampaignProducts and AI-generated contents when a campaign is deleted.

5.3.5 CampaignProductTable

The CampaignProducts table resolves the many-to-many relationship between campaigns and products, allowing multiple products to be grouped within a single campaign while allowing products to participate in multiple campaigns.

Table Structure:

- **Id:** A unique identifier for each relationship record (auto-incremented).
- **CampaignId:** A foreign key referencing the Campaigns table.
- **ProductId:** A foreign key referencing the Products table.
- **CreatedAt:** The date the relationship record was created automatically using UTC time.

A unique constraint on the combination of CampaignId and ProductId prevents duplicate product assignments within the same campaign.

The **ON DELETE CASCADE** constraint on CampaignId automatically removes relationship records when a campaign is deleted, while the ON DELETE RESTRICT constraint on ProductId protects product records from accidental deletion.

5.3.6 AI Generated Contents Table

The AIGeneratedContents table stores every AI-generated content variant produced for a product within a campaign. Each record contains the generated caption, hashtags, language, prompt, and generation settings.

Table Structure:

- **Id:** A unique identifier for each generated content record (auto-incremented).
- **IdealHashtags:** An optional field storing AI-generated hashtags.
- **IdealCaption:** An optional field storing the AI-generated caption.
- **Season:** Stored as a string representation of the Season enum (Spring, Summer, Autumn, Winter, or AllSeason).
- **Tone:** An optional field representing the writing tone.

- **Language**: Stored as a string representation of the CampaignLanguage enum (Arabic or English).
- **VariantNumber:** Identifies the generated variation of the content.
- **ImagePrompt:** The prompt used to generate the AI image.
- **CampaignId:** A foreign key referencing the associated campaign.
- **ProductId:** A foreign key referencing the associated product.
- **GeneratedAt:** The generation timestamp, automatically created using UTC time.

The **ON DELETE CASCADE** constraint ensures that deleting a campaign automatically removes all generated contents related to it. The ON DELETE RESTRICT constraint on ProductId preserves product records even if generated content exists. Each generated content record can also have one or more associated AI-generated images.

5.3.7 AI Generated Content ImagesTable

The AIGeneratedContentImages table stores the AI-generated images associated with each generated content record, allowing multiple generated images for a single content item.

Table Structure:

- **Id:** A unique identifier for each generated image (auto-incremented).
- **GeneratedImageURL:** The URL of the AI-generated image.
- **ContentId:** A foreign key linking the image to its generated content.
- **CreatedAt:** The creation timestamp, automatically generated using UTC time.

The **ON DELETE CASCADE** constraint ensures that when an AI-generated content record is deleted, all of its associated generated images are automatically removed, maintaining database consistency.

5.3.8 Enumerations

The system uses several enumerations (Enums) to restrict input values and ensure data consistency throughout the application. Although these values are represented as enumerations in the application code, they are stored in the database as string values using Entity Framework Core conversions. This approach improves database readability while preventing invalid values from being stored.

The following enumerations are used in the system:

- **ProductCategory:** Footwear, Apparel, Accessories.
- **GenderFocus:** Men, Women, Unisex.
- **CampaignPlatform:** TikTok, Facebook, Instagram.
- **CampaignGoal:** Sales, Brand Awareness, Engagement, Launch/Announcement, Conversion.
- **CampaignLanguage:** Arabic, English.
- **Season:** Spring, Summer, Autumn, Winter, AllSeason.
- **CampaignStatus:** Active, Ended.

Among these, CampaignStatus is not stored as a database column. It is implemented as a computed property using the [NotMapped] attribute and is evaluated dynamically at runtime based on the campaign's EndDate. A campaign is considered Active when it has no EndDate or when the current UTC date is earlier than the EndDate, and Ended when the current UTC date exceeds the EndDate. This design keeps the status synchronized with the campaign lifecycle while allowing future dashboard enhancements and additional status values without modifying the database schema.

5.3.9 Summary of DELETE Actions in the Database Schema

5.1.9 Summary of Delete Behaviors

The database applies different delete behaviors to maintain referential integrity and prevent unintended data loss. Relationships that require dependent records to be removed automatically use CASCADE, while relationships that should preserve existing records use RESTRICT. Table 5.x summarizes the delete behavior configured for each relationship in the system.

Table 5.x Summary of Delete Behaviors

Parent Table	Child Table	Relationship	Delete Behavior
AspNetUsers	Products	One-to-Many	RESTRICT
AspNetUsers	Campaigns	One-to-Many	RESTRICT
Products	ProductImages	One-to-Many	CASCADE
Campaigns	CampaignProducts	One-to-Many	CASCADE
Products	CampaignProducts	One-to-Many	RESTRICT
Campaigns	AIGeneratedContents	One-to-Many	CASCADE
Products	AIGeneratedContents	One-to-Many	RESTRICT
AIGeneratedContents	AIGeneratedContentImages	One-to-Many	CASCADE

Description

The RESTRICT delete behavior prevents deleting a parent record while dependent records still exist, thereby preserving data integrity. In contrast, the CASCADE delete behavior automatically removes dependent records when the parent record is deleted. This strategy ensures consistent relationship management across the database while protecting important business data from accidental deletion.

Figure 22 Delete Action

5.3.10 Explanation of Foreign Keys in the Database Schema

Foreign keys are used to establish relationships between tables, ensuring referential integrity. Below is a breakdown of the foreign keys in your schema and their relationships:

1. Products Table

- **UserID → AspNetUsers(Id)**
  - Links each product to the user who created it.

- **ON DELETE RESTRICT:** Prevents deleting a user while they still own products.

2. ProductImages Table

- **ProductId → Products(Id)**
  - Links each product image to its corresponding product.
  - **ON DELETE CASCADE**: If a product is deleted, all associated product images are automatically deleted.

3. Campaigns Table

- **UserId → AspNetUsers(Id)**
  - Links each campaign to the user who created it.
  - **ON DELETE RESTRICT:** Prevents deleting a user while they still own campaigns.

4. CampaignProducts Table

- **CampaignId → Campaigns(Id)**
  - Links a campaign-product record to a campaign.
  - **ON DELETE CASCADE:** If a campaign is deleted, all related campaign-product records are automatically deleted.
- **ProductId → Products(Id)**
  - Links a campaign-product record to a product.
  - **ON DELETE RESTRICT:** Prevents deleting a product while it is associated with one or more campaigns.

A unique constraint on (CampaignId, ProductId) prevents the same product from being added to the same campaign more than once.

5. AIGeneratedContents Table

- **CampaignId → Campaigns(Id)**
  - Links each AI-generated content record to the campaign in which it was generated.
  - **ON DELETE CASCADE**: If a campaign is deleted, all associated AI-generated content records are automatically deleted.
- **ProductId → Products(Id)**
  - Links each AI-generated content record to its corresponding product.
  - **ONDELETE RESTRICT:** Prevents deleting a product while AI-generated content still references it.

6. AIGeneratedContentImages Table

- **ContentId → AIGeneratedContents(Id)**
  - Links each generated image to its corresponding AI-generated content.
  - **ON DELETE Cascade**: If an AI-generated content record is deleted, all associated generated images are automatically deleted.

5.3.11 Database Schema Mapping

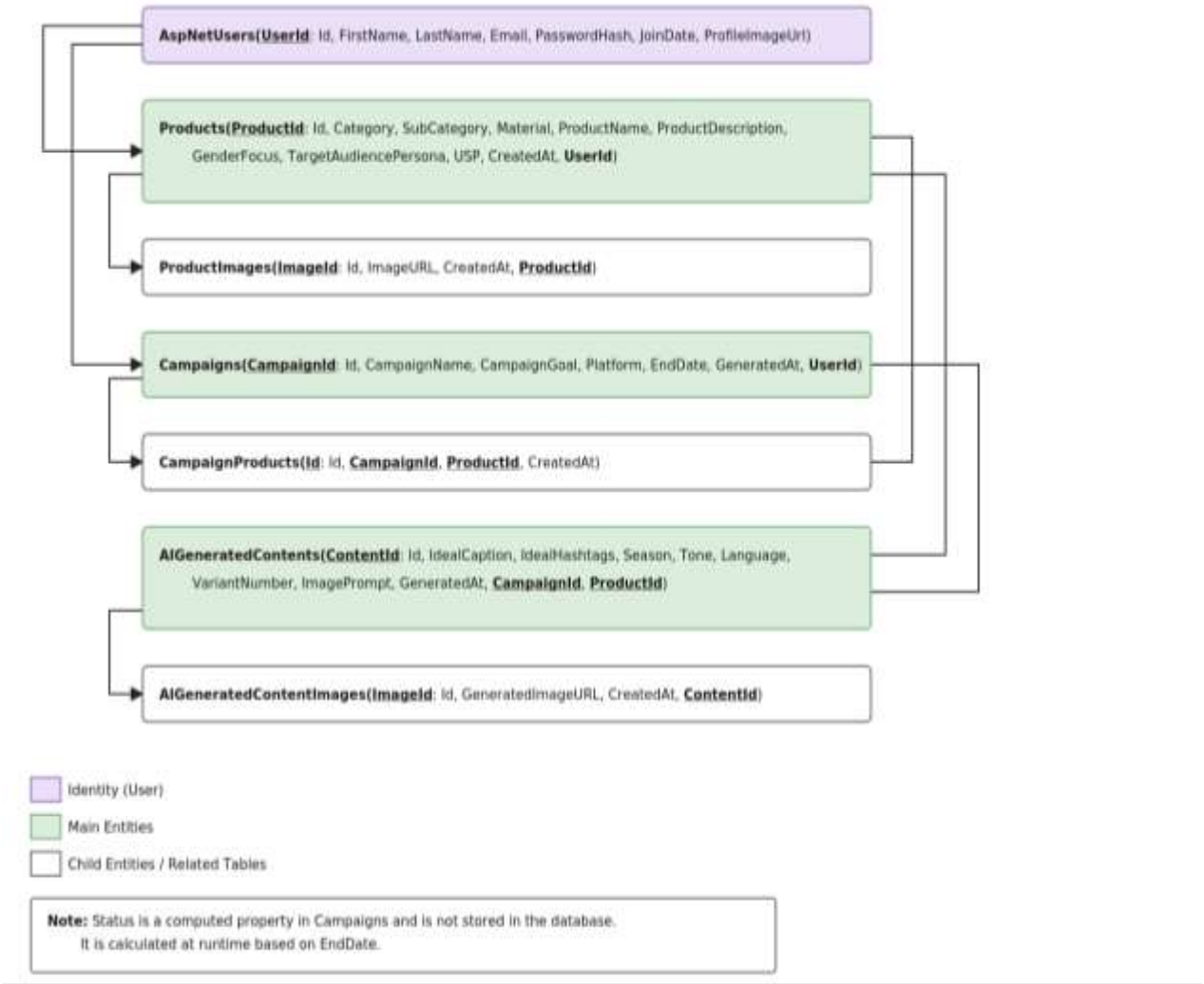


Figure 23 Database Shema Mapping

5.4 Backend Implementation

The backend of the AI Campaign Generator system was developed using ASP.NET Core 8 Web API, providing a secure, scalable, and high-performance architecture for building RESTful services. The backend is responsible for implementing the business logic, managing database operations, authenticating users, integrating with external AI services, and exposing REST APIs consumed by the Flutter mobile application.

- **Technologies Used**
  - **ASP.NET Core 8 Web API** – For developing RESTful APIs.
  - **Entity Framework Core** – As the Object-Relational Mapper (ORM).
  - **SQL Server** – Primary relational database.
  - **ASP.NET Core Identity** – User authentication and management.
  - **Cloudinary** – Cloud storage for product, profile, and AI-generated images.
  - **Google Gemini API** – AI content generation.
  - **OpenRouter API** – AI model integration.
  - **T5-**English AI text Generation
  - **RAG (Retrieval-Augmented Generation) –** Used to power the AI chatbot by combining retrieved knowledge with language model responses.
  - **SQL Server** – The primary relational database.
  - **JWT (JSON Web Token)** – For secure authentication and authorization.
  - **AutoMapper** – For mapping between entities and Data Transfer Objects (DTOs).
  - **Swagger** – For API documentation and testing.
  - **SMTP Email Service**-Password reset and email verification

5.4.1 Project Structure

The backend of the **AD Generator AI**(AI Campaign Generator)follows the Clean Architecture pattern, which separates the application into multiple layers, each responsible for a specific concern. This architecture improves maintainability, scalability, testability, and separation of responsibilities.

The solution is organized into the following layers:

Application Layer:

The Application Layer contains the business logic and application services.

It consists of the following projects:

- **AI_CampaignGenerator.Domain**
  This project contains the core domain of the application, including:
  - Entity classes organized into feature modules.include(Base Entity).
  - Enumerations (Enums).
  - Generic Repository Interface.
  - Unit of Work interface.
  - Specification interfaces.
  - Data initializer interface.
- **AI_CampaignGenerator.Services**
  Implements the application's business logic.
  It contains:
  - Services for Authentication, Users, Products, Campaigns, AI Generated Content, and Chatbot.
  - AI Integration services.
  - Token service.
  - Email service.
  - Cloudinary image storage service.
  - AutoMapper Mapping Profiles.
  - Specification classes.
  - Custom exception classes.
- **AI_CampaignGenerator.ServicesAbstraction**
  Contains all service interfaces implemented by the Services project, including:
  - IAuthenticationService

- IUserService
- IProductService
- ICampaignService
- IAIGeneratedContentService
- IChatbotService
- IEmailService
- ITokenService
- IAIIntegrationService
- IImageStorageService

Infrastructure Layer

The Infrastructure Layer contains the implementation of data access and database operations.

- **AI_CampaignGenerator.Persistence**

This project contains:

- Entity Framework Core DbContext.
- Entity Configurations.
- Generic Repository implementation.
- Unit of Work implementation.
- Specification Evaluator.
- Database Seeder.
- JSON seed files for Arabic and English data.
- Entity Framework Migrations.

Presentation Layer

The Presentation Layer exposes the RESTful APIs to the client application.

- **AI_CampaignGenerator.Presentation**

  Contains the API Controllers, including:

- AuthenticationController
- UsersController
- ProductsController
- CampaignsController
- AIGeneratedContentController
- ChatbotController

It also contains the ApiBaseController, which centralizes API response formatting and error handling.

- **AI_CampaignGenerator.Web**

Acts as the startup project,It contains:

- Program.cs:The application's entry point responsible for configuring services and middleware, including Dependency Injection, Entity Framework Core,JWT Authentication , Swagger, AutoMapper, CORS, custom services, exception handling, routing, endpoint mapping, and loading application configuration from environment variables.

- **appsettings.json:**Main configuration file for connection strings, JWT keys, and custom settings.

- **appsettings.Development.json**:Overrides for development environment settings:

  - Custom Middleware.
  - Extensions(WepApplication Registeration).
  - Publish profile.(monster deployment)
  - Factory(APi Response Fcatory)
  - wwwroot folder used during local development for uploaded images.

Shared Layer

- **AI_CampaignGenerator.Shared**

Contains components shared across the application, including:

- DTOs (User,product,identity)
- Result and Error handling classes.
- Pagination Result class.

- Campaign Query params class.
- Campaign Sorting options.
- Chatbot DTOs.
- Authentication DTOs.
- Campaign DTOs.
- AI Generated Content DTOs.
- Posting Suggestion DTOs.
- Commonresult folder (Result,Errorand enum od ErrorTypes)

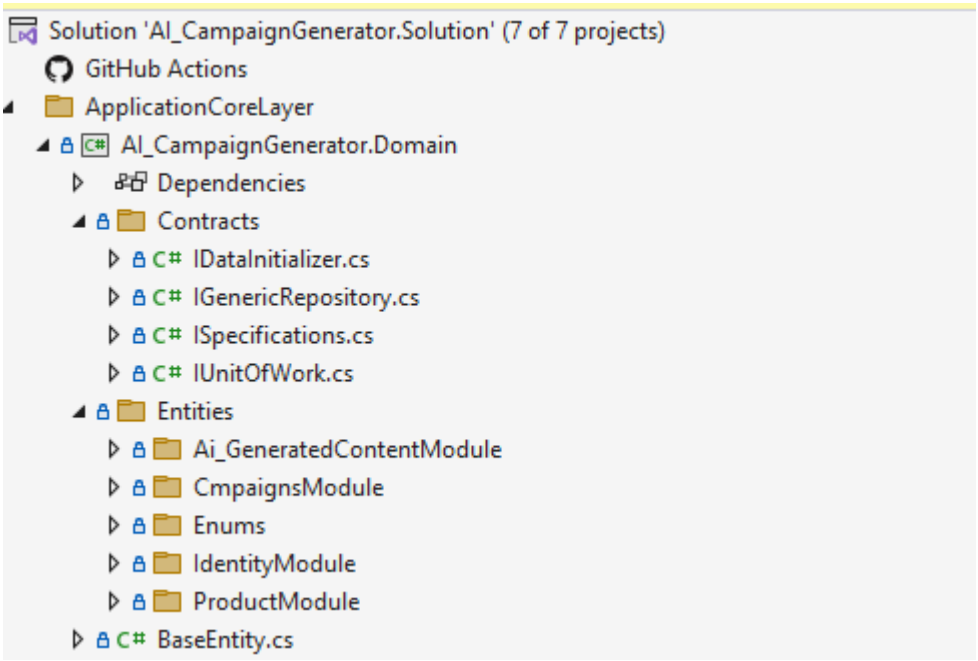


Figure 24Backend Project(Demainlayer)

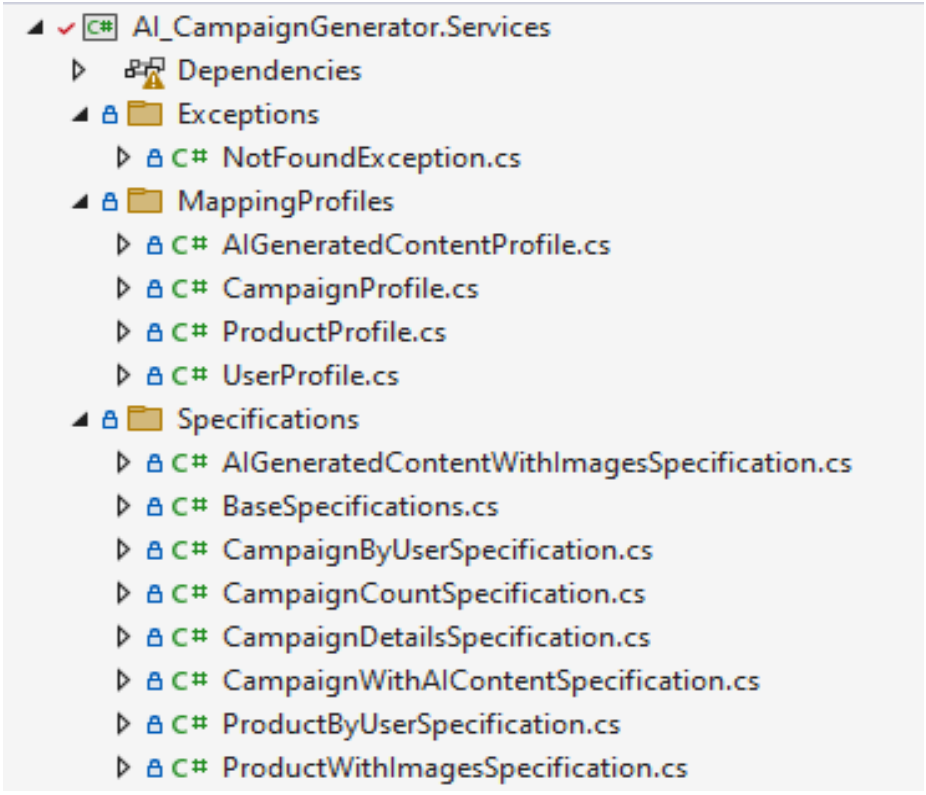
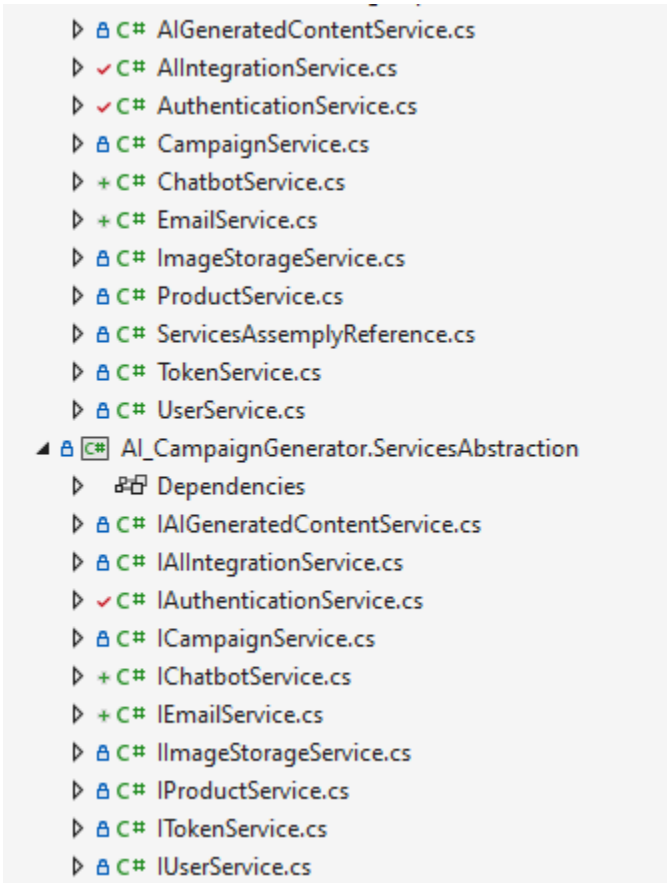


Figure 25Backend Project(Serviceslayer)



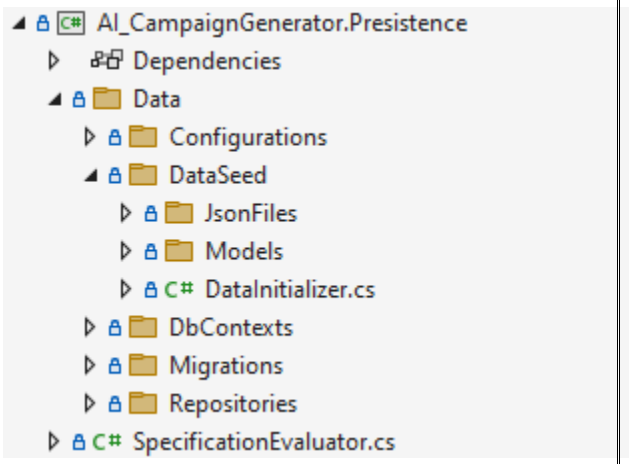


Figure 26 SevicesAbstraction Layer

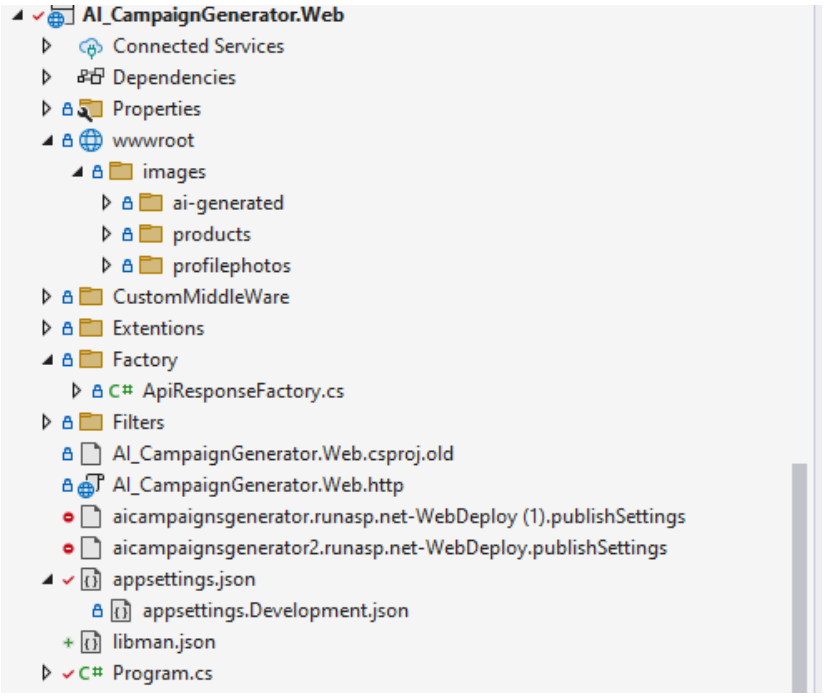


Figure 28 presitence (Web strucure)

Figure 27 Presistence Layer

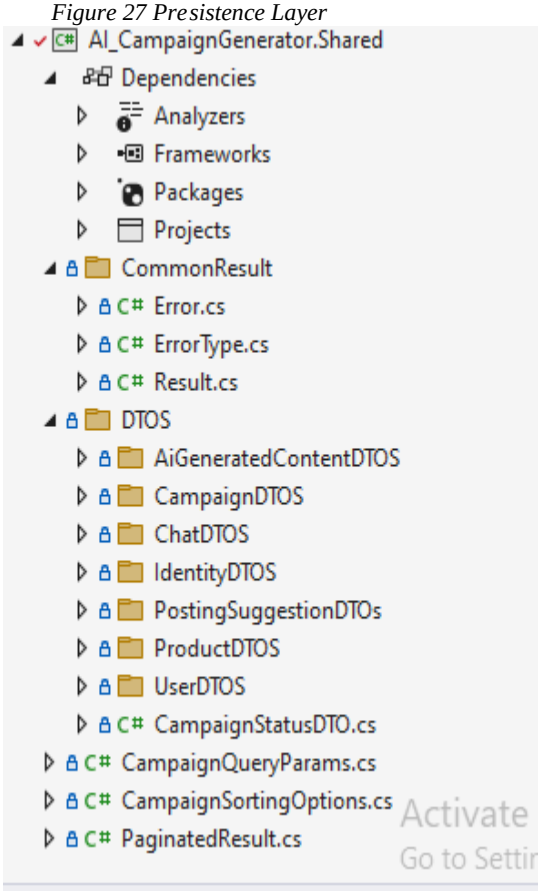


Figure 29 Backend project (Shared Layer)

5.4.2 Application User Model

The ApplicationUser model represents each registered user in the AI Campaign Generator system. It extends the built-in ASP.NET Core IdentityUser class to provide authentication, authorization, and user management while storing additional profile information specific to the application.

The model stores essential user information, including the user's first name, last name, email address, profile image, and account creation date. Authentication is handled securely through ASP.NET Core Identity, which manages password hashing, email uniqueness, security stamps, authentication tokens, and other security-related features.

From a system perspective, the ApplicationUser model serves as the central entity that connects users with the application's core functionalities:

- Each user can create and manage multiple products that are used for AI-generated marketing campaigns.
- Users can create and manage multiple marketing campaigns, each containing one or more products.
- Through their campaigns, users can generate AI-powered marketing content, including captions, hashtags, and AI-generated images.

- Users can update their profile information, including their profile picture, which is securely stored using Cloudinary.
- Authentication is supported through both traditional email/password login and Google Sign-In, providing users with multiple secure authentication options.
- Although the system architecture supports both User and Admin roles using ASP.NET Core Identity, the current implementation focuses on the User role. The Admin role has been prepared as a future enhancement to support dashboard management, monitoring, and administrative functionalities.
- By acting as the owner of products and campaigns, the ApplicationUser model forms the foundation of the system and enables personalized AI-powered marketing experiences for every registered user.

5.4.3 Product Model

The Product model represents the products that users want to promote through AI-generated marketing campaigns. It stores detailed information describing each product, enabling the AI module to generate personalized marketing content tailored to the product's characteristics and target audience.

The model includes essential product information such as the product name, category, subcategory, material, description, target gender, target audience persona, and the product's Unique Selling Proposition (USP). These attributes provide the AI models with sufficient context to generate relevant captions, hashtags, and promotional content.ddress, which also serves as their login credential along with a secured password.

relationship, while a user can own multiple products. This ownership ensures that every product is managed only by its creator.

The model also supports uploading multiple product images. Images are stored securely in Cloudinary, allowing the AI services to analyze product visuals during content generation while keeping media storage separate from the database.

From a system perspective, the Product model is closely integrated with several core modules:
- A product can contain multiple uploaded images.
- A product can participate in multiple marketing campaigns through the CampaignProducts junction table.
- Each product can have multiple AI-generated marketing content records associated with different campaigns.

By serving as the primary source of product information, the Product model plays a fundamental role in producing accurate, personalized, and high-quality AI-generated marketing campaigns.

5.4.4 Porduct Images Model

The ProductImages model stores the images associated with each product in the AI Campaign Generator system. Instead of storing the images directly, it stores their Cloudinary URLs, improving storage efficiency and performance.

Each image belongs to one product, while a product can have multiple images through a one-to-many relationship. The relationship uses ON DELETE CASCADE, ensuring that all associated images are automatically deleted when their parent product is removed.

This model provides the AI module with visual information that enhances the quality of the generated marketing content.

5.4.5 Campaign Model

The Campaign model represents the marketing campaigns created by users within the AI Campaign Generator system. It stores the essential campaign information required to organize products, generate AI-powered marketing content, and manage campaign lifecycle information.

The model includes key campaign details such as the campaign name, target social media platform, campaign goal, optional end date, and the generation date. These attributes provide the necessary context for the AI module to generate platform-specific marketing content that aligns with the campaign objective.

Each campaign belongs to a single registered user through a foreign key relationship, while a user can create multiple campaigns. In addition, a campaign can include multiple products through the CampaignProducts junction table, allowing users to generate marketing content for several products within the same campaign.

The model also maintains a one-to-many relationship with the AIGeneratedContent model, where each campaign can produce multiple AI-generated post variants for its associated products.

Unlike the other properties, the Status property is marked as NotMapped, meaning it is not stored in the database. Instead, it is computed in real time by comparing the current date with the campaign's EndDate. A campaign is considered Active if the current date has not passed the end date or if no end date is specified, and Ended once the current date exceeds the end date. This approach avoids storing redundant data while providing flexibility for future dashboard features and additional campaign statuses.

From a system perspective, the Campaign model is closely integrated with several core modules:

- Each campaign belongs to one registered user.
- A campaign can contain multiple products through the CampaignProducts junction table.
- A campaign can generate multiple AI-generated content records.
- The campaign status is computed dynamically rather than stored in the database.

By acting as the central entity that connects users, products, and AI-generated content, the Campaign model plays a fundamental role in organizing and managing the complete AI-powered marketing campaign workflow.
Powering track-specific assessments and eligibility checks.

5.4.6 CampaignProductModel

The CampaignProduct model serves as a junction table that implements the many-to-many relationship between Campaigns and Products in the AI Campaign Generator system. It enables a single campaign to include multiple products, while the same product can also be associated with different campaigns.

The model contains two foreign keys: CampaignId, which references the associated campaign, and ProductId, which references the associated product. Together, these keys establish the relationship between campaigns and products without duplicating product information.

To maintain data integrity, the model enforces a unique constraint on the combination of CampaignId and ProductId, preventing the same product from being added more than once to the same campaign.

From a system perspective, the CampaignProduct model is closely integrated with the campaign management workflow:

- Each record belongs to one campaign.
- Each record references one product.
- A campaign can contain multiple products.
- A product can participate in multiple campaigns.

By resolving the many-to-many relationship between campaigns and products, the CampaignProduct model provides the flexibility needed to organize multiple products within a single marketing campaign while maintaining a normalized database structure.

5.4.7 AIGeneratedContent Model

The AIGeneratedContent model represents the AI-generated marketing posts created for products within a campaign. It stores the textual content produced by the AI models along with the generation settings used to create each post.

The model includes the generated caption, hashtags, image prompt, language, season, tone, variant number, and the generation date. These attributes define both the generated marketing content and the parameters that guided the AI during content generation.

Each generated content record belongs to a specific Campaign and a specific Product through foreign key relationships. This allows the system to generate multiple post variations for different products within the same campaign while maintaining clear associations between campaigns, products, and their generated content.

The model also supports storing multiple AI-generated images through the AIGeneratedContentImages model. Images are generated based on the stored image prompt and are uploaded to Cloudinary, where only their URLs are saved in the database.

To ensure high-quality and consistent AI output, several generation parameters are validated before the content is generated. The Language field accepts only the supported languages (Arabic and English), while the Tone, Style, Occasion, Emotion, and Call-to-Action (CTA) fields are restricted to predefined values that match the available options in the frontend dropdown menus. Any value outside these predefined options is rejected during request validation before the AI generation process begins.

From a system perspective, the AIGeneratedContent model is closely integrated with several core modules:

- Each generated content record belongs to one campaign.
- Each generated content record belongs to one product.
- Each generated content record can contain multiple AI-generated images.
- Generation parameters are validated before AI processing to ensure consistent and reliable results.

By serving as the repository for AI-generated marketing content, the AIGeneratedContent model forms the core of the AI generation workflow, connecting campaigns, products, and generated media into a unified marketing solution.

5.4.8 AI Generated Content Images Model

The AIGeneratedContentImages model stores the AI-generated images associated with each generated marketing post. It allows a single generated content record to contain multiple AI-generated images, giving users different visual options for the same campaign.

Each image belongs to one AIGeneratedContent record through a foreign key relationship. The images are uploaded to Cloudinary, while only their URLs are stored in the database to improve scalability and reduce database storage.

From a system perspective:

- Each generated image belongs to one AI-generated content record.
- A generated content record can have multiple AI-generated images.
- Only image URLs are stored in the database, while the actual images are managed in Cloudinary.

By separating images from the generated content, the model maintains a clean and scalable database design while supporting multiple AI-generated visuals for each marketing post.

5.4.9 AI Marketing ChatBot

The AI Marketing Chatbot is an intelligent assistant integrated into the Ad Generator AI system to provide users with marketing guidance and support. Unlike the other modules, the chatbot is implemented as an external AI service hosted on Hugging Face and communicates with the ASP.NET Core backend through RESTful APIs. Therefore, it does not require a database entity or model class; instead, communication is handled using Data Transfer Objects (DTOs).

The chatbot is built using a Retrieval-Augmented Generation (RAG) architecture. Before generating a response, it retrieves the most relevant information from its indexed marketing knowledge base, allowing it to produce more accurate and context-aware answers. If the required information is not available in the local knowledge base, the chatbot can optionally perform a web search to retrieve up-to-date marketing information, providing more comprehensive responses when needed.

The chatbot is specialized in digital marketing and campaign creation. It assists users by answering marketing-related questions, suggesting campaign ideas, explaining marketing concepts, and providing recommendations that support the campaign generation process.

The chatbot module supports several core functionalities:

- Users can ask marketing-related questions and receive AI-generated responses.
- Previous conversation history can be sent to maintain conversational context.
- Responses include references to the knowledge sources used by the RAG system whenever available.
- The chatbot supports both standard responses and real-time streaming responses for a more interactive user experience.

- Users may optionally enable web search to complement the local knowledge base with current online information.
- Health monitoring endpoints are available to verify that the chatbot service and vector index are operating correctly.

Although the backend includes administrative endpoints for uploading knowledge files and rebuilding the vector index, these features were implemented as preparation for future dashboard development. In the current application, only authenticated users interact with the chatbot by sending questions, while knowledge base management remains reserved for future administrative functionality.

5.5 APIs Implementation

The Ad Generator AI system exposes a collection of RESTful APIs developed using ASP.NET Core. These APIs act as the communication layer between the Flutter mobile application and the backend services, enabling secure and efficient data exchange. Each module provides a dedicated set of endpoints responsible for user authentication, product management, campaign management, AI content generation, user profile management, and chatbot interaction.

The APIs follow RESTful design principles and support standard HTTP methods, including GET, POST, PUT, and DELETE. All requests and responses are exchanged in JSON format, while image uploads are handled using multipart/form-data. Authentication and authorization are secured using JSON Web Tokens (JWT), ensuring that only authenticated users can access protected resources.

Although the backend includes several administrative endpoints to support future dashboard development, the current mobile application focuses on user-related functionality.

5.5.1 Authentication APIs

- POST /api/Authentication/register: Registers a new user account.
- POST /api/Authentication/login: Authenticates users and returns a JWT token.
- POST /api/Authentication/google-login: Authenticates users using their Google account.
- GET /api/Authentication/current-user: Returns information about the authenticated user.

- GET /api/Authentication/email-exists: Checks whether an email address is already registered.
- POST /api/Authentication/forgot-password: Sends a password reset request.
- POST /api/Authentication/reset-password: Resets the user's password.

These endpoints provide secure authentication and account management services.

5.5.2 User APIs

- GET /api/Users/me: Retrieves the authenticated user's profile.
- PUT /api/Users/me: Updates the user's profile information and profile picture.
- DELETE /api/Users/me: Deletes the authenticated user's account.

These endpoints allow users to manage their personal account information.

5.5.3 Product APIs

- GET /api/Products/my-products: Retrieves all products belonging to the authenticated user.
- GET /api/Products/{id}: Retrieves a specific product.
- POST /api/Products: Creates a new product with its images.
- PUT /api/Products/{id}: Updates an existing product.
- DELETE /api/Products/{id}: Deletes a product.

These APIs enable users to manage the products used for AI campaign generation.GET /api/Doctor/Projects: Returns all projects supervised by the doctor.

5.5.4 Campaign APIs

- GET /api/Campaigns: Retrieves the user's campaigns with pagination and filtering.
- GET /api/Campaigns/{id}: Retrieves campaign details.
- GET /api/Campaigns/{id}/status: Returns the current campaign status.
- POST /api/Campaigns: Creates a new marketing campaign.
- POST /api/Campaigns/{campaignId}/products: Associates a product with a campaign.
- DELETE /api/Campaigns/{id}: Deletes a campaign.

These endpoints allow users to organize and manage AI marketing campaigns.

5.5.5 AI Genertaed content APIs

The AI Content Generation API is responsible for generating AI-powered marketing content and images for a selected product within a marketing campaign.

The endpoint receives the campaign identifier together with the content generation preferences specified by the user. Based on these inputs, the AI module generates captions, hashtags, image prompts, AI-generated images, and platform-specific posting suggestions.

- POST /api/AIGeneratedContent/generate/{campaignId}Path Parameter

CampaignId: A required parameter that identifies the campaign for which the AI content will be generated.

Request Body:

- ProductId: Identifies the product within the selected campaign for which the AI content will be Generated.
- Language: Specifies the language of the generated content. Supported values are Arabic and English.
- Season: Defines the seasonal context used during content generation. Supported values are Spring, Summer, Autumn, Winter, and AllSeason.
- Tone: Specifies the writing tone of the generated content. Supported values include Friendly, Luxury, Bold, Formal, Casual, Minimal, Confidence, and Professional.
- Style: Defines the fashion or visual style reflected in the generated marketing content. Supported values include Casual, Formal, Modest, Luxury, Sporty, Elegant, and Trendy.
- Occasion: Specifies the intended usage scenario of the product. Supported values include Work, Travel, Party, Daily, Wedding, Gym/Beach, Sports, and Ramadan/Eid.
- Emotion: Defines the emotional appeal of the generated marketing content. Supported values include Confidence, Comfort, Elegance, Joy, Excitement, Trust, Desire, and Pride.
- CTA (Call To Action): Specifies the marketing call-to-action included in the generated caption. Supported values include Shop Now, Discover More, Explore Collection, Buy Today, and Limited Offer.

Validation:

Before the AI generation process begins, all request parameters are validated to ensure that the provided values match the supported options. Invalid or unsupported values result in a validation error before reaching the AI generation service. This validation improves prompt consistency, maintains generation quality, and prevents invalid requests from being processed.

5.5.6 AI Marketing Chatbot APIs

- POST /api/Chatbot/ask: Sends a marketing question to the AI chatbot and receives a complete response.

- POST /api/Chatbot/ask/stream: Streams the chatbot response in real time for a more interactive experience.

The chatbot is implemented using a Retrieval-Augmented Generation (RAG) architecture and is hosted on Hugging Face. It retrieves information from its indexed marketing knowledge base and can optionally perform web searches to provide up-to-date marketing information. In the current mobile application, users interact with the chatbot only through these two endpoints.

5.5.7 Future Administrative APIs

The backend also contains several administrative endpoints, including knowledge base ingestion, vector index rebuilding, health monitoring, and user management. These APIs were implemented to support future dashboard functionality and system maintenance. They are not currently exposed in the mobile application and therefore are outside the scope of the current user interface.

5.5.8 Android App links Endpoint

To support Android App Links after deploying the backend on MonsterASP.NET,

a dedicated endpoint was implemented directly inside the Program.cs file:

- GET /.well-known/assetlinks.json

This endpoint dynamically returns the required assetlinks.json file that verifies the

relationship between the deployed backend and the Android application. This approach was adopted

because the hosting environment does not allow serving static files from the .well-known directory. As a result, Android can successfully verify the application and support secure deep linking while allowing authenticated requests to function correctly after deployment.feature.

5.6 AI Implementation — Text and Hashtag Generation

5.6.1 Overview

The **AD GENERATOR AI** system relies on two complementary AI pipelines to produce platform-ready marketing text and hashtags for each generated campaign. The first pipeline is a fine-tuned T5-Small transformer model trained on a 50,000-record English-language fashion marketing dataset, which generates caption text and hashtags directly from structured product attributes. The second pipeline is a Gemini-based API module that handles Arabic-language campaigns and applies strict linguistic post-processing rules to ensure output quality, register consistency, and platform adaptability. Together, these two pipelines cover both the English and Arabic output modes supported by the application.

5.6.2 Dataset Collection (Web Scraping Phase)

The dataset used in this project was collected using web scraping techniques from multiple fashion e-commerce platforms to ensure diversity and realism in marketing content generation.

Data Sources Used:

https://www.zalando.com

https://www.hm.com

https://www.asos.com

https://www.zara.com

https://www.shein.com

https://www.farfetch.com

These platforms were selected because they provide rich fashion-related metadata such as product descriptions, categories, seasonal styles, and marketing-oriented text suitable for AI training.
The collected dataset contains approximately 50,000 records representing different fashion products, styles, and marketing scenarios.

5.6.3 Dataset Description

The dataset used in this project, designated dataset_50k_FINAL_CLEANED, contains exactly 50,000 records representing structured product marketing scenarios in the fashion domain. Each record captures

the full context required to generate a marketing post: product identity, material, style, target audience, platform, tone, emotion, and call-to-action. Alongside these 16 input fields, the dataset provides three pre-generated output columns per record — covering the product-focused, lifestyle-focused, and campaign-focused post variants — as well as their corresponding hashtag columns and combined full-output columns. The total schema spans 25 columns per row.

The dataset includes the following fields:

Column Name	Description
Product_Category / Sub_Category	High-level and specific product classification (e.g., Footwear, Women Fashion, Accessories)
Product_Name	The exact product name, kept in English throughout generation without translation
Product_Description_EN	A short descriptive sentence about the product's design, fit, and features
Material	Primary fabric or material (e.g., Linen, Silk, Suede, Mixed Metal)
Season	Intended seasonal context (Spring, Summer, Autumn, Winter)
Fashion_Style	Style identity such as Minimal Elegant, Casual, Bold, or Streetwear
Occasion	Recommended use context (e.g., Daily, Travel, Casual outing, Work)
Gender	Target gender segment (Women, Men, Unisex)
Target_Audience	Age and lifestyle description of the intended customer
Campaign_Goal	The marketing objective (e.g., Brand Awareness, Launch/Announcement, Conversions)
Platform	Social media or digital platform (Instagram, TikTok, Facebook, Website)
Tone	Intended communication register (e.g., Warm, Bold, Minimal, Professional)
Emotion	Emotional target of the campaign (e.g., Confidence, Excitement, Trust)
Call_to_Action	The explicit CTA phrase (e.g., Shop Now, Discover More)
USP	Unique Selling Proposition — the core product differentiator
generate_text_product / lifestyle /	Pre-generated English marketing captions for each of the

campaign	three post variants
generate_hashtags_product / lifestyle / campaign	Pre-generated English hashtag sets for each variant
full_output_product / lifestyle / campaign	Concatenated text + hashtags — used as the final training target per variant

5.6.4 Fine-Tuned T5-Small Model (English Generation)

5.6.4.1 Model Selection and Architecture

The English text generation pipeline is built on the T5-Small transformer model (Text-to-Text Transfer Transformer), introduced by Raffel et al. (2020) at Google Research. T5 treats every NLP task as a sequence-to-sequence text generation problem by converting structured inputs into a unified text-to-text format, making it well suited for tasks such as controlled marketing copy generation. The T5-Small variant was selected to balance generation quality with computational feasibility during training on Google Colab GPU hardware. The model uses an encoder-decoder architecture and was pre-trained on the Colossal Clean Crawled Corpus (C4) using a span-corruption denoising objective, then fine-tuned on our domain-specific fashion marketing dataset.

Property	Value
Base model	google-t5/t5-small
Total parameters	~60 million
Architecture	Encoder-Decoder Transformer
Training type	Full fine-tuning (no LoRA, no PEFT)
Training hardware	Google Colab GPU (CUDA, FP16 mixed precision)
Input token limit	512 tokens
Output token limit	256 tokens
Framework	Hugging Face Transformers v4.45.0

5.6.4.2 Data Preparation and Training Format

Each of the 50,000 records in the dataset was expanded into three separate training examples — one per variant (product, lifestyle, campaign) — producing a total of 150,000 training pairs. During preprocessing, the three full_output columns available in the dataset were dropped and rebuilt from scratch by combining the individual generate_text and generate_hashtags columns per variant. This ensured clean, deterministic targets without any pre-existing concatenation artifacts.

Each training example follows a prefix-driven text-to-text format. The task instruction prefix is prepended to the structured product input, which contains 11 key fields. The three task prefixes used are:

- generate product text: — instructs the model to focus on the product's material, design features, and USP
- generate lifestyle text: — instructs the model to focus on mood, occasion, season, and audience fit
- generate campaign text: — instructs the model to focus on launch urgency, CTA, and campaign objective

The structured input block provided to the model for all three prefixes is identical and contains the following fields in order: Product, Category, Material, Season, Style, Occasion, Target, Platform, Tone, CTA, and USP. The corresponding output for each variant is the marketing caption text followed by a blank line and the hashtag string, separated as it appears in the individual generate_text and generate_hashtags columns.

The 150,000 training pairs were split sequentially (not shuffled) into a 90% training set of 135,000 examples and a 10% validation set of 15,000 examples, then saved as two JSONL files (train.jsonl and validation.jsonl) for efficient line-by-line loading during training.

5.6.4.3 Training Configuration

The model was trained using the Hugging Face Trainer API with early stopping enabled. The following hyperparameters were used:

Hyperparameter	Value
Number of epochs	3
Batch size (train / eval)	8 per device
Learning rate	3×10^{-4}
Weight decay	0.01
Warmup steps	500
Evaluation strategy	Every 500 steps
Save steps	Every 500 steps
Checkpoints retained	Last 3 only (save_total_limit = 3)
Early stopping patience	3 evaluations
Early stopping threshold	0.01 improvement required
Best model selection metric	eval_loss (lowest)
Mixed precision	FP16 (when CUDA available)
Optimizer	AdamW (Hugging Face Trainer default)

Training log interval	Every 100 steps
Report to	None (no external logging service)

5.6.4.4 Inference Configuration

At inference time, the application constructs a structured input string from the user-submitted campaign details and prepends the appropriate task prefix (product, lifestyle, or campaign). The T5 tokenizer encodes this input and the model generates output using the following decoding parameters:

- max_length: 256 — maximum output token length
- num_beams: 5 — beam search for higher-quality output
- temperature: 0.8 — controlled randomness for natural variation
- do_sample: True — enables sampling-based decoding
- top_p: 0.9 — nucleus sampling threshold

The decoded output string is then passed to the post-processing function before being returned to the application.

5.6.4.5 Output Post-Processing

Raw T5 output is passed through a post-processing function (add_emojis_and_format) that adds visual formatting to the generated marketing text. The function splits the output at sentence boundaries — specifically at periods, Arabic commas, and semicolons — appends a randomly selected fashion-themed emoji before the closing punctuation of each sentence, and joins the resulting lines with newline breaks. A set of 10 emojis is used: □ ,□,□, □,□,□,□,□,□,□. The hashtag section of the output is detected by locating the first # character and is left completely unformatted to preserve the raw hashtag string for platform delivery.

A separate clean_and_split_output function is also applied to cleanly separate the caption text from the hashtag block. It uses regex to find all #word patterns, locates the first hashtag position in the output string, and splits there. Any stray emojis present in the hashtag section are removed. A validate_hashtag_count function then trims or pads the hashtag list to match the target count for the selected platform, using a set of generic fashion hashtags as filler when needed.

Platform	Target Hashtag Count	Notes
Instagram	20	No special required tags
TikTok	5	#fyp and #viral are automatically prepended
Facebook	5	No special required tags
Website / Ads	6	No special required tags

5.6.4.6 Inference



5.6.5 Qwen2.5 Local Pipeline (Arabic Generation)

5.6.5.1 Overview

The **AD GENERATOR AI** system relies on two complementary AI pipelines to produce platform-ready marketing text and hashtags for each generated campaign. The first pipeline is a fine-tuned T5-Small transformer model trained on a 50,000-record English-language fashion marketing dataset, which generates caption text and hashtags directly from structured product attributes. The second pipeline is a locally hosted Qwen2.5-7B-Instruct model, loaded from Hugging Face and run on-device, which handles Arabic-language campaigns using structured prompt engineering and few-shot examples to ensure output quality, register consistency, and platform adaptability. Together, these two pipelines cover both the English and Arabic output modes supported by the application.

5.6.5.2 Model Selection and Architecture

Arabic-language campaign generation is handled by the Qwen2.5-7B-Instruct model, developed by Alibaba Cloud and loaded directly from Hugging Face. This model was selected for the Arabic pipeline for several reasons: it provides strong Modern Standard Arabic (فصحى) language support, its Instruct variant is specifically optimized for following structured instructions and producing consistent outputs — which is essential for prompt engineering and few-shot techniques — and its 7-billion parameter size strikes an effective balance between generation quality and computational feasibility on Google Colab GPU hardware.

The model is loaded locally using the Hugging Face Transformers library with 4-bit quantization (BitsAndBytesConfig) to reduce memory consumption, allowing it to run on a standard T4 GPU without exceeding

VRAM limits. Inference is performed entirely on-device — no external API calls are made for Arabic generation, eliminating API costs and external service dependencies.

Property	Value
Model	Qwen/Qwen2.5-7B-Instruct
Source	Hugging Face Hub (loaded locally)
Parameters	7 billion
Architecture	Decoder-only Transformer (causal language model)
Variant	Instruct — optimized for instruction following
Arabic support	Native — strong Modern Standard Arabic (فصحى) capability
Quantization	4-bit (BitsAndBytesConfig) — reduces VRAM to ~6–8 GB
Training hardware	Google Colab GPU (NVIDIA T4, 15 GB VRAM)
Inference location	Local / on-server — no external API call
Framework	Hugging Face Transformers + BitsAndBytes
Cost per inference	Zero — fully self-hosted

5.6.5.3 Structured Output Schema

The model is instructed via the system prompt to return a strictly formatted JSON object conforming to the following schema. This eliminates the need for unreliable post-hoc parsing of free-form text and ensures all required fields are always present in the response.

```
PostBlock:
    text_ar: str        — Arabic marketing caption, one sentence per line
    hashtags_en: List[str]  — English hashtags, each prefixed with #

PlatformOutput:
    platform: str
    product_post: PostBlock
    lifestyle_post: PostBlock
    campaign_post: PostBlock
```

5.6.5.4 Platform-Specific Profiles

The system defines a PLATFORM_PROFILES dictionary that controls generation behavior per platform. These profiles are injected into the system prompt at generation time, ensuring that caption length, tone, hashtag count, hook style, and selling angle adapt to each platform's audience expectations.

Platform	Hashtags	Lines	Words/Line	Hook Style	Selling Angle
Instagram	20	3 short	6–14	Visual / aspirational	Elegance and confidence

TikTok	5	2 very short	4–10	Fast and catchy	Instant desire and trend energy
Facebook	6	4 medium	7–16	Practical and relatable	Comfort and daily value
Website	5	2 medium	7–16	Benefit-led	Quality and conversion

5.6.5.5 Prompt Engineering and Few-Shot Examples

The Arabic generation pipeline uses a two-layer prompting strategy applied through the Qwen2.5-7B-Instruct model's chat template format. The system prompt defines the model's role, language constraints, and output structure, while the user prompt embeds the product data alongside a complete few-shot example that demonstrates the exact expected output format.

System Prompt — Language Rules:

- All generated text must be in Modern Standard Arabic (فصحى) only — no dialect, slang, or colloquial Arabic is permitted
- The product name must remain in English exactly as provided — must not be translated under any circumstances
- Only gender-neutral singular imperative verb forms are allowed: جرب احصل، انطلق، جدد، تسوق، اكتشف، استمتع،
- Feminine imperative forms (اكتشفي استمتعي،) and plural forms (اكتشفوا استمتعوا،) are explicitly forbidden
- All hashtags must be in English and must begin with the # character
- Return only valid JSON — no preamble, no explanation, no markdown code fences

System Prompt — Formatting Rules:

- Each line of the post must appear on a separate line — block paragraph format is rejected
- One relevant emoji must be placed at the end of each line, immediately before the closing punctuation
- Number of lines per post must exactly match the platform profile specification
- Number of hashtags per variant must exactly match the platform profile target count
- Product posts focus on specs and material only — lifestyle posts on mood and occasion — campaign posts on urgency and CTA — no cross-contamination between angles

User Prompt — Few-Shot Example:

Before the actual product data, the user prompt includes one complete few-shot example demonstrating the exact JSON output structure. This example is constructed using a neutral reference product and is injected at the start of every user prompt. The few-shot example eliminates structural ambiguity and significantly improves output consistency, reducing formatting errors and invalid JSON responses compared to zero-shot prompting alone.

```
المطلوب على مثال:
المنتج: Linen Summer Dress | المنسوج: كتان | الموسم: صيف | النبرة: أنيقة | المنصة:
Instagram
{
```

```
  "product_post": {
    "text_ar": "فستان اكتشف Linen Summer Dress من المصنوع الفاخر الكتان 
               واحد آنٍ في والراحة بالأناقة استمتع
               مميزة صيفية إطلالة وعش الآن تسوق ",
    "hashtags_en": ["#LinenDress", "#SummerFashion", "#ElegantStyle", ...]
  },
  "lifestyle_post": { ... },
  "campaign_post": { ... }
}

التالي للمنتج الهيكل نفس أنشئ الآن:
[الفعلي المنتج بيانات]
```

5.6.5.6 Inference Configuration

At inference time, the product data and selected platform profile are injected into the user prompt alongside the few-shot example. The complete prompt is formatted using the Qwen2.5 chat template and tokenized before being passed to the model. The following generation parameters are used to balance output quality and response consistency:

Parameter	Value
max_new_tokens	512 — sufficient for all 3 post variants + hashtags
temperature	0.7 — controlled creativity without excessive randomness
top_p	0.9 — nucleus sampling for natural language variation
do_sample	True — enables sampling-based decoding
repetition_penalty	1.1 — reduces repetitive phrases across post variants

5.6.5.7 Post-Processing: Arabic Verb Normalization

Despite the strict prompt rules and few-shot grounding, the model occasionally produces feminine or plural imperative forms that violate the required neutral register. A post-processing normalization layer (fix_verbs) is applied to every generated response before it is returned to the application. The function uses two lookup lists — VERB_TO_NEUTRAL for verb forms and PHRASE_TO_NEUTRAL for possessive phrases — and applies all corrections via regular expression substitution.

Feminine Form (Forbidden)	Plural Form (Forbidden)	Normalized Neutral Form
استمتعي	استمتعوا	**استمتع**
اكتشفي	اكتشفوا	**اكتشف**
تسوقي	تسوقوا	**تسوق**
جددي	جددوا	**جدد**
انطلقي	انطلقوا	**انطلق**

احصلي	احصلوا	**احصل**
جربي	جربوا	**جرب**
إطلالتكم / إطلالتي	—	**إطلالتك**

A separate fix_hashtags function ensures every hashtag in the response begins with the # character, and a JSON repair step attempts to recover and parse any malformed JSON output before falling back to an empty response.

5.6.6 Pipeline Comparison Summary

Dimension	T5-Small (English Pipeline)	Qwen2.5-7B-Instruct (Arabic Pipeline)
Output language	English	Arabic (Modern Standard only)
Model type	Fine-tuned local transformer	Locally hosted instruction-tuned LLM
Model source	Hugging Face (google-t5/t5-small)	Hugging Face (Qwen/Qwen2.5-7B-Instruct)
Parameters	~60 million	7 billion
Training on domain data	Full fine-tuning on 150,000 pairs	No fine-tuning — prompt engineering only
Task specification	Prefix in input string	System prompt + platform profile + few-shot example
Prompt technique	Not applicable	Prompt engineering + few-shot examples
Platform adaptation	Learned from training labels	Injected dynamically per request
Structured output	Text split by delimiter + regex	JSON schema enforced via prompt + repair
Post-processing	Emoji formatting + hashtag validation	Verb normalization + hashtag prefix fix
Hallucination mitigation	Domain-constrained training	Forbidden-content rules + few-shot grounding
Inference location	Local / on-server	Local / on-server (4-bit quantized)
Cost per inference	Zero	Zero (self-hosted)

5.6.6 Gemini API Pipeline (Arabic Generation)

5.6.6.1 Model and Integration

Arabic-language campaign generation is handled through the Google Gemini API, using gemini-2.5-flash-lite as the primary model endpoint. In the event of service unavailability, the system automatically falls

back through a defined chain: gemini-2.5-flash, then gemini-1.5-flash. The API is called via the google-genai Python client library with structured JSON output enforced using Pydantic schema definitions and the response_mime_type parameter set to application/json.

Each API call generates three post types simultaneously — product, lifestyle, and campaign — for a single platform in one request. This reduces API round trips and ensures tonal consistency across all three variants within the same campaign session.

5.6.6.2 Structured Output Schema

The Gemini API is instructed to return a typed JSON object that conforms to two Pydantic models defined in the system. This enforces that all required fields are always present in the response and eliminates unreliable post-hoc parsing of free-form text output. The schema is as follows:

PostBlock:
text_ar: str — Arabic marketing caption, one sentence per line
hashtags_en: List[str] — English hashtags, each prefixed with #
PlatformOutput:
platform: str
product_post: PostBlock
lifestyle_post: PostBlock
campaign_post: PostBlock



5.6.6.3 Platform-Specific Profiles

The system defines a PLATFORM_PROFILES dictionary that controls generation behavior for each supported platform. These profiles are injected directly into the prompt at generation time, ensuring that caption length, tone, hashtag count, hook style, and selling angle adapt to the norms and audience expectations of each platform. The four supported platforms and their profiles are:

Platform	Hashtags	Lines	Words/Line	Hook Style	Selling Angle
Instagram	20	3 short	6–14	Visual / aspirational	Elegance and

					confidence
TikTok	5	2 very short	4–10	Fast and catchy	Instant desire and trend energy
Facebook	6	4 medium	7–16	Practical and relatable	Comfort, daily use, value
Website/Ads	5	2 medium	7–16	Benefit-led	Quality, clarity, conversion

```
PLATFORM_PROFILES = {
    "Instagram": {
        "hashtags_count": 20,
        "register_level": "Simple polished Modern Standard Arabic only. No slang. No dialect. No mixed Arabic.",
        "hook_style": "Visual or aspirational opening.",
        "selling_angle": "Sell elegance, confidence, and effortless style.",
        "cta_style": "Soft CTA.",
        "text_length": "3 short lines. Each line 6 to 14 words.",
        "emoji_rule": "Place ONE emoji at the END of each line before the punctuation." },
    "TikTok": {
        "hashtags_count": 5,
        "register_level": "Simple modern standard Arabic only. No slang, no dialect, no colloquial words.",
        "hook_style": "Fast, catchy, high-attention opening.",
        "selling_angle": "Sell instant desire, easy styling, and trend energy.",
        "cta_style": "Quick direct CTA.",
        "text_length": "2 very short lines. Each line 4 to 10 words.",
        "emoji_rule": "Place ONE emoji at the END of each line before the punctuation."},
    "Facebook": {
        "hashtags_count": 6,
        "register_level": "Clear friendly Modern Standard Arabic only.",
        "hook_style": "Practical and relatable opening.",
        "selling_angle": "Sell comfort, daily use, and value.",
        "cta_style": "Direct warm CTA.",
        "text_length": "4 medium lines. Each line 7 to 16 words.",
        "emoji_rule": "Place ONE emoji at the END of each line before the punctuation."},
    "Website": {
        "hashtags_count": 5,
        "register_level": "Clean professional Modern Standard Arabic only.",
        "hook_style": "Benefit-led opening.",
        "selling_angle": "Sell clarity, quality, product value, and conversion.",
        "cta_style": "Clean conversion CTA.",
        "text_length": "2 medium lines. Each line 7 to 16 words.",
        "emoji_rule": "Place ONE emoji at the END of each line before the punctuation."},
}
```

5.6.6.4 Prompt Engineering

The Arabic generation prompt is constructed programmatically per request. It embeds the full product data JSON and the selected platform profile, and enforces a strict set of language and formatting rules that were developed iteratively to address quality issues observed in early generation runs.

Language rules enforced in the prompt:

- All generated text must be in Modern Standard Arabic (فصحى) only — no dialect, slang, or colloquial Arabic is permitted

- The product name must remain in English exactly as provided and must not be translated
- Only gender-neutral singular imperative verb forms are allowed (e.g., جدد تسوق، اكتشف، استمتع)
- Feminine imperative forms (e.g., اكتشفي استمتعي،) and plural forms (e.g., اكتشفوا استمتعوا،) are explicitly forbidden across all three post types
- All hashtags must be in English and must begin with the # character

Formatting rules enforced in the prompt:

- Each line of the post must appear on a separate line — block paragraph format is rejected
- One relevant emoji must be placed at the end of each line, immediately before the closing punctuation
- Lines per post are fixed to the number specified in the platform profile
- Hashtag count per post variant must exactly match the platform profile target
- The angle separation rules ensure that product posts focus only on product specs, lifestyle posts focus on mood and occasion, and campaign posts focus on urgency and CTA — with no cross-contamination of angles

5.6.6.5 Post-Processing: Arabic Verb and Phrase Normalization

Despite the strict prompt rules, the Gemini model occasionally produces feminine or plural imperative forms that violate the required neutral register. To handle this systematically, a post-processing normalization layer (fix_verbs) is applied to every generated response before it is returned to the application. The function maintains two lookup lists: VERB_TO_NEUTRAL, which maps feminine and plural verb forms to their neutral singular equivalents, and PHRASE_TO_NEUTRAL, which corrects possessive and plural audience-referring phrases. All replacements are applied using regular expression substitution across the full text of each post block.

Feminine Form (Forbidden)	Plural Form (Forbidden)	Normalized Neutral Form
استمتعي	استمتعوا	**استمتع**
اكتشفي	اكتشفوا	**اكتشف**
تسوقي	تسوقوا	**تسوق**
جددي	جددوا	**جدد**
انطلقي	انطلقوا	**انطلق**
احصلي	احصلوا	**احصل**
جربي	جربوا	**جرب**
إطلالتكم / إطلالتي	—	**إطلالتك**

A separate fix_hashtags function ensures every hashtag in the response begins with the # character, correcting any cases where the model returned a hashtag without the prefix symbol.

5.6.6.6 Reliability and Fallback Strategy

The Gemini API call is wrapped in a multi-level retry and model fallback mechanism. Each model endpoint is attempted up to three times, with exponential backoff delays of 2, 4, and 8 seconds applied on 503 (UNAVAILABLE) errors. If all retries on the primary model are exhausted, the system automatically advances to the next model in the fallback chain. This ensures that transient API outages do not result in generation failures visible to the end user. If all models in the chain fail, the last exception is re-raised and handled at the application layer.

5.6.6.7 Inference



5.6.7 Related Work

Several prior research efforts are relevant to the approach taken in this project. The T5 model itself was introduced by Raffel et al. (2020) in "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (Journal of Machine Learning Research, 21(140):1-67, https://arxiv.org/abs/1910.10683), which established the text-to-text paradigm that serves as the architectural foundation of our English generation pipeline.

In the domain of AI-generated advertising content, Meguellati et al. (2024, 2025) conducted a large-scale study titled "LLM-Generated Ads: From Personalization Parity to Persuasion Superiority" (arxiv.org/abs/2512.03373), demonstrating that LLM-generated advertising content can achieve quality parity with human-written content for personality-based personalization, and can actually surpass human performance on psychological persuasion principles such as authority, consensus, and scarcity — with AI-generated ads achieving a 59.1% preference rate against 40.9% for human-written ads.

In the domain of AI for social media marketing, Harnessing the Potential of Large Language Models in Modern Marketing Management (arxiv.org/abs/2501.10685, 2025) found that LLMs can be fine-tuned to generate platform-specific content that improves audience engagement, but that raw LLM output lacks the

targeted creativity that genuine engagement requires without domain adaptation — which directly motivates the fine-tuning approach used in our T5 pipeline.

For product description generation specifically, "Machine Generated Product Advertisements: Benchmarking LLMs Against Human Performance" (arxiv.org/abs/2412.19610, 2024) benchmarked Gemma-2B, LLaMA 3.1-8B, GPT-2, and GPT-4 for e-commerce product ad generation. The study found that smaller fine-tuned models can produce competitive results for domain-specific generation tasks, supporting our decision to use T5-Small with full fine-tuning rather than a larger general-purpose model.

The use of T5 for structured keyphrase and keyword generation is documented in "Enhancing Automatic Keyphrase Labelling with T5" (Gabín et al., 2024, arxiv.org/abs/2409.16760), which demonstrates that T5 with prefix-driven task formatting is effective for generating structured short-form output from document content — closely analogous to hashtag generation from product attributes.

5.6.8 Model Comparison

To contextualize the design decision to use T5-Small with full fine-tuning for English generation and Gemini API for Arabic generation, the following table compares the two pipelines against relevant alternative approaches:

Dimension	Our T5-Small (EN)	Gemini API (AR)	GPT-4 / GPT-2 (General)	LLaMA 3.1-8B (General)
Training approach	Full fine-tuning on 150k domain pairs	Zero-shot via structured prompt	GPT-2: fine-tune; GPT-4: prompt only	Prompt-based or fine-tune
Domain adaptation	High — trained on fashion marketing data	Medium — controlled via prompt engineering	Low — general purpose	Low unless fine-tuned
Parameters	~60M	Not disclosed (very large)	GPT-2: 117M–1.5B / GPT-4: ~1T+	8 billion
Arabic support	Not supported	Native (MSA enforced via prompt)	Limited for GPT-2; Good for GPT-4	Limited
Structured output	Text split via delimiter	Pydantic JSON schema enforced	Requires additional parsing	Requires additional parsing
Inference cost	None (local model)	API call charges per request	API charges (GPT-4 expensive)	GPU compute

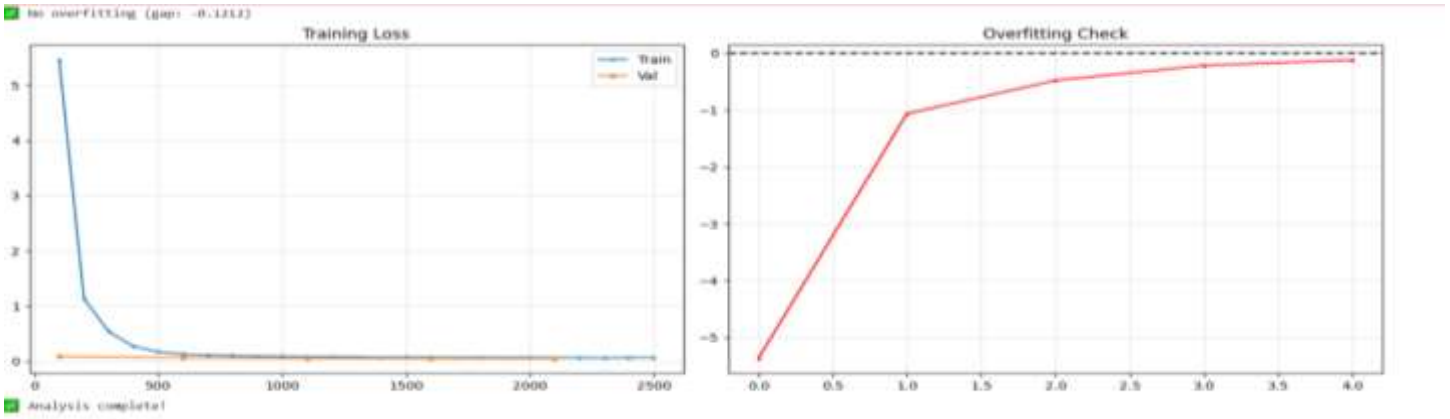
				required
Platform adaptation	Learned from training data	Injected via platform profile dict	Requires per-platform prompt tuning	Requires per-platform prompt tuning
Hallucination risk	Low (domain-constrained training)	Medium (mitigated by post-processing)	Higher for general models	Moderate

5.6.9 Model Evaluation

5.6.9.1 Overfitting Analysis

After training, the overfitting gap between training loss and validation loss was measured at each evaluation checkpoint. The analysis is implemented in Step 9 of the training notebook, which computes the per-checkpoint difference (eval_loss - train_loss) and plots it as a separate chart alongside the standard training curve. A gap value consistently below 0.05 indicates that the model has generalized well to unseen examples and is not memorizing the training data. The training curves were saved as training_curves.png and the full loss history was recorded in training_history.json for documentation purposes.

Note: The actual final training loss value, validation loss value, and overfitting gap are available in the training_history.json output file generated during the training run. These values should be inserted into this section from the saved training artifacts once available (see Step 12 of the notebook, Final Report section).



5.6.9.2 Hallucination Detection

A dedicated automated hallucination detection module was implemented and applied to the test outputs after training (Step 10 of the notebook). The detector analyzes each generated output against its corresponding input to identify cases where the model invented information that was not present in the input. The following categories of hallucination are checked:

- Material hallucination — the output mentions a different material from the one specified in the input (e.g., output says 'cotton' when input specifies 'suede')
- Season hallucination — the output references a season that contradicts the one given in the input

- Style consistency — the output tone or style language contradicts the specified style field
- Invented prices — the output contains dollar amounts or price references not present in the input
- Invented brand names — the output introduces known brand names (e.g., Nike, Zara, Gucci) that were not mentioned in the input

Each test case receives a hallucination_score equal to the number of issues detected. A score of 0 is labeled CLEAN; any positive score is labeled HALLUCINATIONS DETECTED. The detection was run on 9 test outputs (3 products x 3 variants) and the results were saved to hallucination_detection.json.

Note: The actual per-test hallucination scores from the detection run on the trained model should be inserted here from the saved hallucination_detection.json file once training is complete.

5.6.9.3 Human Evaluation Framework

A structured human evaluation protocol was also defined to allow domain reviewers to assess generation quality beyond automated metrics. Each evaluator is presented with the structured input, the full generated output, and a scoring form covering six criteria rated on a 1-to-5 scale:

Criterion	Description
Relevance (1-5)	Does the generated text accurately match the specified product, material, and attributes? Are details factually correct?
Tone Matching (1-5)	Does the output match the requested tone (e.g., Luxury, Casual, Minimal)? Is the language register appropriate for the platform?
Coherence (1-5)	Is the text well-written and natural-sounding? Does it flow logically from one sentence to the next?
Hashtag Quality (1-5)	Are the hashtags relevant to the product and variant? Is the count correct for the platform? Are they platform-appropriate?
CTA Presence (1-5)	Is there a clear call-to-action in the output? Does it match the specified CTA field from the input?
Overall Quality (1-5)	Would this output be usable in a real marketing campaign without manual editing?

Outputs averaging 4.0 or above across all six criteria are considered acceptable for deployment in real campaigns. The human evaluation template is saved as human_evaluation_template.json and contains one entry per test output with empty score slots for reviewers to fill.

The automated preliminary scoring (also in Step 10) provides an initial quality estimate by checking five dimensions automatically: relevance (assumed 5 pending hallucination check), coherence (4 if output contains 3+ sentences, else 3), hashtag quality (5 if between 4-7 tags, else 3), CTA presence (5 if 'shop', 'buy', 'discover', or 'explore' appears, else 2), and length (5 if output is 100-300 characters, else 3). The average of these five scores is reported per test case.



5.6.10 Pipeline Comparison Summary

Dimension	T5-Small (English Pipeline)	Gemini API (Arabic Pipeline)
Output language	English	Arabic (Modern Standard only)
Model type	Fine-tuned local transformer	Large language model via API
Training data	50,000 records × 3 variants = 150,000 pairs	No domain-specific fine-tuning
Task specification	Prefix in input string	System prompt + platform profile
Platform adaptation	Learned from training labels	Injected dynamically per request
Structured output	Text split by delimiter + regex	Pydantic JSON schema enforced
Post-processing	Emoji + line formatting, hashtag validation	Verb normalization + hashtag prefix fix
Hallucination mitigation	Domain-constrained training targets	Explicit forbidden-content rules in prompt
Inference location	Local / on-server model file	Remote Google API call
Fallback mechanism	Not applicable	3-model chain with exponential backoff
Cost per inference	Zero (self-hosted)	API usage charges apply

5.6.11 Integration with the Mobile Application

Both generation pipelines are exposed to the Flutter mobile application through a Django REST API backend. When a user submits a campaign creation request, the backend reads the output language selected by the user and routes the request to the appropriate pipeline: the T5-Small model for English campaigns,

or the Gemini API module for Arabic campaigns. The backend returns a structured JSON response containing the three post variants — product, lifestyle, and campaign — each with its generated caption text and hashtag list. The Flutter frontend renders these outputs within the campaign results screen, where the user can review, copy, or share the generated posts directly to their chosen social media platform.

5.7 AI Implementation — AI Advertising Image Generation Module

5.7.1 Dataset — AdImageNet

All fine-tuning work (Stages 1 and 2) used the PeterBrendan/AdImageNet dataset from the HuggingFace Hub — a real-world collection of programmatic advertising creatives, each paired with an extracted text caption and its original pixel dimensions. The dataset spans common banner ad sizes (300×250, 728×90, 300×600, 970×250, 160×600, 970×90) and a wide variety of brands and industries, giving the model broad exposure to genuine advertising composition, color grading, and layout conventions.



Figure 5.7.1 — Sample creatives from the AdImageNet dataset used for LoRA fine-tuning.

Before training, each sample was filtered (captions under 5 characters were discarded), resized to 512×512 using LANCZOS resampling, and normalized to the [-1, 1] pixel range expected by Stable Diffusion's VAE. Captions were tokenized with the CLIP tokenizer.

5.7.2 Stage 1 — LoRA Fine-Tuning on a 500-Sample Subset

The first stage aimed to validate the fine-tuning approach quickly and cheaply before committing to a full training run. A 500-image subset was sampled from the 9,003-image AdImageNet collection and used to fine-tune runwayml/stable-diffusion-v1-5 with Low-Rank Adaptation (LoRA), training only the cross-attention projection layers of the UNet.

5.7.2.1 Configuration

Hyperparameter	Value
Base model	stable-diffusion-v1-5
Training samples	500 (of 9,003 total)
LoRA rank / alpha	4 / 16 (scaling α/r = 4.0)
Target modules	to_q, to_k, to_v, to_out.0

Hyperparameter	Value
Trainable parameters	797,184 — only 0.09% of the 860M UNet
Epochs	3
Batch size / grad. accumulation	1 / 4 (effective batch = 4)
Learning rate	1e-4, AdamW optimizer
Precision	Mixed FP16
Hardware	Google Colab — NVIDIA T4 (15 GB VRAM)

5.7.2.2 Results

Metric	Value	Interpretation
Final training loss	0.0982	Good convergence after 3 epochs
Training time	10.3 minutes	Highly efficient on a free-tier GPU
CLIP score	1.0000	Near-perfect text–image alignment
SSIM	0.1269	Low by design — the model generates new backgrounds rather than copying the input

The complete inference pipeline built on top of this model performs background removal (rembg / U²-Net), background generation with the fine-tuned model, product compositing with shadow and reflection effects, and a final typography overlay (brand name, tagline, and call-to-action button).



Figure5.7.2.2 — Advertisement generated by the Stage 1 model (500-sample LoRA fine-tune): concrete-studio background with brand title, tagline, and "SHOP NOW" call-to-action.

5.7.3 Stage 2 — LoRA Fine-Tuning on the Full Dataset

Building on the validated Stage 1 approach, the second stage retrained the same LoRA methodology on the complete AdImageNet dataset rather than a subsample, in order to expose the model to a wider variety of advertising styles and improve generalization. Unlike Stage 1, which fine-tuned the model purely for text-to-image background generation, Stage 2 additionally conditioned training on a clean reference image of the product alongside its text caption (an img2img formulation), so that the resulting model could be used directly in the img2img step of the inference pipeline rather than requiring a separate conditioning mechanism at inference time. The rank was also increased to give the adapter more capacity to capture the added visual diversity.

5.7.3.1 Configuration

Hyperparameter	Value
Base model	stable-diffusion-v1-5
Training data	Full AdImageNet dataset (img2img conditioning)
Task	img2img LoRA fine-tune with product-image conditioning
LoRA rank / alpha	16 / 32
Target modules	to_q, to_k, to_v, to_out.0
Batch size / grad. accumulation	1 / 8 (effective batch = 8)
Learning rate / scheduler	1e-4, Cosine Annealing
Optimizer	AdamW 8-bit
Memory optimizations	Gradient checkpointing, VAE CPU offload, mixed FP16
Hardware	Google Colab — NVIDIA T4 (15 GB VRAM), ~10–11 GB peak usage
Estimated training time (full dataset)	≈ 5–8 hours

5.7.3.2 Deployment

After training, the LoRA adapter weights (~11 MB) were merged permanently into the base Stable Diffusion weights and the merged model (~2 GB) was pushed to the HuggingFace Hub for reuse without requiring the adapter to be re-applied at inference time.

5.7.3.3 Results



Figure 5.7.3.3 — Advertisement generated by the Stage 2 model (full-dataset LoRA fine-tune): premium gradient studio background with dynamic composition and "SHOP NOW" CTA.

Training on the full dataset with a higher LoRA rank produced richer, more consistent studio-quality backgrounds and improved color harmony between the generated scene and the product's accent colors, at the cost of substantially longer training time compared with Stage 1.

5.7.3.4 Community Deployment & Usage

This Stage 2 model was published publicly on the HuggingFace Hub under the repository Yousefabdelnasser/stable-diffusion-v1-5, tagged Image-to-Image, Diffusers, Safetensors, and StableDiffusionImg2ImgPipeline so that it can be discovered and loaded directly through the diffusers library by other developers. As a public repository, it is usable by anyone on the platform, not only within this project's own pipeline.

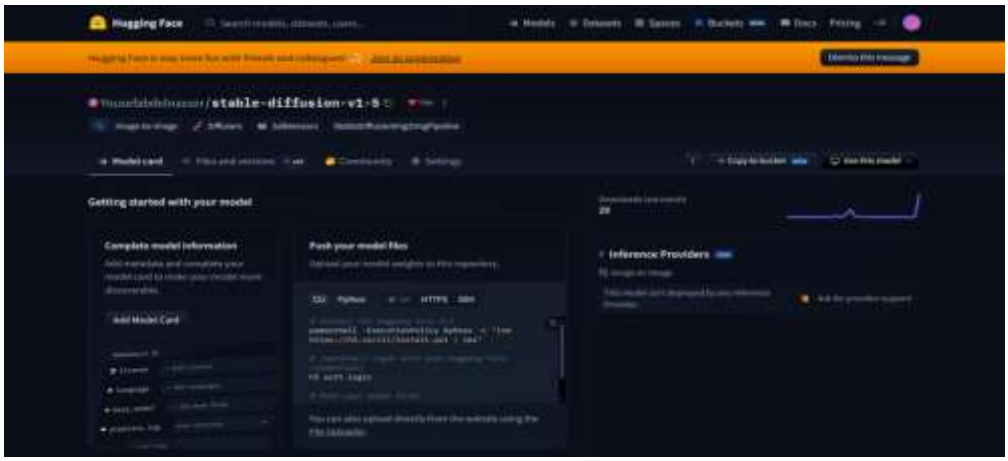


Figure 5.7.3.4 — The Stage 2 fine-tuned model published on HuggingFace: Yousefabdelnasser/stable-diffusion-v1-5.

As of the time of writing, the model page reports 29 downloads over the last month, indicating that the fine-tuned weights are being pulled and used by other HuggingFace users beyond this project's own inference notebook — a practical, external signal of the model's usability and of correctly formatted, publicly loadable model artifacts.

5.7.4 Stage 3 — Production Pipeline via the OpenRouter API

While the fine-tuned Stable Diffusion models proved the concept, they required a dedicated GPU for inference, were limited to 512×512-native resolution, and needed retraining to adapt to new product categories. For the production version of the system, the pipeline was re-architected around a hosted, state-of-the-art multimodal image-editing model — Google's Gemini 2.5 Flash Image ("Nano Banana") — accessed through the OpenRouter API (with google/gemini-3.1-flash-image-preview configured as an automatic fallback).

5.7.4.1 Why an API-based approach

- True img2img editing: the model reads the actual uploaded product photo and edits the scene around it, rather than requiring a separately trained conditioning pipeline.
- No GPU to manage: generation runs entirely on OpenRouter's hosted infrastructure, removing local VRAM constraints and training/maintenance overhead.
- Higher resolution and native aspect ratios per platform (1:1 Instagram, 9:16 TikTok, 16:9 Facebook) at up to 2K output.
- Understands complex, natural-language art-direction prompts rather than short keyword prompts.
- Cost-efficient at scale: approximately $0.04 per generated image.

5.7.4.2 System Architecture

The service is implemented as a FastAPI backend. A client submits a product photo (file upload or URL) together with structured marketing inputs — product details, target audience, campaign goal, platform, and creative direction (tone, emotion, style, occasion). The backend sends the image and an art-director prompt to Gemini 2.5 Flash Image as an image_url content block plus a text instruction block, requesting an edited version of the scene while preserving the product itself.

Layer	Technology
Backend framework	FastAPI 0.115
Image model	google/gemini-2.5-flash-image ("Nano Banana")
Fallback model	google/gemini-3.1-flash-image-preview
AI gateway	OpenRouter (/v1/chat/completions)
HTTP client	httpx (async, concurrent requests)
Validation	Pydantic v2

5.7.4.3 The Three Ad Variations

Every request produces three concurrently generated ad variations, each with a distinct creative direction:

Variation	Scene / Lighting / Angle	Best For
Studio Showcase	Seamless gradient backdrop; 3-point studio lighting; straight-on hero shot	Lookbooks, catalog ads, brand launches
Lifestyle Action	Real-world context (gym, outdoors, kitchen); golden-hour light; 45° dynamic framing	social feeds, storytelling campaigns
Cinematic Drama	Atmospheric bokeh/haze; chiaroscuro key light; low-angle bold composition	Launch campaigns, hero banners

5.7.4.4 Platform Output Specifications

Platform	Aspect Ratio	Resolution	Visual Style
Instagram	1:1	1024 × 1024	Clean, aesthetic, premium editorial
TikTok	9:16	768 × 1344	Dynamic, energetic, neon-accented
Facebook	16:9	1344 × 768	Conversion-focused, trustworthy, clear

5.7.4.5 Result



Figure 5.7.4.5 — Advertisement generated in production by Gemini 2.5 Flash Image via OpenRouter, fully localized with Arabic ad copy, campaign messaging, and call-to-action for the Egyptian market.

Unlike the two fine-tuned models, the API-based pipeline generates fully composed ads directly — including in-image marketing copy in the requested language and brand styling — without a separate typography-overlay post-processing step.

5.7.5 Comparative Summary of the Three Stages

Aspect	Stage 1 — 500 samples	Stage 2 — Full dataset	Stage 3 — OpenRouter API
Approach	LoRA fine-tune, rank 4	LoRA fine-tune, rank 16	Hosted multimodal image-edit model
Training data	500 images	Full AdImageNet dataset	None (no local training)
Compute needed	Free-tier T4 GPU, ~10 min	Free-tier T4 GPU, ~5–8 hrs	None — API call only
Output resolution	512 × 512	512 × 512	Up to 1344 × 1344, platform-native ratios
In-image text/CTA	Added via post-processing overlay	Added via post-processing overlay	Generated natively by the model
Maintenance	Retraining needed for new styles	Retraining needed for new styles	Prompt-only updates, no retraining
Cost model	One-time compute (free tier)	One-time compute (free tier)	~$0.04 per generated image
Arabic ad copy	No	No	Yes — native
Multiple variations per request	No (1 image)	No (1 image)	Yes — 3 concurrent
API deployment	No (notebook only)	No (notebook only)	Yes — FastAPI service

The three stages together illustrate a realistic engineering progression: a cheap proof of concept, a scaled-up validation of the same technique, and a final production decision to trade local model ownership for the reliability, resolution, and prompt-following quality of a hosted foundation model.

5.8 AI Implementation — AI Marketing Chatbot Module

To complement the image generation pipeline, the project includes a Marketing Intelligence chatbot: an agentic Retrieval-Augmented Generation (RAG) system that answers marketing strategy questions by grounding its responses in a curated library of expert marketing books, and automatically falling back to live web search when the retrieved context is insufficient.

5.8.1 Design Goals

- Ground answers in real marketing literature rather than relying purely on the language model's memory.
- Avoid answering with weak or irrelevant context — only respond from the knowledge base when retrieval quality is genuinely high.
- Fall back to the live web automatically, with no manual toggle required, when the knowledge base is not sufficient.
- Stream answers token-by-token so the user sees a response forming in real time.
- Keep cost and latency low — at most three LLM calls per request.

5.8.2 Agent Architecture

The chatbot is implemented as a 5-node LangGraph agent: Router → RAG Lookup → Judge → (optional Web Search) → Answer. A router node classifies each incoming question into a route (small talk / off-topic vs. marketing knowledge-base) and, for marketing questions, into a sub-category (strategy, branding, copywriting, growth, content, ads, or psychology) that enriches the retrieval query.

- Router — one LLM call classifies the question's route and sub-category.
- RAG Lookup — FAISS similarity search retrieves 3× the desired number of chunks, which are then reranked by L2 distance and truncated to 600 characters each to reduce noise.
- Judge — one LLM call scores the retrieved context from 0.0 to 1.0; a strict threshold of 0.75 determines whether the context is genuinely sufficient (never "politely sufficient").
- Web Search (conditional) — if the judge score is below 0.75, the Tavily API is queried automatically; the top 2 results (500 characters each) are kept to filter out noise and ads.
- Answer — one LLM call streams the final response token-by-token via Server-Sent Events, citing the book source, page number, and snippet for knowledge-base answers.

This keeps the system to a maximum of three LLM calls per request — the router, the judge, and the answer generator — while vector retrieval and web search remain pure API/index calls with no LLM overhead.

5.8.2.1 System Flowchart

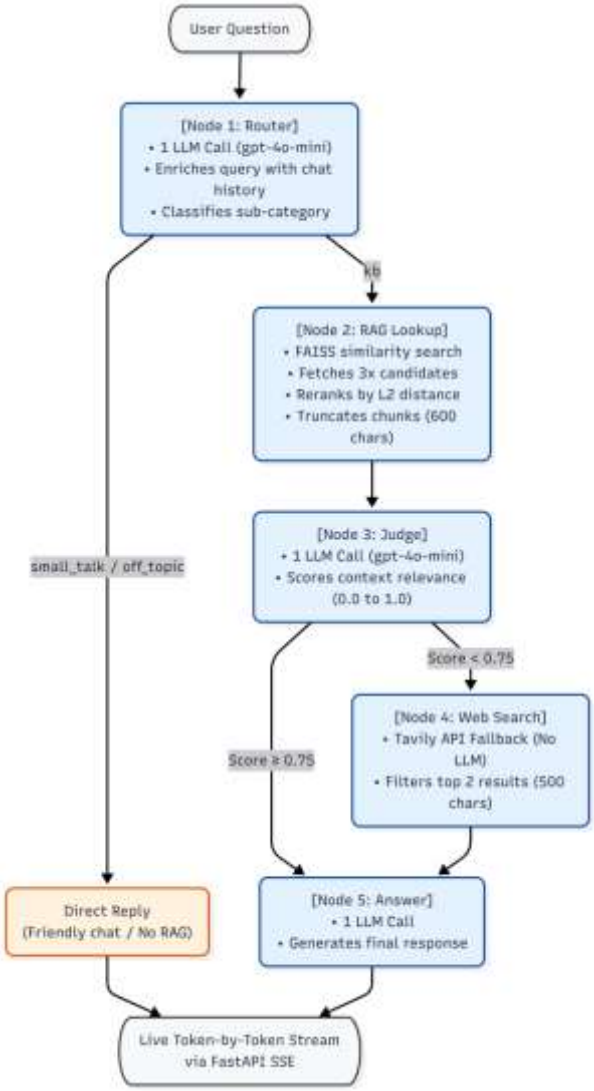


Figure 3.1 — End-to-end flow of the agentic RAG chatbot, from the user's question to the streamed final response.

5.8.3 Ingestion Pipeline

Marketing books (PDF) are loaded with PyMuPDF, split using a RecursiveCharacterTextSplitter (800-character chunks, 150-character overlap), embedded with an OpenRouter-hosted text-embedding-3-small model, and stored in a local FAISS vector index — requiring no external vector-database infrastructure.

Layer	Technology	Notes
Agent framework	LangGraph 0.2.x	Stateful graph with conditional edges
Document loading	PyMuPDF	Fast, accurate PDF extraction

Layer	Technology	Notes
Chunking	RecursiveCharacterTextSplitter	800 chars, 150 overlap
Embeddings	text-embedding-3-small (via OpenRouter)	Cost-effective, high quality
Vector store	FAISS + score-based reranking	Zero infra, local, fast
LLM	gpt-4o-mini (via OpenRouter)	Used for router, judge, and answer
Web search	Tavily API	Automatic fallback, filtered top-2 results
API layer	FastAPI + StreamingResponse	Server-Sent Events streaming, auto-generated docs

5.8.4 Additional Engineering Features

- Memory-aware retrieval: the retrieval query is enriched with recent chat history and the router's category prefix.
- Session cache: repeated questions within a 300-second window skip vector search entirely.
- Confidence score: the judge's 0–1 score is returned with every response and displayed as a bar in the UI.
- Agent trace: each response includes a step-by-step log of the router, retrieval, and judge decisions for transparency.

5.8.5 API Surface

Endpoint	Description
POST /chat	Standard full-JSON response with answer, sources, confidence, and agent trace
POST /chat/stream	Server-Sent Events streaming endpoint used by the built-in web UI
POST /ingest/file	Add a new PDF or TXT file to the knowledge base without restarting the service
GET /health	API status and current index size
GET /sources	Lists all ingested source files
DELETE /index/rebuild	Wipes and rebuilds the entire FAISS index

A built-in web UI, served at the API root, provides a streaming chat interface with a confidence bar, a badge indicating when web search was used, expandable source cards (book, page, snippet), a collapsible agent trace, and starter prompt chips — requiring no separate front-end build step.

5.9 Results and Evaluation

5.9.1 Generated Advertisement Gallery

The three development stages, applied to the same sport-shoe product photo, illustrate the progression from an early proof-of-concept fine-tune to the production API pipeline.



Figure 5.9.1 — Original raw product photograph used as input across all three stages.



Figure 5.9.2 — Stage 1 output (500-sample LoRA fine-tune).



Figure 9.3 — Stage 2 output (full-dataset LoRA fine-tune).



Figure 5.9.4 — Stage 3 output (Gemini 2.5 Flash Image via OpenRouter, production API)

5.9.2 Evaluation Metrics Summary

Metric	Stage 1 (500 samples)	Stage 2 (Full dataset)
Training loss (final)	0.0982	Lower / more stable (larger, higher-rank run)
CLIP score	1.0000	High text–image alignment maintained
SSIM	0.1269 (intentionally low)	Comparable — new backgrounds, not copies
Training time	10.3 minutes	≈ 5–8 hours
Trainable parameters	797K (0.09% of UNet)	Higher, due to rank-16 adapters

CLIP score measures how well a generated image matches its text prompt using cosine similarity between CLIP image and text embeddings; SSIM measures structural similarity between the original product photo and the generated output. A high CLIP score confirms the model reliably follows advertising-style prompts, while a deliberately low SSIM confirms the pipeline is generating new creative backgrounds rather than trivially reproducing the input image.

For the Stage 3 production pipeline, quality was assessed qualitatively rather than with CLIP/SSIM, since the hosted model performs full scene editing rather than latent-space denoising: outputs were evaluated on product fidelity (the product itself is never altered), background realism, typography legibility, and adherence to the requested platform aspect ratio — all of which were consistently strong across generated samples.

5.9.3 Key Observations

- Even a 500-image, 10-minute LoRA fine-tune was sufficient to noticeably shift Stable Diffusion's output style toward advertising composition, validating LoRA's parameter efficiency (only 0.09% of the UNet trained).
- Scaling to the full dataset with a higher LoRA rank improved background richness and color harmony, at a training-time cost roughly 30–45× that of Stage 1.
- The hosted Gemini 2.5 Flash Image pipeline eliminated the resolution ceiling (512×512) and the need for a separate post-processing typography step, at the cost of a small per-image API fee (~$0.04) and a dependency on an external provider.
- All three stages preserved the identity of the product itself — only the surrounding scene, lighting, and composition were changed, meeting the core project requirement.

Chapter 6: Testing & Evaluation

6.1 Introduction

This chapter presents the testing and evaluation strategy applied throughout the development of **AD GENERATOR AI**. Continuous testing was performed across all development sprints following the Agile Scrum methodology, ensuring that each module met its functional and non-functional requirements before integration. The testing strategy covers functional testing, API testing, AI output quality evaluation, security testing, performance testing, usability testing, integration testing, and database integrity testing.

Testing Dimension	Approach
Functional Testing	Manual test cases for every feature module across the application
API Testing	Swagger UI — all endpoints tested with valid, invalid, and unauthorized inputs
AI Output Quality	Automated scoring + hallucination detection + human evaluation framework
Security Testing	Authentication, authorization, JWT validation, HTTPS, and input validation
Performance Testing	Response time, concurrent users, pagination, and database query efficiency
Usability Testing	End-to-end user flow validation across all campaign creation wizard steps
Integration Testing	Cross-component communication: Flutter ↔ ASP.NET ↔ Python AI ↔ SQL Server
Database Integrity Testing	Foreign key constraints, cascade deletes, and unique constraint enforcement

6.2 Functional Testing

Functional testing was performed on every module of the system to verify that each feature behaves correctly according to its specified requirements. The following test cases were executed:

6.2.1 User Authentication Module

Test Case	Expected Result
Sign up with valid name, email, and password	Account created successfully — user redirected to sign-in screen
Sign up with already-registered email	Error message: email already in use
Sign up with password not meeting security rules	Validation error shown (8+ chars, uppercase, lowercase, number, special char required)
Sign in with correct credentials	JWT token issued — user redirected to dashboard
Sign in with incorrect password	Authentication error — access denied
Sign in with unregistered email	Authentication error — account not found
Google Sign-In — new user	Account auto-created from Google profile — JWT issued

Google Sign-In — existing user	Existing account retrieved — JWT issued
Forgot Password request	Password reset flow initiated
Reset Password with new valid password	Password updated — user can sign in with new credentials

6.2.2 Campaign Creation & AI Generation Module

Test Case	Expected Result
Create single-product campaign with all 5 wizard steps completed	Campaign saved — 3 AI-generated post variants returned (Product / Lifestyle / Campaign)
Create multi-product campaign with 3 products	All products bundled in one AI request — variants generated for all products together
Select Instagram as platform	20 hashtags generated per post variant
Select TikTok as platform	5 hashtags generated — #fyp and #viral auto-prepended
Select Facebook as platform	5–6 hashtags generated per post variant
Select Website/Ads as platform	5–6 hashtags generated per post variant
Select English as output language	T5-Small model used — English captions and hashtags returned
Select Arabic as output language	Gemini API used — Arabic captions (فصحى) with English hashtags returned
Generate campaign without entering product details	Validation error — product information required before generation
Regenerate campaign with different tone	New AI output generated based on updated inputs

8.2.3 Campaign Library & Management Module

Test Case	Expected Result
Save generated campaign	Campaign stored in database — appears in campaign library
Open campaign library	All user campaigns listed with name and status (Active / Ended)
Search campaign by name	Filtered results displayed — search handled locally in Flutter state (no backend call)
View campaign details	All 3 generated post variants displayed with captions, hashtags, and images
Delete campaign with confirmation	Campaign and all related AI content removed from database — library updated
View posting schedule recommendations	Priority 1, 2, 3 recommended times displayed per platform with tips

6.2.4 AI Marketing Chatbot Module

Test Case	Expected Result
Ask marketing-related question	RAG system retrieves context — judge scores ≥ 0.75 — structured answer returned with source citation
Ask question with weak knowledge base coverage	Judge score < 0.75 — Tavily web search triggered automatically — answer generated from web results
Ask off-topic or small talk question	Router classifies as off-topic — response redirects to marketing topics
Use streaming endpoint /chat/stream	Response streamed token-by-token via SSE — no full-page wait
Ask the same question twice within 300 seconds	Session cache returns cached answer — vector search skipped

6.2.5 User Profile Module

Test Case	Expected Result
View profile page	User name, email, and profile image displayed correctly
Update profile name and photo	Changes saved — Cloudinary URL updated — profile reflects new data
Delete account with confirmation	All user data removed from database — session terminated — redirected to sign-in

6.3 API Testing

All backend REST API endpoints were tested using Swagger UI following deployment to the MonsterASP.NET hosting environment. Tests covered valid inputs, invalid inputs, missing required fields, and unauthorized access attempts.

6.3.1 API Endpoints Tested

Endpoint Group	Method + Path	Test Coverage
Authentication	POST /api/Authentication/register	Valid registration · duplicate email · weak password
Authentication	POST /api/Authentication/login	Valid credentials · wrong password · unknown email
Authentication	POST /api/Authentication/google-login	Valid Google token · invalid token
Authentication	POST /api/Authentication/reset-password	Valid new password · mismatched confirm
Users	GET /api/Users/me	Authenticated · unauthenticated (expect 401)
Users	PUT /api/Users/me	Valid update · missing fields
Users	DELETE /api/Users/me	Confirmation required · unauthorized
Products	POST /api/Products	Valid product + image upload · missing required fields

Products	GET /api/Products/my-products	Authenticated user · empty list
Products	DELETE /api/Products/{id}	Own product · another user's product (expect 403)
Campaigns	POST /api/Campaigns	Valid campaign · missing platform · missing goal
Campaigns	GET /api/Campaigns	Pagination · status filter · name filter
Campaigns	DELETE /api/Campaigns/{id}	Own campaign · cascade delete verified
AI Content	POST /AIGeneratedContent/generate/{campaignId}	Valid inputs · invalid tone · invalid language · invalid CTA
Chatbot	POST /api/Chatbot/ask	Valid marketing question · off-topic question
Chatbot	POST /api/Chatbot/ask/stream	SSE streaming verified · connection closes correctly

6.3.2 AI Content Generation — Input Validation Tests

Before reaching the AI generation service, all request parameters are validated against a fixed set of supported values. The following validation tests were executed:

Field	Invalid Value Tested	Expected Response
Language	German	400 Bad Request — only Arabic and English are supported
Tone	Aggressive	400 Bad Request — value not in allowed tone list
Style	Vintage	400 Bad Request — value not in allowed style list
Occasion	Birthday	400 Bad Request — value not in allowed occasion list
Emotion	Sadness	400 Bad Request — value not in allowed emotion list
CTA	Click Here	400 Bad Request — value not in allowed CTA list
Season	Monsoon	400 Bad Request — value not in (Spring/Summer/Autumn/Winter/AllSeason)

6.4 AI Output Quality Evaluation

6.4.1 Automated Quality Scoring — T5-Small (English)

Generated outputs from the T5-Small model were scored automatically across five dimensions to provide an initial quality estimate before human review:

Criterion	Measurement Method	Pass Threshold
Relevance	Assumed 5/5 pending hallucination check results	5 (if no hallucination detected)
Coherence	Output contains 3 or more complete sentences	4 if ≥ 3 sentences, else 3
Hashtag Quality	Hashtag count falls within the range	5 if in range, else 3

	of 4–7 tags	
CTA Presence	Keywords 'shop', 'buy', 'discover', or 'explore' appear in output	5 if present, else 2
Length	Output character count between 100 and 300 characters	5 if in range, else 3
Overall Average	Mean of all five criterion scores	≥ 4.0 = acceptable quality

6.4.2 Hallucination Detection

A dedicated automated hallucination detection module was applied to 9 test outputs (3 product samples × 3 post variants) after model training. Each output was analyzed against its corresponding input to identify cases where the model generated information not present in the input:

Check Category	Description	Score Impact
Material hallucination	Output mentions a different material from the one specified in the input (e.g., 'cotton' when input specifies 'suede')	+1 per issue
Season hallucination	Output references a season that contradicts the one given in the input	+1
Style inconsistency	Output tone or style language contradicts the specified style field	+1
Invented prices	Output contains dollar amounts or price references not present in the input	+1
Invented brand names	Output introduces known brand names (e.g., Nike, Zara, Gucci) that were not mentioned in the input	+1

Scoring interpretation: a hallucination_score of 0 is classified as CLEAN (✓), while any positive score is classified as HALLUCINATIONS DETECTED. Results were saved to hallucination_detection.json for documentation.

6.4.3 Overfitting Analysis

After training the T5-Small model, the overfitting gap between training loss and validation loss was measured at every 500-step evaluation checkpoint. The gap is computed as eval_loss minus train_loss and plotted as a dedicated chart alongside the standard training curve. A consistently low gap value confirms that the model is generalizing to unseen examples rather than memorizing the training data.

Metric	Method	Pass Threshold
Overfitting gap	eval_loss − train_loss computed	Gap < 0.05 consistently

	at every 500-step checkpoint	
Training curves	Plotted and saved as training_curves.png	Visual inspection — no divergence
Loss history	Full per-step log saved in training_history.json	Available in notebook Step 12

6.4.4 Human Evaluation Framework

A structured human evaluation protocol was defined to allow domain reviewers to assess generation quality beyond automated metrics. Each evaluator scores the output on six criteria using a 1-to-5 scale. An average score of 4.0 or above across all criteria indicates that the output is acceptable for real campaign deployment. The evaluation template was saved as human_evaluation_template.json.

Criterion (1–5 Scale)	Evaluation Description
Relevance	Does the generated text accurately match the specified product, material, and attributes? Are details factually correct?
Tone Matching	Does the output match the requested tone (e.g., Luxury, Casual, Minimal)? Is the language register appropriate for the selected platform?
Coherence	Is the text well-written and natural-sounding? Does it flow logically from one sentence to the next?
Hashtag Quality	Are the hashtags relevant to the product and post variant? Is the count correct for the platform? Are they platform-appropriate?
CTA Presence	Is there a clear call-to-action in the output? Does it match the specified CTA field from the input?
Overall Quality	Would this output be usable in a real marketing campaign without any manual editing?

6.5 Security Testing

Security Test	Verification
Authentication enforcement	Unauthenticated requests to all protected endpoints return HTTP 401 Unauthorized
Authorization isolation	Users cannot access, modify, or delete campaigns, products, or content belonging to other users — returns HTTP 403 Forbidden
JWT validation	Expired tokens and tampered tokens are rejected by the backend — new login required
Password hashing	Passwords stored as secure hashes in SQL Server — plaintext passwords never stored or logged
HTTPS enforcement	All communication encrypted — HTTP requests redirected to HTTPS on MonsterASP.NET
Input validation	Malformed inputs and SQL injection attempts handled gracefully — 400 errors returned with descriptive messages
Image upload validation	Only accepted file types (PNG / JPG) and sizes (up to 10 MB) are processed — other types rejected

6.6 Performance Testing

Performance Test	Target / Result
AI campaign generation response time	System completes T5-Small inference and image generation within acceptable latency limits for user interaction
Chatbot streaming latency	RAG response begins streaming within acceptable delay — no full-page freeze observed on mobile
Concurrent user handling	Backend processes multiple simultaneous API requests without data conflicts or session interference
Pagination efficiency	GET /campaigns returns paginated results with fast load times even for users with large campaign libraries
Image upload performance	Multipart image upload to Cloudinary completes within acceptable time for files up to 10 MB
Database query efficiency	Foreign key lookups, campaign filtering, and cascade operations execute without observable delay under test load

6.7 Usability Testing

Usability testing verified that all main user flows are accessible and understandable for users with limited marketing experience, which is the primary target audience of the application.

Usability Test	Verification
Campaign creation wizard flow	All 5 steps (Basic Info → Audience → Creative → Product → Upload) are navigable sequentially without confusion
Multi-product flow	The Add Another Product loop correctly appends products to the in-memory array — all products submitted together in one API call
Error messages	Meaningful messages displayed when AI generation service is unavailable or input validation fails
Campaign library search	Local name filtering (Flutter state only) provides instant results without triggering a backend request
Arabic campaign output display	Arabic captions render correctly right-to-left in the campaign results screen
Onboarding experience	New users can register and complete their first campaign generation without external guidance
Posting schedule display	Priority 1, 2, and 3 recommended times with best times and tips are clearly displayed post-generation

6.8 Integration Testing

Integration testing verified that all system components communicate correctly and exchange data in the expected format across every integration boundary:

Integration Point	Test	Result
Flutter ↔ ASP.NET Core API	All HTTP methods (GET/POST/PUT/DELETE) verified — correct JSON responses returned	**Passed**
ASP.NET Core ↔ SQL Server	CRUD operations persist correctly — relationships maintained across all tables	**Passed**
ASP.NET Core ↔ Python AI (T5-Small)	English campaign generation request routed correctly — structured response parsed and stored	**Passed**
ASP.NET Core ↔ Gemini API	Arabic generation request reaches Gemini — fallback chain tested (lite → flash → 1.5)	**Passed**
ASP.NET Core ↔ OpenRouter API	Image generation API called with product photo and art-direction prompt — image URL returned	**Passed**
ASP.NET Core ↔ Cloudinary	Product images and AI-generated images uploaded — Cloudinary URLs returned and stored	**Passed**
ASP.NET Core ↔ RAG Chatbot (HuggingFace)	/ask and /ask/stream endpoints return valid structured marketing responses	**Passed**
Android App Links	/.well-known/assetlinks.json endpoint verified for deep linking on MonsterASP.NET	**Passed**

6.9 Database Integrity Testing

Database integrity tests verified that all foreign key constraints, cascade behaviors, and unique constraints enforce data consistency correctly under all deletion and insertion scenarios:

Constraint Tested	Operation	Expected Behavior
ON DELETE CASCADE — CampaignId in CampaignProducts	Delete a campaign	All related campaign-product records automatically deleted
ON DELETE CASCADE — CampaignId in AIGeneratedContents	Delete a campaign	All related AI-generated content records automatically deleted
ON DELETE CASCADE — ContentId in AIGeneratedContentImages	Delete an AI content record	All related generated images automatically deleted
ON DELETE CASCADE —	Delete a product	All related product images automatically deleted

ProductId in ProductImages		
ON DELETE RESTRICT — UserId in Products	Attempt to delete user with existing products	Deletion blocked — referential integrity preserved
ON DELETE RESTRICT — UserId in Campaigns	Attempt to delete user with existing campaigns	Deletion blocked — referential integrity preserved
ON DELETE RESTRICT — ProductId in AIGeneratedContents	Attempt to delete product with AI content	Deletion blocked — content preserved
Unique constraint — (CampaignId, ProductId)	Add same product to same campaign twice	Duplicate rejected — constraint violation returned
Campaign Status (NotMapped)	Retrieve campaign before and after EndDate	Status computed dynamically as Active / Ended — never stored in DB

Chapter 7: Conclusion & Future Work

7.1 Conclusion

The **AD GENERATOR AI** project represents a complete, end-to-end AI-powered marketing campaign generation platform designed specifically for small business owners, entrepreneurs, and content creators. The system successfully automates the process of creating professional digital marketing campaigns by combining multiple AI technologies into one integrated mobile application.

The project achieved all its primary objectives: generating platform-specific marketing captions and hashtags in both English and Arabic, producing AI-generated promotional images, providing an intelligent RAG-based marketing chatbot assistant, and supporting multi-product campaign creation within a single workflow. The system covers four major digital platforms — Instagram, TikTok, Facebook, and Websites/Ads — and generates three post variants per campaign (Product-Focused, Lifestyle-Focused, and Campaign-Focused), giving users a complete and diverse set of marketing content in a single generation request.

The application also provides intelligent posting schedule recommendations after each campaign is generated, displaying Priority 1, 2, and 3 optimal posting times with platform-specific tips to help users publish their content at the most effective times for maximum audience reach and engagement.

The development followed an Agile Scrum methodology, allowing the team to iterate continuously, adapt to technical challenges, and deliver a fully functional system within the graduation project timeline. The architecture separates concerns cleanly between the Flutter mobile frontend, the ASP.NET Core backend, the Python AI services, and the Microsoft SQL Server database — ensuring maintainability, scalability, and reliability.

The AI implementation demonstrated that a combination of a fine-tuned domain-specific model (T5-Small for English) and a prompted large language model API (Gemini for Arabic) can produce high-quality, platform-adapted marketing content at zero and low inference cost respectively. The three-stage image generation progression — from LoRA fine-tuning on a subset, to full-dataset training, to a production API pipeline — illustrated a realistic engineering path from experimentation to deployment.

By integrating campaign generation, image creation, intelligent chatbot assistance, posting schedule recommendations, and campaign library management into one unified platform, **AD GENERATOR AI** provides a comprehensive solution that addresses real challenges faced by small businesses in the digital marketing space.

7.1.1 Project Achievements Summary

Objective	Achievement
AI text & hashtag generation	T5-Small (English) + Gemini API (Arabic) pipelines fully implemented and integrated

Platform-specific content	Instagram · TikTok · Facebook · Website/Ads — each with adapted hashtag counts, tone, and style
Bilingual support	English and Arabic with فصحى enforcement, gender-neutral forms, and English-only hashtags
Three post variants per campaign	Product-Focused · Lifestyle-Focused · Campaign-Focused generated for every campaign
AI image generation	Three-stage pipeline: LoRA 500-sample → full-dataset fine-tune → Gemini via OpenRouter API
Posting schedule recommendations	Platform-specific Priority 1, 2, 3 optimal posting times with tips displayed post-generation
AI marketing chatbot	5-node LangGraph agentic RAG system with FAISS knowledge base and Tavily web search fallback
Multi-product campaigns	Multiple products bundled in a single campaign and AI generation request
Campaign library	Full save, retrieve, filter, and delete functionality for all generated campaigns
Backend & API	ASP.NET Core 8 Web API with JWT auth, Cloudinary storage, and MonsterASP.NET deployment
Mobile application	Flutter Android app with full UI/UX, multi-step wizard, and Google Sign-In support

7.2 Limitations

Despite the achievements of the project, several limitations were identified during development and testing:

- The LoRA fine-tuned image generation models (Stages 1 and 2) are limited to a native resolution of 512×512 pixels and were primarily validated on footwear product photography.
- The production image generation pipeline (Stage 3) depends on the availability of the OpenRouter and Gemini external APIs, introducing uptime and cost dependencies not present in the self-hosted stages.
- The posting schedule recommendations currently display optimal times and tips to the user but do not integrate with social media platform APIs to auto-publish posts directly at those times.
- The T5-Small English model does not support Arabic and requires a completely separate pipeline for bilingual output.
- The AI marketing chatbot does not currently support uploading files or documents — users can only ask questions via text input.
- The chatbot knowledge base quality is bounded by the marketing books currently ingested; more specialized topics rely on the Tavily web search fallback.
- The current implementation targets Android only; iOS support requires additional platform configuration and an Apple Developer account.
- Campaign performance analytics (tracking real post engagement after publishing) are not yet implemented.

7.3 Future Work

The following enhancements and extensions are planned to further improve the capabilities, performance, and reach of the **AD GENERATOR AI** platform. These improvements are organized by system area:

7.3.1 Campaign Features & Scheduling Enhancements

Enhancement	Description
Direct social media auto-publishing	Extend the existing posting schedule recommendations feature to allow users to directly schedule and publish generated posts to Instagram, TikTok, and Facebook at the AI-recommended optimal times — eliminating the current manual copy-paste step.
Campaign performance analytics	Integrate with social media platform APIs (Instagram Graph API, Facebook API, TikTok API) to track real engagement metrics (likes, shares, reach, impressions) per generated campaign after publishing.
Campaign A/B testing	Generate two caption variants for the same post and track which performs better to enable data-driven campaign optimization based on real audience response.
Product catalog import	Allow users to import product lists from Shopify, WooCommerce, or a CSV file instead of entering product details manually one by one in the campaign wizard.

7.3.2 AI Marketing Chatbot Improvements

Enhancement	Description
Marketing file upload support for the chatbot	Allow users to upload marketing-related documents (PDF, Word, or text files) directly into the chatbot interface — such as a business plan, marketing strategy, product catalog, or campaign report — so the chatbot can read and answer questions specifically about that file. Uploaded files would be validated to ensure they contain marketing-relevant content before being processed and added to the RAG retrieval context for that session.
Image upload support for the chatbot	Allow users to upload product images or existing advertisement creatives to the chatbot and receive AI-powered visual marketing analysis, improvement suggestions, or campaign ideas based on the uploaded visuals.
Expand the knowledge base	Ingest additional marketing books, industry case studies, platform advertising guides, and research papers to improve answer quality and reduce reliance on web search fallback.
Voice input	Allow users to submit marketing questions via voice input instead of typing, improving accessibility and convenience on mobile devices.
Persistent conversation memory	Implement conversation history that persists across sessions so the chatbot can reference prior interactions for more contextual and personalized marketing advice.
Chatbot-to-campaign integration	Allow users to directly trigger campaign generation from the chatbot — for example, by saying 'Generate a campaign for this product' — connecting the chatbot assistant with the campaign creation module.

7.3.3 AI & Image Generation Improvements

Enhancement	Description
Upgrade to SDXL or newer diffusion model	Replace Stable Diffusion v1.5 with SDXL or a more recent backbone to overcome the current 512×512 resolution ceiling and produce higher-quality promotional images natively.
ControlNet-based product conditioning	Add precise control over product placement, pose, and background composition in generated images using ControlNet adapters for more consistent and professional results.
Short-form video ad generation	Integrate Stable Video Diffusion or a similar model to generate short video advertisements suitable for TikTok Reels and Instagram Stories.
Dedicated Arabic language model fine-tuning	Fine-tune a purpose-built Arabic NLP model on domain-specific fashion marketing data instead of relying solely on Gemini API prompt engineering for Arabic generation.
Personalized generation models	Train user-adaptive models that adjust generation style, tone, and content based on each user's campaign history and feedback over time.

7.3.4 Mobile Application & Platform Expansion

Enhancement	Description
iOS Support	Extend the Flutter application to support iOS devices, requiring Apple Developer account registration and minor platform-specific UI adjustments — Flutter's cross-platform nature makes this a straightforward next step.
Push Notifications	Deliver real-time alerts for campaign generation completion, posting schedule reminders, and chatbot replies to improve user engagement and campaign management.
Web Application	Build a responsive web version of the platform to serve users who prefer desktop access for campaign creation and management, leveraging the existing ASP.NET Core backend.
Offline Mode	Allow users to browse and review previously saved campaigns without requiring an active internet connection, using local caching in the Flutter application.

7.3.5 Security, Administration & Infrastructure

Enhancement	Description
Admin dashboard	Build a web-based administration panel for managing users, monitoring campaign generation volume, tracking AI API usage and costs, and maintaining overall system health. All required backend endpoints are already implemented.
Activate Admin role	The Admin role is already prepared in the ASP.NET Core Identity configuration. Activating it will enable supervised access control, user management, and knowledge base administration from a dedicated interface.
Audit logging	Record all significant user and system actions (campaign creation, deletion, file uploads, API calls) for transparency, traceability, and compliance purposes.
Scalable cloud migration	Migrate from MonsterASP.NET to Azure or AWS for improved uptime guarantees, auto-scaling capabilities, and global content delivery.

Async processing queue	Introduce a task queue (e.g., Celery with RabbitMQ) to handle heavy AI generation requests asynchronously, preventing API timeouts and improving responsiveness under load.

7.3.6 Localization & Accessibility

Enhancement	Description
Additional language support	Extend campaign generation to support French, Turkish, and other regional languages beyond the current English and Arabic output modes.
Accessibility features	Add screen reader compatibility, high-contrast color modes, and keyboard navigation to make the application accessible to users with disabilities.
Refined RTL interface	Improve Arabic right-to-left UI rendering in the Flutter application to ensure a fully native Arabic user experience across all screens.

Chapter 8: Appendix

Appendix

This appendix presents supporting screenshots and implementation artifacts from the **AD GENERATOR AI** system, covering the AI model training process, dataset structure, generated output samples, evaluation results, mobile application implementation, database schema, and the retrieval-augmented generation (RAG) and image generation training pipelines. Each figure is preceded by a short description of its content and purpose.

A.1 T5 Model Training Setup and Execution

The following screenshot shows the Hugging Face Trainer configuration used to fine-tune the T5 model on the marketing text generation task, including training arguments such as batch size, learning rate, evaluation strategy, and the early stopping callback.



Figure A.1: Trainer and TrainingArguments configuration for the T5 fine-tuning process.

This screenshot shows the training execution log after calling trainer.train(), including the step-by-step training and validation loss values recorded during the fine-tuning process, along with the final completion summary.



Figure A.2: Training execution log showing training/validation loss per step and completion summary.

Before training begins, the base T5-Small model and tokenizer are loaded from Hugging Face. This screenshot shows the model loading log, confirming that the model was successfully placed on the GPU (CUDA) with a total of 60,506,624 trainable parameters.

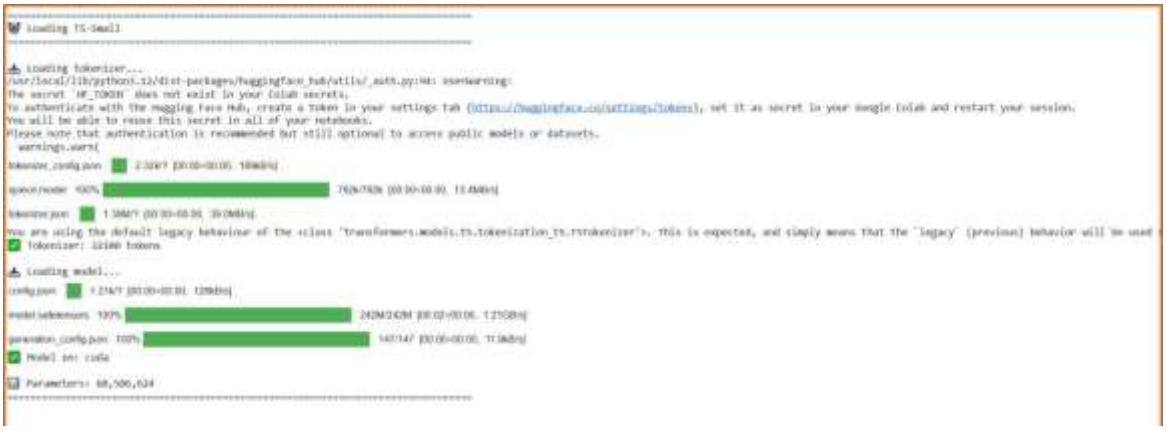


Figure A.3: T5-Small tokenizer and model loading log, confirming GPU allocation and parameter count.

This screenshot shows the dataset preparation step, where the training and validation splits (135,000 and 15,000 examples respectively) are generated and tokenized in preparation for training.



Figure A.4: Dataset loading and tokenization log showing training and validation split sizes.

To evaluate whether the model was overfitting, the training and validation loss curves were plotted together with a log-scale overfitting gap check. The negative gap value indicates that the model was not overfitting on the training data.

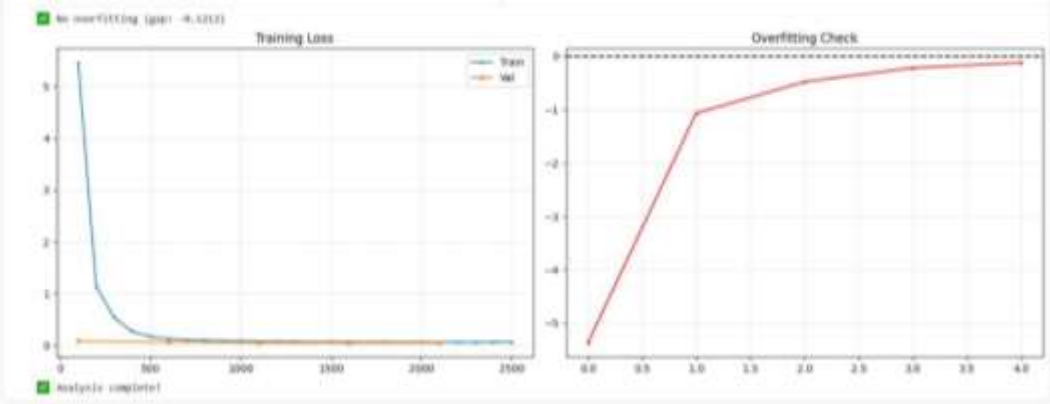


Figure A.5: Training loss curve and overfitting gap check for the fine-tuned T5 model.

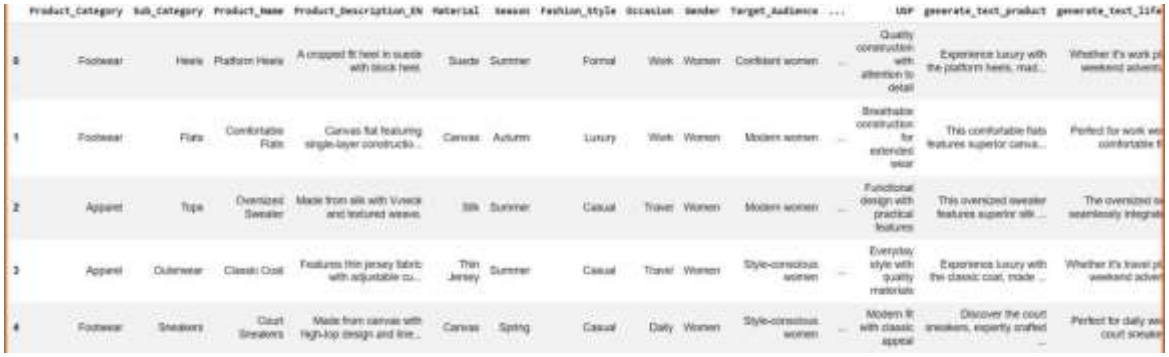
After training completed, a final evaluation report was generated summarizing the evaluation loss, runtime, and the deliverables produced by the training pipeline, including the saved model, tokenizer, inference script, and API contract.



Figure A.6: Final evaluation report and list of training pipeline deliverables.

A.2 Dataset Structure

The training dataset consists of structured product records describing fashion items (category, material, season, style, occasion, target audience, etc.). This screenshot shows a preview of the first columns of the dataset used to train the text generation model.



	Product_Category	Sub_Category	Product_Name	Product_Description_EN	Material	Season	Fashion_Style	Occasion	Gender	Target_Audience	...	USP	generate_text_product	generate_text_life
0	Footwear	Heels	Platform Heels	A cropped fit heel in suede with block heel.	Suede	Summer	Formal	Work	Women	Confident women	...	Quality construction with attention to detail	Experience luxury with the platform heels, mad...	Whether it's work pl weekend advent
1	Footwear	Flats	Comfortable Flats	Canvas flat featuring single-layer constructio...	Canvas	Autumn	Luxury	Work	Women	Modern women	...	Breathable construction for extended wear	This comfortable flats features superior canva...	Perfect for work we comfortable f
2	Apparel	Tops	Oversized Sweater	Made from silk with V-neck and textured weave.	Silk	Summer	Casual	Travel	Women	Modern women	...	Functional design with practical features	This oversized sweater features superior silk ...	The oversized s seamlessly integra
3	Apparel	Outerwear	Classic Coat	Features thin jersey fabric with adjustable cu...	Thin Jersey	Summer	Casual	Travel	Women	Style-conscious women	...	Everyday style with quality materials	Experience luxury with the classic coat, made ...	Whether it's travel pl weekend adver
4	Footwear	Sneakers	Court Sneakers	Made from canvas with high-top design and line...	Canvas	Spring	Casual	Daily	Women	Style-conscious women	...	Modern fit with classic appeal	Discover the court sneakers, expertly crafted ...	Perfect for daily we court sneake

Figure A.7: Preview of the product-attribute columns in the training dataset.

In addition to product attributes, each record includes the target marketing texts and hashtags generated for the product, lifestyle, and campaign categories. This screenshot shows these additional dataset columns used as training labels.



	generate_text_lifestyle	generate_text_campaign	generate_hashtags_product	generate_hashtags_lifestyle	generate_hashtags_campaign	full_output_product	full_output_lifestyle	full_output_campaign
	Whether it's work plans or weekend adventures,...	Going viral on TikTok right now: the platform's ...	#Top #Viral #Fashion #Justforthewin #Turnup	#Tonyou #Summer #Ootd #Dailylook #Trending	#Bookclubbehindyou #Bringcrew #Virtualhave #Viral #...	Experience luxury with the platform heels, mad...	Whether it's work plans or weekend adventures,...	Going viral on TikTok right now: the platform's ...
	Perfect for work wear, the comfortable flats f...	Your next insta-worthy moment deserves the com...	#Fashion #style #currentclothesflats #currvue #fla...	#lifestyle #work #autumn #dailywear #luxuryst...	#shoprow #trending #mustshave #sale #shoptoday #...	This comfortable flats features superior canva...	Perfect for work wear, the comfortable flats f...	Your next insta-worthy moment deserves the com...
	The oversized sweater seamlessly integrates in...	Elevate your Instagram aesthetic with the river...	#Fashion #style #oversizedsweater #ootd #ootn...	#lifestyle #travel #summer #dailywear #casuals...	#shoprow #trending #mustshave #sale #shoptoday #h...	This oversized sweater features superior silk ...	The oversized sweater seamlessly integrates in...	Elevate your Instagram aesthetic with the river...
	Whether it's travel plans or weekend adventure...	Turn heads on Instagram with the stunning clas...	#fashion #style #classiccoat #thingsmary #ootd...	#lifestyle #travel #summer #dailywear #casuals...	#shoprow #trending #mustshave #sale #shoptoday #h...	Experience luxury with the classic coat, made ...	Whether it's travel plans or weekend adventure...	Turn heads on Instagram with the stunning clas...
	Perfect for daily wear, the court sneakers fit...	Elevate your Instagram aesthetic with the court...	#fashion #style #courtsneakers #currvue #sneaker...	#lifestyle #daily #spring #dailywear #casualst...	#shoprow #trending #mustshave #sale #shoptoday #h...	Discover the court sneakers, expertly crafted ...	Perfect for daily wear, the court sneakers fit...	Elevate your Instagram aesthetic with the court...

Figure A.8: Preview of the generated-text and hashtag columns used as training labels.

A.3 Generated Output Variants and Evaluation

Once trained, the model was tested to generate three distinct content variants (Product-Focused, Lifestyle-Focused, and Campaign-Focused) for a given product, each with its own text and a set of twenty hashtags.

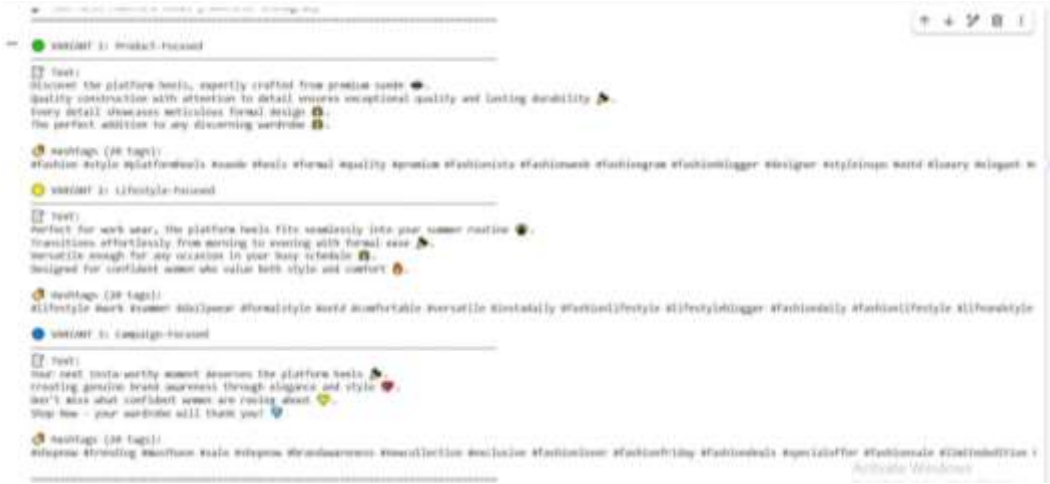


Figure A.9: Code cell showing the three generated content variants with their hashtags.

This screenshot shows a full test case for a specific product (Platform Heels) generated for the Instagram platform, displaying the three output variants side by side for qualitative review.

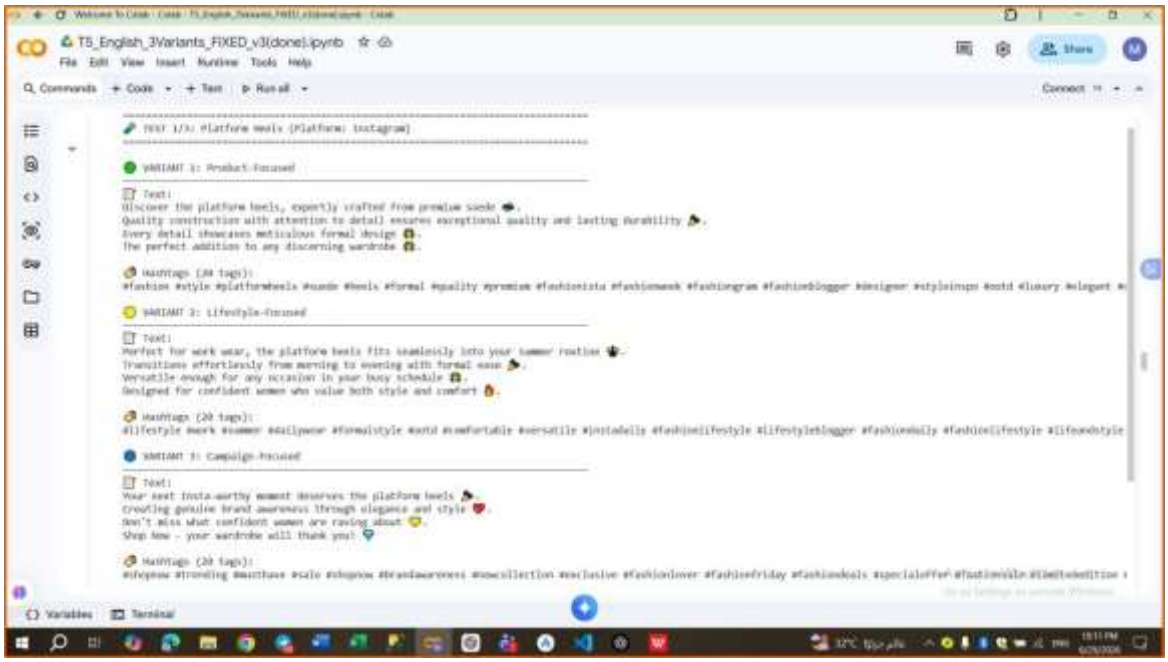


Figure A.10: Sample test case output for the Instagram platform (Platform Heels product).

To assess output quality automatically, each generated test case was scored on a 5-point scale. This screenshot shows the automated preliminary scores across the first eight test cases, with quality labels ranging from "Needs improvement" to "Excellent quality."



Figure A.11: Automated preliminary quality scores for the first eight generated test cases.

A.4 API Structured Output and Platform Configuration

To ensure the generated content follows a consistent, machine-readable format, Pydantic models were defined for the API layer. This screenshot shows the PostBlock and PlatformOutput classes used to structure the Arabic text and English hashtags returned by the API.



Figure A.12: Pydantic structured-output classes (PostBlock, PlatformOutput) used by the generation API.

Each supported platform (Instagram, TikTok, Facebook, Website) has its own content profile controlling hashtag count, tone, hook style, selling angle, CTA style, and text length. This screenshot shows the PLATFORM_PROFILES configuration dictionary.



Figure A.13: PLATFORM_PROFILES configuration defining tone and formatting rules per platform.

This screenshot shows a final sample of Arabic marketing content generated by the system for the Instagram platform, covering the Product, Lifestyle, and Campaign post types for a linen summer dress, together with their English hashtags.



Figure A.14: Final generated Arabic campaign content for Instagram (Product, Lifestyle, and Campaign posts).

A.5 Mobile Application Implementation (Flutter)

Authentication state in the mobile application is handled using the BLoC pattern. This screenshot shows the AuthPage widget, which conditionally displays either the home page or the authentication form based on the current authentication state.

Auth State Management

```
class AuthPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: BlocBuilder<AuthBloc, AuthState>(
        builder: (context, state) {
          return state is Authenticated ? HomePage() : AuthForm();
        },
      ),
    );
  }
}
```

Figure A.15: AuthPage widget managing authentication state with BLoC.

Campaign creation data is collected and managed through a dedicated controller class. This screenshot shows the CampaignCreationController, which stores the campaign name, selected platforms, and tone before converting them into a Campaign entity.

Campaign Data Controller

```
class CampaignCreationController {
  String campaignName = "";
  List<String> selectedPlatforms = [];
  String tone = "";

  Campaign toEntity() {
    return Campaign(name: campaignName, platforms: selectedPlatforms);
  }
}
```

Figure A.16: CampaignCreationController used to collect campaign creation data.

Once a campaign is generated, its results are displayed to the user through a dedicated results page. This screenshot shows the CampaignResultPage widget, which renders each generated variant using a PostCard component in a scrollable list.

Displaying AI Results

```
class CampaignResultPage extends StatelessWidget {
  final Campaign campaign;

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: campaign.variants.length,
      itemBuilder: (context, index) => PostCard(variant: campaign.variants[index]),
    );
  }
}
```

Figure A.17: CampaignResultPage widget displaying generated campaign variants.

The application also includes a chatbot interface for conversational interaction. This screenshot shows the ChatbotPage widget, composed of a scrollable message list and a chat input field connected to the ChatBloc.

Chat Interface Layout

```
class ChatbotPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Expanded(child: MessageList()),
        ChatInputField(onSend: (msg) => context.read<ChatBloc>().add(SendMessage(msg))),
      ],
    );
  }
}
```

Figure A.18: ChatbotPage widget implementing the chat interface layout.

Previously generated campaigns are saved and displayed in a dedicated library screen. This screenshot shows the LibraryPage widget, which renders the user's saved campaigns in a grid using the LibraryCubit state.

Campaign Library Grid

```
class LibraryPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return BlocBuilder<LibraryCubit, LibraryState>(
      builder: (context, state) {
        return CampaignGrid(campaigns: state.savedCampaigns);
      },
    );
  }
}
```

Figure A.19: LibraryPage widget displaying saved campaigns in a grid layout.

User account information and settings are presented through a profile screen. This screenshot shows the ProfilePage widget, which displays the current user's header information along with a list of profile settings options.

User Profile View

```
class ProfilePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        UserHeader(user: currentUser),
        SettingsList(items: profileOptions),
      ],
    );
  }
}
```

Figure A.20: ProfilePage widget displaying user information and settings.

A.6 Database Schema (SQL Server)

Product images are stored in a dedicated table linked to each product. This screenshot shows the ProductImages table schema, storing the image URL, the related product ID, and the creation timestamp.

DESKTOP-RAT...roductImages

Column Name	Data Type	Allow Nulls
Id	int	☐
ImageURL	varchar(500)	☐
ProductId	int	☐
CreatedAt	datetime2(7)	☐

Figure A.21: ProductImages table schema.

Each marketing campaign created by a user is stored in the Campaigns table. This screenshot shows its schema, including the campaign name, platform, language, campaign goal, and the linked user ID.

DESKTOP-RAT...bo.Campaigns

Column Name	Data Type	Allow Nulls
Id	int	☐
CampaignName	nvarchar(100)	☐
EndDate	datetime2(7)	☑
Platform	nvarchar(20)	☐
CampaignGoal	nvarchar(50)	☐
UserId	nvarchar(450)	☐
GeneratedAt	datetime2(7)	☐

Figure A.22: Campaigns table schema.

This screenshot shows the Entity Framework Core migration code that creates the Campaigns table, including its columns, primary key, and foreign key relationships to the Products and Users tables.



Figure A.23: Migration code generating the Campaigns table with its foreign key constraints.

To support campaigns that reference multiple products, a many-to-many relationship is implemented through a junction table. This screenshot shows the migration code for the CampaignProducts table, along with its foreign keys to Campaigns and Products.



Figure A.24: Migration code generating the CampaignProducts junction table.

This screenshot shows the resulting schema of the CampaignProducts table as viewed in SQL Server, confirming the columns defined by the migration.

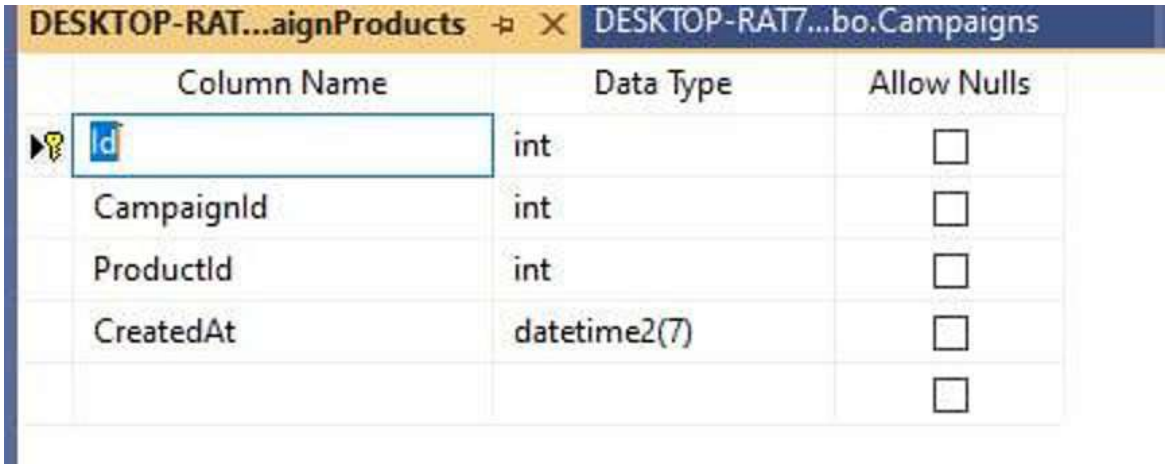


Figure A.25: CampaignProducts table schema as shown in SQL Server.

Generated AI content (captions, hashtags, tone, and image prompts) is stored per campaign variant. This screenshot shows the migration code for the AIGeneratedContents table, which links back to the Campaigns table.

```
migrationBuilder.CreateTable(
    name: "AIGeneratedContents",
    columns: table => new
    {
        Id = table.Column<int>(type: "int", nullable: false)
            .Annotation("SqlServer:Identity", "1, 1"),
        IdealHashtags = table.Column<string>(type: "nvarchar(MAX)", nullable: true),
        IdealCaption = table.Column<string>(type: "nvarchar(MAX)", nullable: true),
        Season = table.Column<string>(type: "nvarchar(30)", maxLength: 30, nullable: false),
        Tone = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable: true),
        Language = table.Column<string>(type: "nvarchar(20)", maxLength: 20, nullable: false),
        VariantNumber = table.Column<int>(type: "int", nullable: false),
        ImagePrompt = table.Column<string>(type: "nvarchar(MAX)", nullable: false),
        CampaignId = table.Column<int>(type: "int", nullable: false),
        GeneratedAt = table.Column<DateTime>(type: "datetime2", nullable: false, defaultValueSql: "GETUTCDATE()"
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_AIGeneratedContents", x => x.Id);
        table.ForeignKey(
            name: "FK_AIGeneratedContents_Campaigns_CampaignId",
            column: x => x.CampaignId,
            principalTable: "Campaigns",
            principalColumn: "Id",
            onDelete: ReferentialAction.Cascade);
    });
```

Figure A.26: Migration code generating the AIGeneratedContents table.

This screenshot shows the resulting AIGeneratedContents table schema, including fields for the ideal hashtags, ideal caption, season, tone, language, variant number, and image prompt.

DESKTOP-RAT...atedContents

Column Name	Data Type	Allow Nulls
Id	int	☐
IdealHashtags	nvarchar(MAX)	☑
IdealCaption	nvarchar(MAX)	☑
Season	nvarchar(30)	☐
Tone	nvarchar(100)	☑
Language	nvarchar(20)	☐
VariantNumber	int	☐
ImagePrompt	nvarchar(MAX)	☐
CampaignId	int	☐
GeneratedAt	datetime2(7)	☐
ProductId	int	☐

Figure A.27: AIGeneratedContents table schema.

Generated images associated with each piece of AI content are stored separately. This screenshot shows the migration code for the AIGeneratedContentImages table, linked to the AIGeneratedContents table via a foreign key.

```
migrationBuilder.CreateTable(
    name: "AIGeneratedContentImages",
    columns: table => new
    {
        Id = table.Column<int>(type: "int", nullable: false)
            .Annotation("SqlServer:Identity", "1, 1"),
        GeneratedImageURL = table.Column<string>(type: "varchar(500)", maxLength: 500, nullable: false),
        ContentId = table.Column<int>(type: "int", nullable: false),
        CreatedAt = table.Column<DateTime>(type: "datetime2", nullable: false, defaultValueSql: "GETUTCDATE()"
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_AIGeneratedContentImages", x => x.Id);
        table.ForeignKey(
            name: "FK_AIGeneratedContentImages_AIGeneratedContents_ContentId",
            column: x => x.ContentId,
            principalTable: "AIGeneratedContents",
            principalColumn: "Id",
            onDelete: ReferentialAction.Cascade);
    });
```

Figure A.28: Migration code generating the AIGeneratedContentImages table.

This screenshot shows the resulting AIGeneratedContentImages table schema, storing the generated image URL, the related content ID, and the creation timestamp.

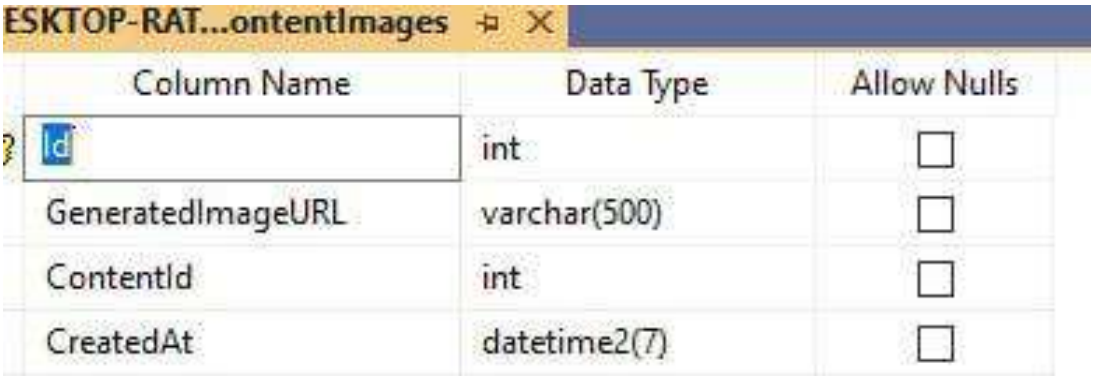


Figure A.29: AIGeneratedContentImages table schema.

A.7 Retrieval-Augmented Generation (RAG) Pipeline

To improve the relevance of generated content, a RAG pipeline was implemented using LangGraph-style agent nodes. This screenshot shows the rag_node function, which retrieves relevant context chunks from a FAISS vector store, and the judge_node function, which scores how well the retrieved context answers the query before deciding whether to fall back to a web search.



```
app > rag_chain.py > rag_node
def rag_node(state: AgentState) -> AgentState:
    logger.info("[agent] rag_node")
    steps  = list(state.get("steps_taken", []))
    vs: FAISS = state["vectorstore"]

    enriched = _enrich_query(
        state["question"],
        state.get("category", "general"),
        [],
    )

    docs, avg_score = _rag_retrieve(vs, enriched)
    steps.append(f"rag → {len(docs)} chunks | avg_relevance={avg_score}")
    return {**state, "retrieved_docs": docs, "context": _fmt_context(docs), "steps_taken": steps}


def judge_node(state: AgentState) -> AgentState:
    logger.info("[agent] judge_node")
    steps = list(state.get("steps_taken", []))

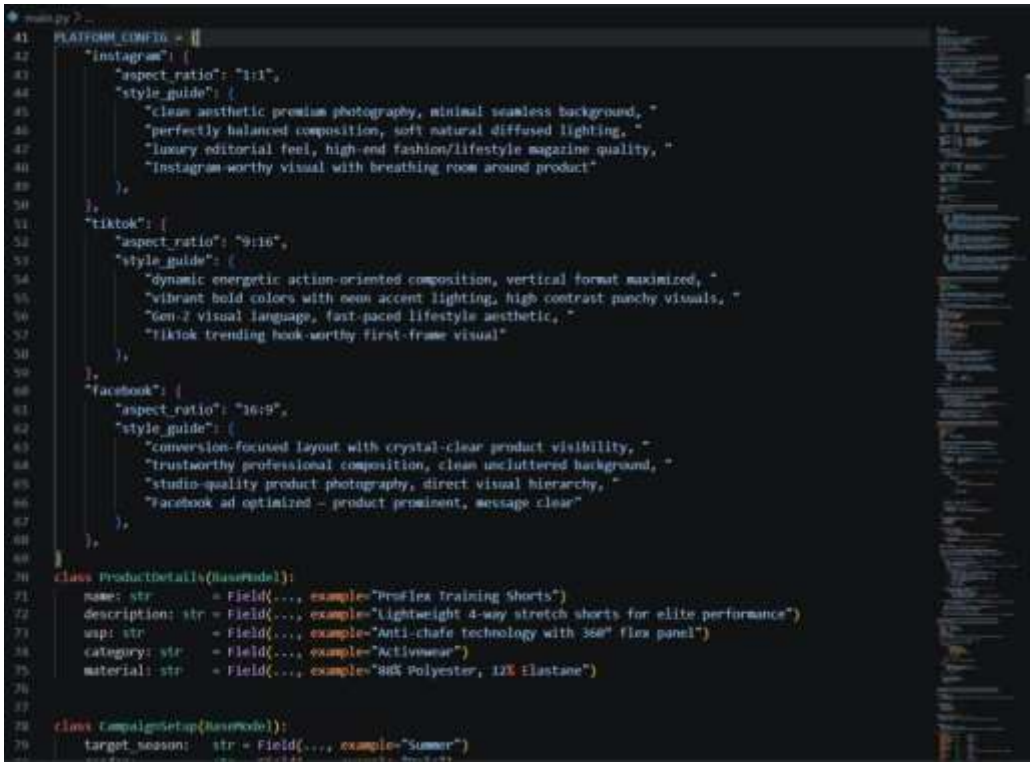
    verdict: JudgeDecision = _judge_llm.invoke([
        ("system", (
            "Score how well the retrieved context answers the question.\n"
            "score: 0.0 = completely irrelevant, 0.5 = partial answer, 1.0 = fully answers it.\n"
            "Be strict: only score ≥ 0.75 if the context directly and completely addresses the question.\n"
            "JSON: {\"score\": float, \"reason\": \"one sentence\"}"
        )),
        ("user", f"Q: {state['question']}\n\nContext:\n{state.get('context','') or '(empty)'}"),
    ])

    sufficient = verdict.score >= JUDGE_THRESHOLD
    label = f"score={verdict.score:.2f} ({'✓ sufficient' if sufficient else '✗ → web'})"
    steps.append(f"judge → {label} | {verdict.reason[:60]}")
    return {**state, "sufficient": sufficient, "confidence": verdict.score, "steps_taken": steps}


def web_node(state: AgentState) -> AgentState:
    logger.info("[agent] web_node")
    steps = list(state.get("steps_taken", []))
```

Figure A.30: rag_node and judge_node functions implementing context retrieval and relevance scoring.

Each supported platform also has its own visual style guide for AI image generation. This screenshot shows the PLATFORM_CONFIG dictionary, defining the aspect ratio and style guide for Instagram, TikTok, and Facebook, along with the ProductDetails and CampaignSetup input models.



```
PLATFORM_CONFIG = {
    "instagram": {
        "aspect_ratio": "1:1",
        "style_guide": (
            "clean aesthetic premium photography, minimal seamless background, "
            "perfectly balanced composition, soft natural diffused lighting, "
            "luxury editorial feel, high-end fashion/lifestyle magazine quality, "
            "Instagram-worthy visual with breathing room around product"
        ),
    },
    "tiktok": {
        "aspect_ratio": "9:16",
        "style_guide": (
            "dynamic energetic action-oriented composition, vertical format maximized, "
            "vibrant bold colors with neon accent lighting, high contrast punchy visuals, "
            "Gen-Z visual language, fast-paced lifestyle aesthetic, "
            "TikTok trending hook-worthy first-frame visual"
        ),
    },
    "facebook": {
        "aspect_ratio": "16:9",
        "style_guide": (
            "conversion-focused layout with crystal-clear product visibility, "
            "trustworthy professional composition, clean uncluttered background, "
            "studio-quality product photography, direct visual hierarchy, "
            "Facebook ad optimized — product prominent, message clear"
        ),
    },
}

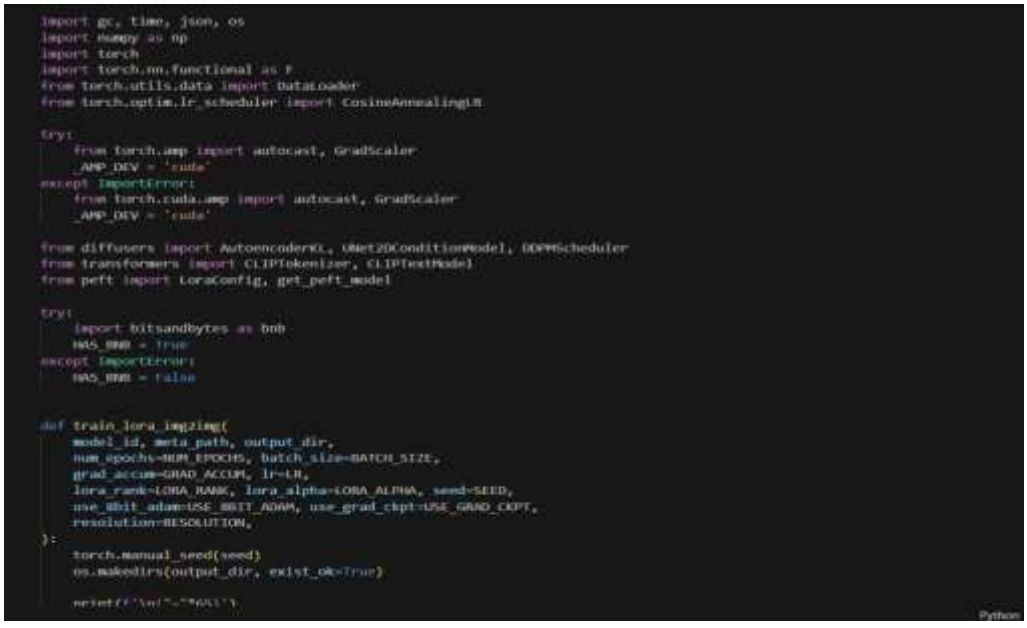
class ProductDetails(BaseModel):
    name: str        = Field(..., example="ProFlex Training Shorts")
    description: str = Field(..., example="Lightweight 4-way stretch shorts for elite performance")
    usp: str         = Field(..., example="Anti-chafe technology with 360° flex panel")
    category: str    = Field(..., example="Activewear")
    material: str    = Field(..., example="88% Polyester, 12% Elastane")


class CampaignSetup(BaseModel):
    target_season:   str = Field(..., example="Summer")
```

Figure A.31: PLATFORM_CONFIG dictionary and input models defining platform-specific image generation styles.

A.8 LoRA Image Generation Training Script

For product image generation, a LoRA (Low-Rank Adaptation) fine-tuning approach was used on top of a diffusion model. This screenshot shows the train_lora_img2img function signature, along with the diffusion, transformer, and PEFT library imports required for training.



```
import gc, time, json, os
import numpy as np
import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torch.optim.lr_scheduler import CosineAnnealingLR

try:
    from torch.amp import autocast, GradScaler
    _AMP_DEV = 'cuda'
except ImportError:
    from torch.cuda.amp import autocast, GradScaler
    _AMP_DEV = 'cuda'

from diffusers import AutoencoderKL, UNet2DConditionModel, DDPMScheduler
from transformers import CLIPTokenizer, CLIPTextModel
from peft import LoraConfig, get_peft_model

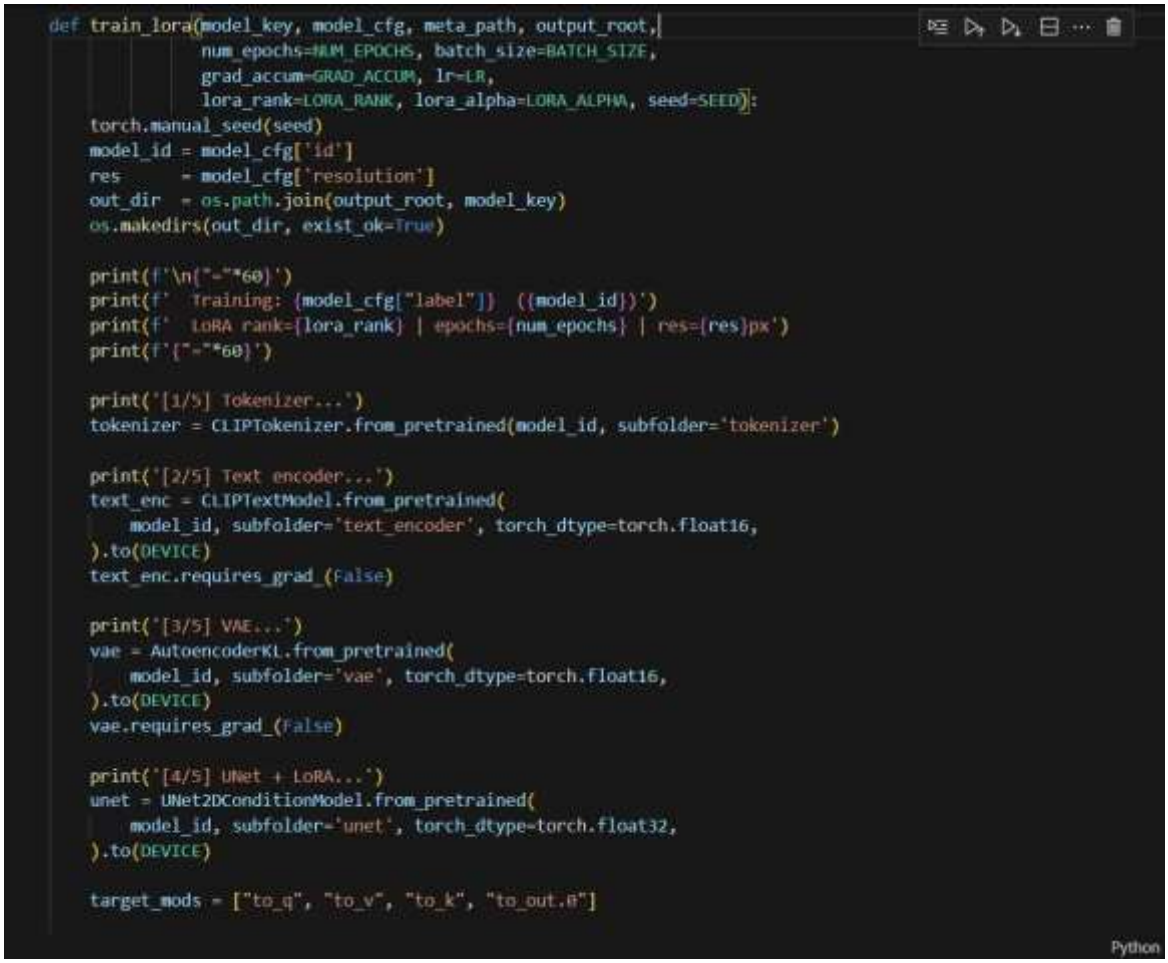
try:
    import bitsandbytes as bnb
    HAS_BNB = True
except ImportError:
    HAS_BNB = False


def train_lora_img2img(
    model_id, meta_path, output_dir,
    num_epochs=NUM_EPOCHS, batch_size=BATCH_SIZE,
    grad_accum=GRAD_ACCUM, lr=LR,
    lora_rank=LORA_RANK, lora_alpha=LORA_ALPHA, seed=SEED,
    use_8bit_adam=USE_8BIT_ADAM, use_grad_ckpt=USE_GRAD_CKPT,
    resolution=RESOLUTION,
):
    torch.manual_seed(seed)
    os.makedirs(output_dir, exist_ok=True)

    print(f"\n{'='*60}")
```

Figure A.32: train_lora_img2img function and required library imports for LoRA-based image model training.

This screenshot shows the train_lora function, which loads the tokenizer, text encoder, VAE, and UNet components of the diffusion model and prepares the UNet's attention layers (to_q, to_v, to_k, to_out.0) for LoRA fine-tuning.



```python
def train_lora(model_key, model_cfg, meta_path, output_root,
               num_epochs=NUM_EPOCHS, batch_size=BATCH_SIZE,
               grad_accum=GRAD_ACCUM, lr=LR,
               lora_rank=LORA_RANK, lora_alpha=LORA_ALPHA, seed=SEED):
    torch.manual_seed(seed)
    model_id = model_cfg['id']
    res      = model_cfg['resolution']
    out_dir  = os.path.join(output_root, model_key)
    os.makedirs(out_dir, exist_ok=True)

    print(f'\n{"="*60}')
    print(f'  Training: {model_cfg["label"]}  ({model_id})')
    print(f'  LoRA rank={lora_rank} | epochs={num_epochs} | res={res}px')
    print(f'{"="*60}')

    print('[1/5] Tokenizer...')
    tokenizer = CLIPTokenizer.from_pretrained(model_id, subfolder='tokenizer')

    print('[2/5] Text encoder...')
    text_enc = CLIPTextModel.from_pretrained(
        model_id, subfolder='text_encoder', torch_dtype=torch.float16,
    ).to(DEVICE)
    text_enc.requires_grad_(False)

    print('[3/5] VAE...')
    vae = AutoencoderKL.from_pretrained(
        model_id, subfolder='vae', torch_dtype=torch.float16,
    ).to(DEVICE)
    vae.requires_grad_(False)

    print('[4/5] UNet + LoRA...')
    unet = UNet2DConditionModel.from_pretrained(
        model_id, subfolder='unet', torch_dtype=torch.float32,
    ).to(DEVICE)

    target_mods = ["to_q", "to_v", "to_k", "to_out.0"]
```

Python

Figure A.33: train_lora function loading model components and configuring LoRA target modules.

References

(References List)

1 **Google Developers**, "Flutter Documentation: Build apps for any screen," [Online]. Available: https://docs.flutter.dev. (Used for Mobile App Development).

2 **Microsoft**, "Entity Framework Core Overview," [Online]. Available: https://learn.microsoft.com/en-us/ef/core. (Used for Database ORM).

3 **Microsoft**, "SQL Server Documentation," [Online]. Available: https://learn.microsoft.com/en-us/sql/sql-server. (Used for Data Management).

4 **Alibaba Cloud**, "Qwen2.5: A Party of Foundation Models," [Online]. Available: https://github.com/QwenLM/Qwen2.5. (Used for Arabic Content Generation).

5 **Hugging Face**, "Transformers Library Documentation," [Online]. Available: https://huggingface.co/docs/transformers. (Used for LLM Implementation).

6 **Meta AI**, "FAISS: A Library for Efficient Similarity Search," [Online]. Available: https://github.com/facebookresearch/faiss. (Used for RAG Pipeline).

7 **LangChain**, "LangGraph: Building Language Agents," [Online]. Available: https://python.langchain.com/docs/langgraph. (Used for Agentic Workflow).

8 **Diederik P. Kingma and Jimmy Ba**, "Adam: A Method for Stochastic Optimization," *arXiv preprint arXiv:1412.6980*, 2014. (Used for LoRA Training Optimization).

9 **Edward J. Hu et al.**, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint arXiv:2106.09685*, 2021. (Used for Image Generation Fine-tuning).

10 **OpenAI**, "ChatGPT: Optimizing Language Models for Dialogue," [Online]. Available: https://openai.com/blog/chatgpt. (Referenced for Chatbot Assistant concepts).

(Tools & Libraries)

11 **Dart Programming Language**, "Dart Language Tour," [Online]. Available: https://dart.dev/guides/language/language-tour.

12 **NVIDIA**, "CUDA Toolkit Documentation," [Online]. Available: https://docs.nvidia.com/cuda. (Used for GPU Acceleration in Colab).

13 **JSON.org**, "Introducing JSON," [Online]. Available: https://www.json.org/json-en.html. (Used for Structured Data Exchange).